UNIVERSIDADE DO ESTADO DO AMAZONAS
ESCOLA SUPERIOR DE TECNOLOGIA
Engenharia de Computação
Disciplina: Processamento Digital de Imagens

Relatório Técnico – Atividade 3

Análise de desempenho de CPU e GPU utilizando OpenGL: quantidade de polígonos, iluminação, texturas e o limite de 60 FPS

Henrique da Silva Furtado

Relatório técnico apresentado à disciplina de Processamento Digital de Imagens, contendo a aplicação gráfica em C++/OpenGL/FreeGLUT, a metodologia e os resultados das medições de desempenho, a investigação dos gargalos e o código-fonte completo em anexo.

Professor(a): Ricardo da Silva Barboza

Manaus-AM
9 de novembro de 2026

Sumário

1 Objetivos **1**

2 Introdução teórica **1**

2.1 Quadros por segundo e tempo por quadro 1

2.2 O pipeline gráfico: o papel da CPU e da GPU 2

2.3 Gargalos 2

2.4 Iluminação 3

2.5 Texturas 3

2.6 Modos de envio da geometria 3

3 Ambiente de desenvolvimento **4**

4 Caracterização do hardware **4**

4.1 Qual GPU executa a aplicação? 5

5 Descrição da cena e da estratégia de polígonos **7**

5.1 A cena “Jardim de Orbes” 7

5.2 Estratégia de aumento dos polígonos 9

5.3 Critério de contagem dos polígonos 9

6 Metodologia de coleta e medição **10**

6.1 Cálculo do FPS e dos tempos 10

6.2 Utilização de CPU, GPU e memória 10

6.3 Protocolo experimental 11

6.4 Experimentos realizados 11

7 Implementação das funcionalidades gráficas **12**

7.1 Organização do código 12

7.2 Geometria 12

7.3 Iluminação e texturas com GLSL 12

7.4 Comandos de teclado e reprodução dos experimentos 13

8 Resultados do experimento principal (E1) **13**

8.1 Execução da coleta 13

8.2 FPS em função da quantidade de triângulos 14

8.3 Tabela-síntese 16

8.4 Utilização de CPU e GPU 17

9 Investigação dos gargalos **18**
9.1 Decomposição do tempo do quadro 18
9.2 Teste das hipóteses: levando o gargalo para a CPU (E2 e E3) 20
9.3 Mesma aplicação, GPU integrada (E4) 22
9.4 Síntese dos regimes 23

10 Influência da iluminação e das texturas **24**

11 Conclusão **25**

Apêndice — Todos os pontos medidos **26**

Referências bibliográficas **30**

Anexo — Código-Fonte **32**

Lista de Figuras

1 Gerenciador de Tarefas do Windows aos 50 s de uma medição na configuração F com $T = 576$ (84,8 milhões de triângulos), com as duas GPUs acompanhadas separadamente. 6

2 Janela da aplicação aos 50,5 s da mesma medição (configuração F, $T = 576$), com o painel de medições em tempo real. 7

3 As seis configurações experimentais no mesmo instante da animação ($T = 48$, $N = 64$, 594 960 triângulos por quadro). O painel superior mostra o FPS e as medições em tempo real. 8

4 Desenho em arame (tecla `w`) mostrando o aumento do detalhe T dos orbes. 9

5 FPS médio × triângulos por quadro na configuração A (E1). O marcador vazado indica a menor carga testada com FPS médio abaixo de 60. 14

6 FPS médio × triângulos para os três modos de iluminação sem textura (A, B e C). 15

7 Efeito das texturas: cada painel compara a mesma iluminação sem e com textura (A×D, B×E, C×F). 15

8 As seis configurações. À direita, a região próxima de 60 FPS em escala linear, com barras de ±1 desvio-padrão das 3 repetições (menores que os marcadores). 16

9 Utilização da CPU × triângulos (configurações A e F). “Thread mais ocupada” e “thread principal” em % de um núcleo; “processo” em % da CPU inteira (22 threads lógicas), como no Gerenciador de Tarefas. À direita, o mesmo para o modo imediato (E2). 17

10 Utilização do motor 3D da GPU pelo processo (contadores PDH) × triângulos. À esquerda, a NVIDIA no E1 (e a Intel, que só compõe a tela); à direita, a GPU que desenha em cada um dos demais experimentos. 17

11 Potência e clock dos multiprocessadores da GPU NVIDIA no E1 (NVML). Os “×” marcam os pontos em que o NVML informou redução de clock por limite de potência. 18

12 Decomposição do tempo por quadro nos quatro experimentos (configuração A). A linha tracejada vermelha é o orçamento de 16,7 ms de um quadro a 60 FPS. 19

13 E2: modo imediato (`glBegin`/`glEnd`), seis configurações. 20

14 E3: aumento do número de orbes N (cada um uma chamada de desenho), $T = 8$. O eixo superior mostra N. 21

15 E4: a mesma varredura do E1 executada na GPU integrada Intel Arc. . . . 22

16 Configuração A nos quatro experimentos: a mesma cena atinge 60 FPS com 0,5 a 82 milhões de triângulos, conforme onde está o gargalo. 23

Lista de Tabelas

1 Ferramentas e bibliotecas utilizadas. 4
2 Caracterização do hardware e do software de sistema. 5
3 Experimentos. Em todos: janela 1280×720, V-Sync desligado, 3 repetições de 10 s. 11
4 Arquivos do projeto (código completo no Anexo). 12
5 Comandos de teclado da aplicação. 13
6 Tabela-síntese do experimento principal E1 (GPU dedicada, VBO, aumento de T): menor quantidade testada de triângulos com FPS médio abaixo de 60 em cada configuração. 16
7 Tabela-síntese do experimento E2 (modo imediato, GPU dedicada). 21
8 Tabela-síntese do experimento E3 (aumento do número de orbes, T = 8). . 22
9 Tabela-síntese do experimento E4 (GPU integrada Intel Arc, VBO, aumento de T). 23
10 Gargalo identificado em cada experimento, no ponto do limite de 60 FPS (configuração A). 24
11 Variação do FPS em relação à configuração A no E1 (positivo = mais rápido que A). 24
12 Todos os pontos medidos no experimento E1 (médias de 3 repetições de 10 s). "r" = ponto de refino; ▶ = primeiro ponto com FPS médio < 60. . . . 26
13 Pontos medidos no experimento E2 (modo imediato). 27
14 Pontos medidos no experimento E3 (N orbes com T = 8). 28
15 Pontos medidos no experimento E4 (GPU integrada). Potência e clock da NVML não se aplicam. 29

1 Objetivos

Objetivo geral. Investigar experimentalmente o desempenho de uma aplicação gráfica em C++ com OpenGL e FreeGLUT, analisando a influência da quantidade de polígonos, da iluminação e das texturas sobre a taxa de quadros por segundo (FPS) e sobre a utilização da CPU e da GPU, e determinar, para cada uma das seis configurações gráficas exigidas, a quantidade de polígonos a partir da qual o FPS *médio* fica abaixo de 60.

Objetivos específicos.

- Caracterizar o hardware e identificar qual GPU executa a aplicação OpenGL num notebook com GPU integrada (Intel Arc) e dedicada (NVIDIA GeForce RTX 4060 Laptop).
- Desenvolver uma única aplicação com cena original e animação contínua, com controle da quantidade de polígonos, iluminação omnidirecional e *spot*, texturas e contagem exata dos triângulos renderizados por quadro.
- Medir FPS, tempo por quadro, tempo de GPU, tempo de CPU e utilização de CPU, GPU, memória, potência e clock, com protocolo repetível (3 repetições de 10 s após estabilização, V-Sync desligado).
- Localizar, por varredura progressiva e refino por bisseção, a menor quantidade testada de triângulos com FPS médio inferior a 60 nas configurações A a F.
- Identificar o gargalo (*bottleneck*) de cada situação, distinguindo conclusões sustentadas pelos dados de hipóteses, e avaliar o efeito da iluminação e das texturas.

2 Introdução teórica

2.1 Quadros por segundo e tempo por quadro

Uma aplicação gráfica interativa produz uma sequência de imagens (*quadros*). A **taxa de quadros** é $\text{FPS} = n/\Delta t$, o número de quadros n concluídos num intervalo Δt. A grandeza equivalente e mais útil para análise é o **tempo por quadro** $t_q = 1000/\text{FPS}$ (ms), porque os custos das etapas se *somam* em tempo, e não em FPS. O limite de 60 FPS corresponde a um orçamento de $1000/60 \approx 16{,}7$ ms por quadro, o valor típico da taxa de atualização de monitores e uma referência comum de fluidez [1]. Como t_q e FPS são inversos, uma mesma economia de tempo (por exemplo, 1 ms) vale muito mais FPS quando o FPS é alto do que quando é baixo; por isso as médias deste trabalho são calculadas sobre quadros contados num intervalo longo (10 s), e não sobre valores instantâneos.

O **V-Sync** (sincronismo vertical) faz a troca de *buffers* esperar o retraço do monitor, limitando o FPS à frequência de atualização (60 ou 120 Hz). Uma leitura de 60 FPS com V-Sync ligado não revela a capacidade excedente; por isso os testes desligam o V-Sync (`wglSwapIntervalEXT(0)`).

2.2 O pipeline gráfico: o papel da CPU e da GPU

No OpenGL, a aplicação, executada na **CPU**, prepara o estado (matrizes, texturas, programas de *shader*) e emite comandos de desenho. O *driver* traduz esses comandos e os coloca numa fila consumida pela **GPU**, que executa o *pipeline* gráfico [1, 2]:

1. **processamento de vértices**: o *vertex shader* transforma cada vértice (matrizes de modelagem, visualização e projeção) e calcula atributos;
2. **montagem e rasterização**: os triângulos são montados, recortados, descartados se estiverem de costas (*back-face culling*) e convertidos em fragmentos (pixels);
3. **processamento de fragmentos**: o *fragment shader* calcula a cor de cada fragmento (iluminação, textura), seguido do teste de profundidade e da escrita no *framebuffer*.

CPU e GPU trabalham em paralelo, como uma linha de produção: enquanto a GPU desenha o quadro k, a CPU já prepara o quadro $k+1$. Por isso o tempo por quadro é aproximadamente o **máximo** (e não a soma) entre o tempo de CPU e o tempo de GPU de cada quadro. A etapa mais lenta determina o FPS: é o **gargalo** (*bottleneck*) [1, 9].

2.3 Gargalos

- **Gargalo de CPU**: a GPU fica ociosa esperando comandos. Ocorre quando o custo de *emitir* o trabalho (chamadas de função, validação de estado pelo *driver*, uma chamada de desenho por objeto) é maior que o custo de *executá-lo*. Como a emissão de comandos OpenGL de um contexto é essencialmente sequencial, o gargalo de CPU costuma ser **uma única thread saturada**, com utilização total da CPU baixa: em um processador com 22 threads lógicas, uma thread a 100% corresponde a apenas $100/22 \approx 4{,}5\%$ da CPU inteira.
- **Gargalo de GPU**: a CPU espera a GPU (tipicamente bloqueada na troca de *buffers*). Pode estar no processamento de vértices (muitos vértices), na rasterização (muitos triângulos minúsculos), nos fragmentos (muitos pixels, *shaders* caros, texturas) ou na largura de banda de memória.
- **Outros limites**: potência e temperatura (a GPU reduz o clock ao atingir o limite de potência ou térmico), sincronização (V-Sync, esperas do *driver*), transferência de dados entre CPU e GPU e, em notebooks com duas GPUs, a cópia do quadro da GPU dedicada para a integrada, que é a ligada à tela.

Utilização próxima de 100% é um *indício*, não uma prova: uma thread pode aparecer ocupada porque está em espera ativa (*busy-wait*) pela GPU, e uma GPU pode estar "100% ocupada" apenas no sentido de ter sempre algum trabalho na fila. Este trabalho, por isso, cruza três tipos de evidência: (i) utilização dos recursos, (ii) decomposição do tempo do quadro (tempo de GPU medido pela própria GPU *versus* tempo efetivo de CPU) e (iii) a forma como o FPS escala quando se muda a carga de um recurso específico.

2.4 Iluminação

O modelo de reflexão usado é o de **Blinn-Phong** [7, 6], calculado *por fragmento*:

$$\mathbf{c} = \mathbf{k}_d \odot (\mathbf{a} + f_{at}\, f_{spot}\, \max(\mathbf{n}\cdot\mathbf{l}, 0)\, \mathbf{i}_L) + k_s\, f_{at}\, f_{spot}\, \max(\mathbf{n}\cdot\mathbf{h}, 0)^{\alpha}\, \mathbf{i}_L, \qquad \mathbf{h} = \frac{\mathbf{l}+\mathbf{v}}{\|\mathbf{l}+\mathbf{v}\|}, \quad (1)$$

em que $\mathbf{k}_d$ é a cor difusa do material (modulada pela textura), $\mathbf{a}$ a luz ambiente, $\mathbf{n}$ a normal, $\mathbf{l}$ a direção para a luz, $\mathbf{v}$ a direção para o observador, $\alpha = 48$ o expoente especular e $\mathbf{i}_L$ a cor da luz. A atenuação com a distância d é $f_{at} = 1/(1 + 0{,}010\, d + 0{,}002\, d^2)$.

- **Luz omnidirecional** (*point light*): emite igualmente em todas as direções a partir de um ponto; $f_{spot} = 1$.
- **Luz spot** (*spotlight*): emite dentro de um cone de eixo $\mathbf{s}$. Com θ o ângulo entre $-\mathbf{l}$ e $\mathbf{s}$, $f_{spot} = \text{smoothstep}(\cos\theta_{ext}, \cos\theta_{int}, \cos\theta)$, com $\theta_{int} = 18°$ e $\theta_{ext} = 26°$ (borda suave).

Calcular a iluminação por fragmento custa operações proporcionais ao número de pixels cobertos; no OpenGL de *pipeline* fixo ela seria calculada por vértice (modelo de Gouraud), o que torna o *spot* visualmente pobre em superfícies com poucos vértices.

2.5 Texturas

Uma textura é uma imagem amostrada a cada fragmento por coordenadas (u, v). Para evitar *aliasing* ao reduzir a imagem, usam-se **mipmaps** (versões reduzidas pré-calculadas) com filtragem **trilinear** (interpolação entre 8 texels de dois níveis) e, aqui, também **anisotrópica** 8x, que amostra mais texels quando a superfície é vista em ângulo rasante [1, 8]. O custo de uma textura está nos acessos à memória por fragmento; a cache de texturas da GPU torna esse custo baixo quando os acessos são coerentes.

2.6 Modos de envio da geometria

- **Modo imediato** (`glBegin`/`glVertex`/`glEnd`): cada atributo de cada vértice é uma chamada de função, a cada quadro. Todo o trabalho de enviar a geometria recai sobre a CPU (e o barramento). Foi removido do perfil *core* do OpenGL 3.1.
- **Vertex Buffer Objects** (VBO): vértices e índices ficam na memória de vídeo e um único `glDrawElements` desenha milhares ou milhões de triângulos [3].

3 Ambiente de desenvolvimento

Tabela 1: Ferramentas e bibliotecas utilizadas.

Item	Versão / observação
Linguagem	C++17
Compilador	Microsoft Visual C++ 14.44 (Visual Studio 2022), x64, Release (`/O2`)
Sistema de compilação	CMake 3.31 (gerador "Visual Studio 17 2022"), script `build.bat`
Janelas e entrada	FreeGLUT 3.8.0
API gráfica	OpenGL 4.6 em perfil de compatibilidade; *shaders* GLSL 1.20
Extensões OpenGL	carregador próprio (`gl_ext.cpp`) via `glutGetProcAddress`, sem GLEW/GLAD
Monitoramento	API Win32 (PDH, Toolhelp32, DXGI) e NVML (NVIDIA Management Library), carregada dinamicamente; Gerenciador de Tarefas para conferência visual
Capturas de tela	OpenCV 4.14.0 (opcional, só para gravar PNG)
Gráficos e tabelas	Python 3.14 com matplotlib 3.11 (`tools/graficos.py`)
Relatório	LaTeX (MiKTeX, `pdflatex`)

Todo o projeto é um único CMake com os mesmos arquivos-fonte, compilado em dois executáveis que diferem só numa definição de compilação: `desempenho_gl` exporta o símbolo `NvOptimusEnablement`, que pede ao *driver* que crie o contexto OpenGL na GPU NVIDIA dedicada (usado em todos os experimentos principais), e `desempenho_gl_integrada` não o exporta, deixando o Windows escolher a GPU (no equipamento usado, a integrada). Isso permitiu comparar as duas GPUs sem alterar o código.

4 Caracterização do hardware

Os dados da Tabela 2 foram obtidos pela própria aplicação (opção `-info`, que consulta o registro do Windows, DXGI, NVML e `glGetString`) e conferidos no Gerenciador de Tarefas e com `nvidia-smi`.

Tabela 2: Caracterização do hardware e do software de sistema.

Componente	Descrição
Equipamento	Notebook Dell Inspiron 16 Plus 7640, ligado à tomada (bateria 100%), plano de energia "Equilibrado"
CPU	Intel Core Ultra 7 155H (Meteor Lake): 16 núcleos (6 de desempenho, 8 de eficiência, 2 de baixo consumo) e 22 threads lógicas; até 4,8 GHz
Memória RAM	32 GB (2 × 16 GB) DDR5-5600 (31,5 GB visíveis)
GPU dedicada	NVIDIA GeForce RTX 4060 Laptop GPU (Ada Lovelace), 8 GB GDDR6 dedicados (7 956 MB informados pelo DXGI), *driver* 621.01, limite de potência observado ≈ 60 W
GPU integrada	Intel Arc Graphics (Xe-LPG, 8 núcleos Xe), 128 MB dedicados + memória compartilhada da RAM (até 18 GB), *driver* 32.0.101.8860
Tela	2560 × 1600, 120 Hz, ligada à GPU integrada (arquitetura NVIDIA Optimus)
Sistema operacional	Windows 11 Pro 25H2 (build 26200), 64 bits
OpenGL (GPU dedicada)	`GL_VERSION` = 4.6.0 NVIDIA 621.01; `GL_RENDERER` = NVIDIA GeForce RTX 4060 Laptop GPU/PCIe/SSE2; GLSL 4.60
OpenGL (GPU integrada)	`GL_VERSION` = 4.6.0 - Build 32.0.101.8860; `GL_RENDERER` = Intel(R) Arc(TM) Graphics
Recursos usados	consultas de tempo da GPU (`GL_TIME_ELAPSED`), filtragem anisotrópica, controle do V-Sync (`WGL_EXT_swap_control`)

4.1 Qual GPU executa a aplicação?

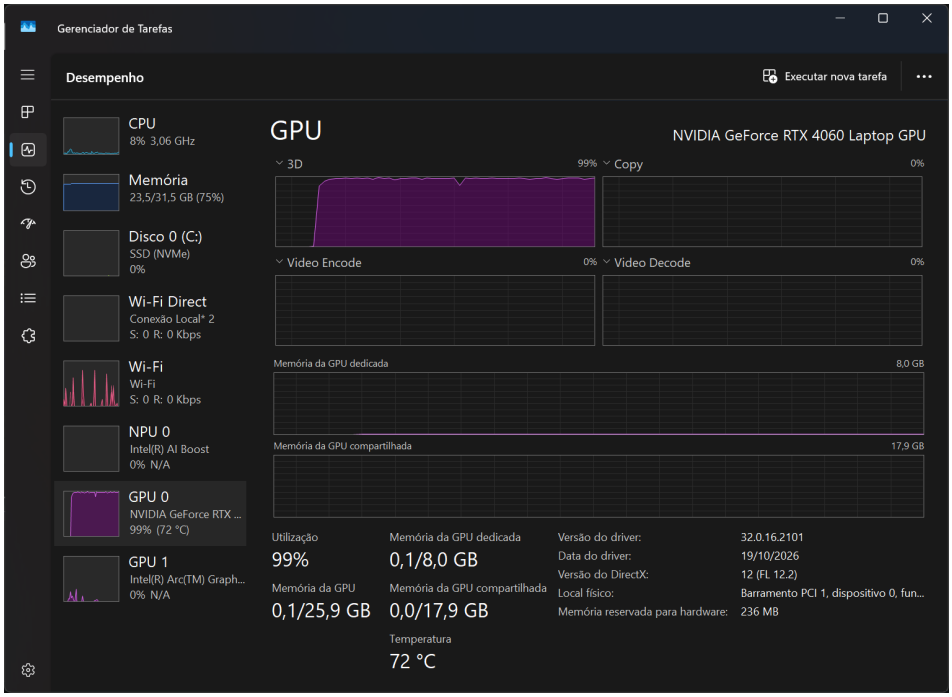
O equipamento possui duas GPUs. A tela está ligada à GPU integrada; a GPU NVIDIA renderiza e copia o quadro pronto para a integrada, que o compõe na área de trabalho (Optimus). Três evidências independentes identificam a GPU que executa a aplicação:

1. `glGetString(GL_RENDERER)` informa o dispositivo que atende ao contexto OpenGL: "NVIDIA GeForce RTX 4060 Laptop GPU" no executável principal e "Intel(R) Arc(TM) Graphics" no executável `_integrada`;
2. os contadores de desempenho do Windows `\GPU Engine(*)\Utilization Percentage`, a mesma fonte do Gerenciador de Tarefas, trazem no nome de cada instância o *pid* do processo, o LUID do adaptador e o tipo de motor (3D, cópia, vídeo...). A aplicação filtra as instâncias pelo seu próprio *pid* e associa o LUID ao nome do adaptador via DXGI. Durante os testes com o executável principal, o motor 3D do processo esteve ativo apenas na NVIDIA (0% de 3D na Intel);
3. a NVML mostra potência, clock e utilização da NVIDIA subindo com a carga.

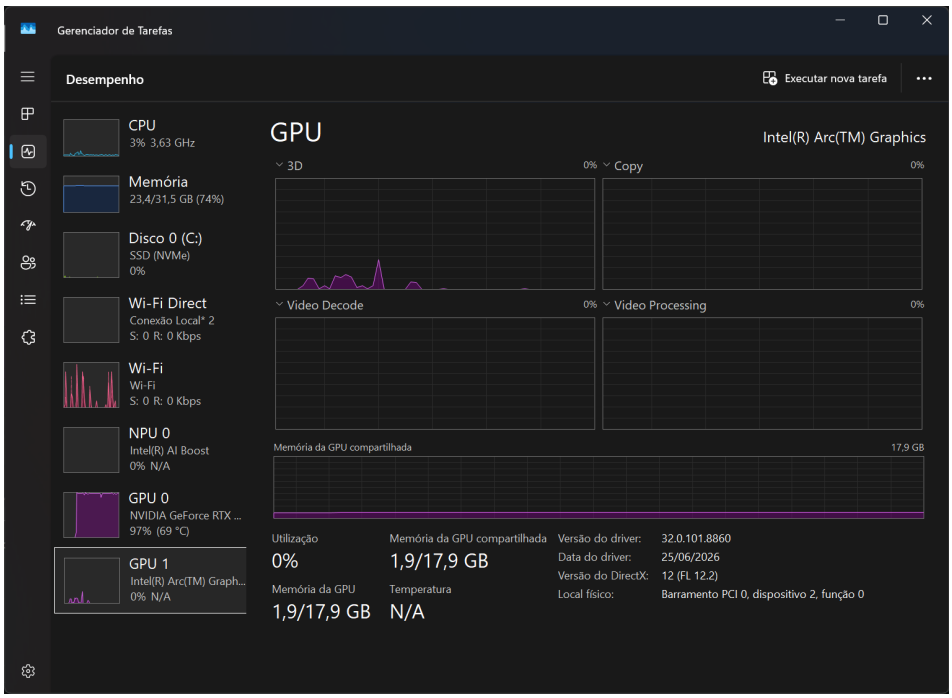
Sim, é possível acompanhar separadamente o percentual de cada GPU: o Gerenciador de Tarefas mostra "GPU 0" (NVIDIA) e "GPU 1" (Intel) e a aplicação registra ambas em todas as medições (colunas `gpu_nv_*` e `gpu_intel_*` dos CSV).

As capturas da Figura 1 foram feitas aos 50 s de uma medição de 60 s na configuração F com $T = 576$ (84,8 milhões de triângulos, logo acima do limite de 60 FPS; FPS médio de 57,4 nessa medição), uma para cada GPU. O motor 3D da NVIDIA está a 97–99%, com

72 °C, enquanto o motor 3D da Intel fica em 0%: a GPU integrada apenas exibe o quadro pronto. Note que a CPU inteira está a 3–8%, coerente com o gargalo de GPU discutido na Seção 9. A versão de *driver* 32.0.16.2101 informada pelo Windows corresponde à versão 621.01 da NVIDIA.



(a) GPU 0: NVIDIA GeForce RTX 4060 Laptop (3D a 99%)



(b) GPU 1: Intel Arc Graphics (3D a 0%)

Figura 1: Gerenciador de Tarefas do Windows aos 50 s de uma medição na configuração F com $T = 576$ (84,8 milhões de triângulos), com as duas GPUs acompanhadas separadamente.

A Figura 2 mostra o painel da própria aplicação no mesmo instante (50,5 s da medição).

As medições internas concordam com o Gerenciador de Tarefas: motor 3D do processo a 95% na NVIDIA e 0% na Intel, NVML com 100% de utilização, 60 W (o limite de potência) e clock reduzido a 2 280 MHz, CPU do processo a 0,8%. A decomposição do quadro mostra o gargalo diretamente: dos 17,43 ms do quadro, a GPU trabalha 17,40 ms, enquanto a thread principal executa só 0,16 ms de CPU. O tempo de emissão (17,25 ms) é quase todo *espera* dentro do *driver*, que bloqueia a thread quando a fila de comandos para a GPU está cheia; por isso a thread aparece com apenas 3% de uso apesar de passar o quadro inteiro "dentro" da emissão.

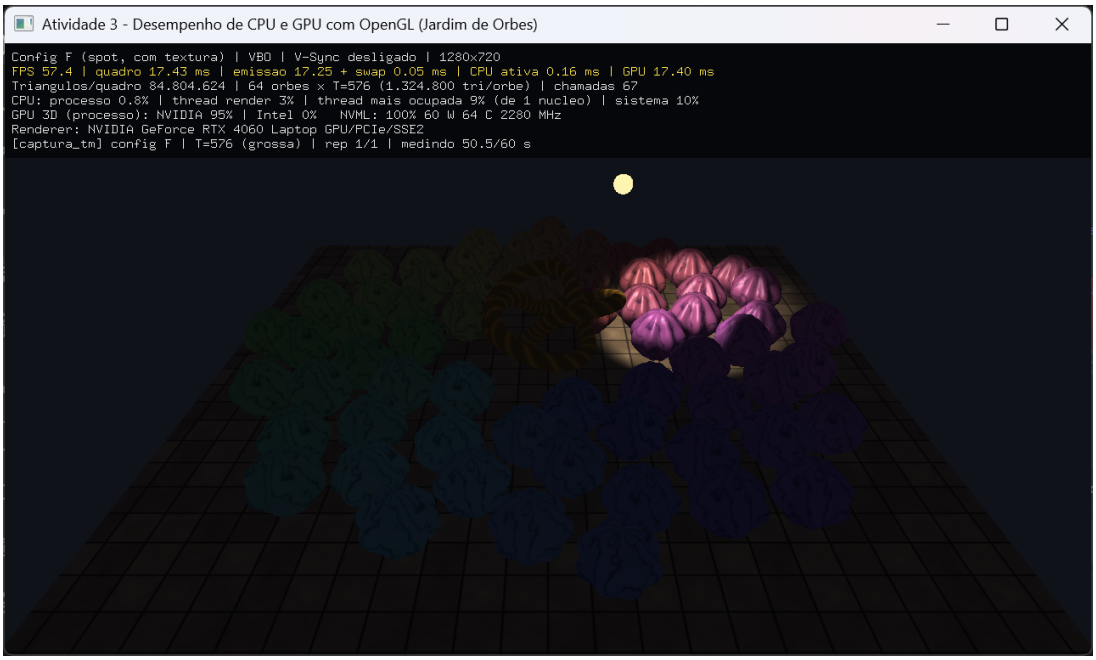


Figura 2: Janela da aplicação aos 50,5 s da mesma medição (configuração F, $T = 576$), com o painel de medições em tempo real.

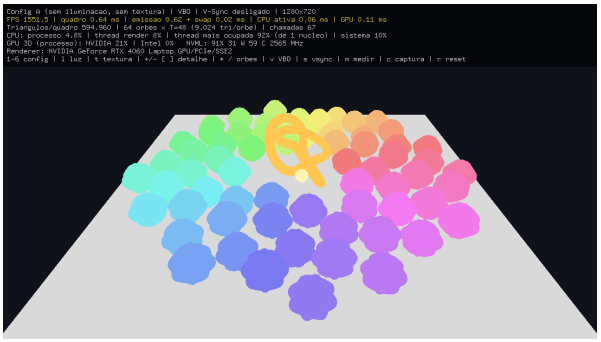
5 Descrição da cena e da estratégia de polígonos

5.1 A cena "Jardim de Orbes"

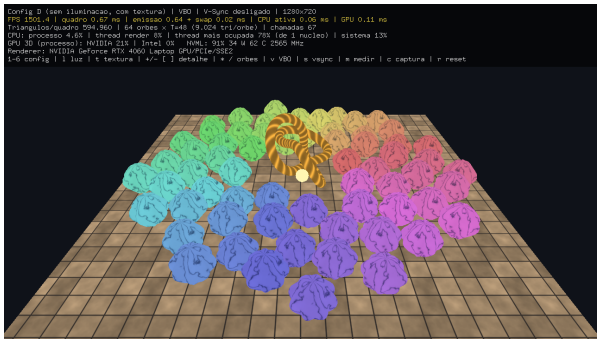
A cena, de autoria própria, é composta por (Figura 3):

- um **piso** de ladrilhos (48×48 quadrados, 4 608 triângulos);
- um **nó toroidal** dourado do tipo $(2, 3)$ no centro, com seção circular (256×24 quadriláteros, 12 288 triângulos), que **gira continuamente** em dois eixos e flutua;
- N **orbes** de mármore (esferas com relevo em gomos) dispostos em espiral filotáxica — o padrão das sementes de girassol, com ângulo de ouro $\pi(3 - \sqrt{5}) \approx 137{,}5°$ entre orbes consecutivos e raio $\propto \sqrt{i}$ — que giram em carrossel em torno do nó, cada um girando sobre si e oscilando na vertical;
- uma **lâmpada**, pequena esfera emissiva que marca a posição da fonte de luz (528 triângulos), desenhada em todas as configurações.

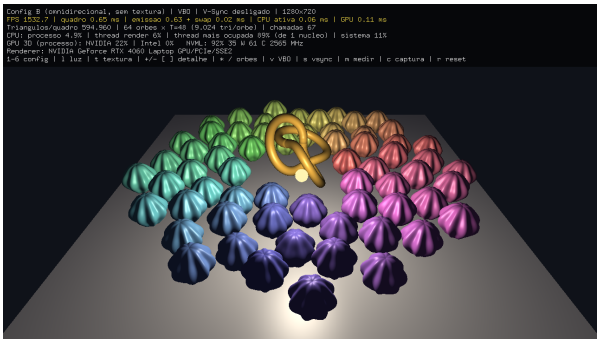
A câmera é fixa, a janela tem sempre 1280×720 pixels e a animação depende só do tempo real, de modo que todas as configurações mostram a mesma cena. A luz omnidirecional orbita o centro a 4,5 unidades de altura; o *spot* fica a 7 unidades de altura e varre o piso com seu cone.



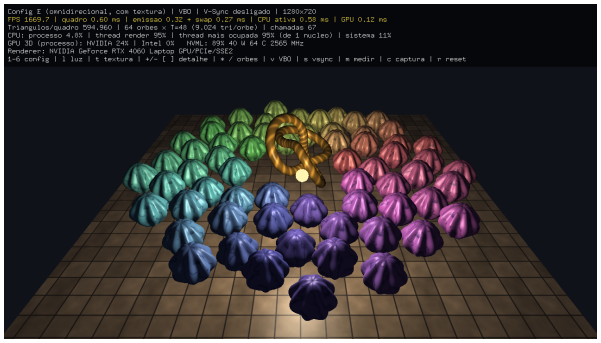
(a) A: sem luz, sem textura



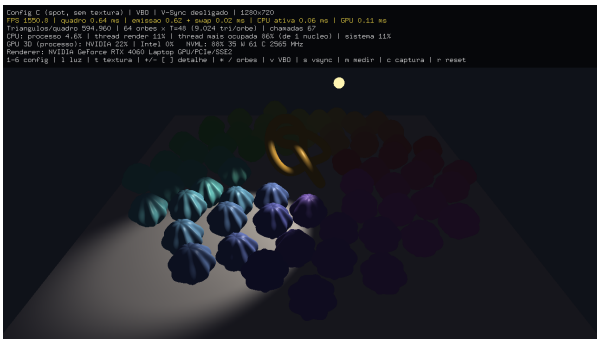
(b) D: sem luz, com textura



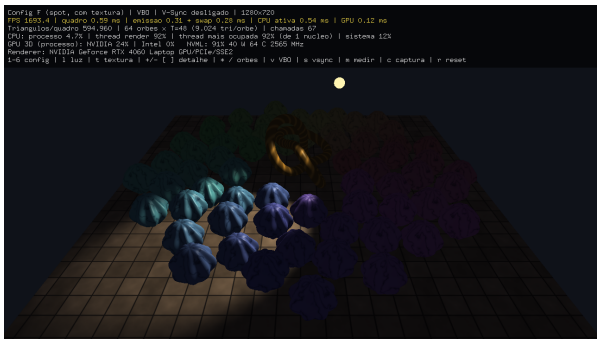
(c) B: omnidirecional, sem textura



(d) E: omnidirecional, com textura



(e) C: spot, sem textura



(f) F: spot, com textura

Figura 3: As seis configurações experimentais no mesmo instante da animação ($T = 48$, $N = 64$, 594 960 triângulos por quadro). O painel superior mostra o FPS e as medições em tempo real.

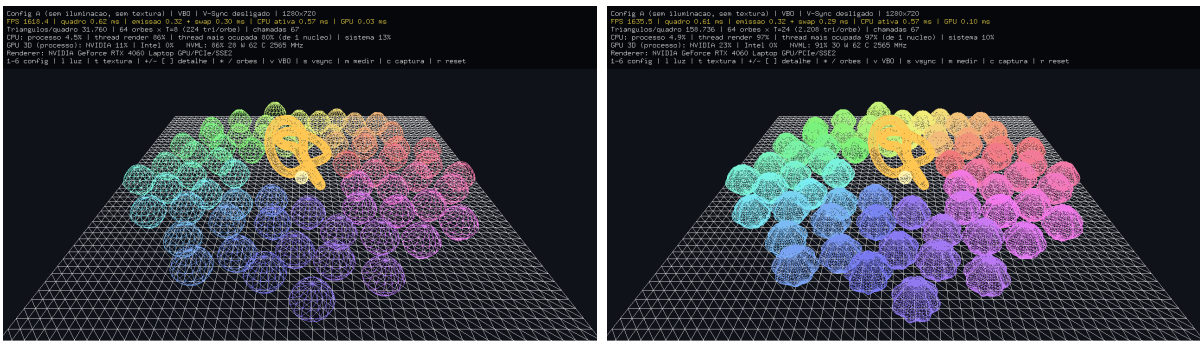
A Figura 3 confirma que a iluminação produz resultado verificável: em A e D os objetos têm cor uniforme (sem sombreamento); em B e E a luz omnidirecional (esfera clara à frente do nó) produz sombreamento difuso, reflexos especulares e atenuação com a distância; em C e F apenas a região dentro do cone do *spot* é iluminada, com borda suave. As texturas (D, E, F) mostram os ladrilhos do piso, os veios de mármore dos orbes e as faixas trançadas do nó.

5.2 Estratégia de aumento dos polígonos

A carga geométrica é controlada por dois parâmetros independentes:

- T, o **nível de detalhe** de cada orbe: o orbe é uma superfície paramétrica com T faixas de latitude e $2T$ de longitude. Aumentar T aumenta os triângulos de cada objeto sem mudar o número de objetos, de chamadas de desenho ou de pixels cobertos;
- N, o **número de orbes**: aumentar N aumenta os triângulos e também o número de chamadas de desenho (uma por orbe). Para manter a área ocupada na tela aproximadamente constante, o raio de cada orbe diminui com $\sqrt{N}$.

O experimento principal (E1) varia T com $N = 64$ fixo, isolando o efeito da quantidade de polígonos. O experimento E3 varia N com $T = 8$, para observar o efeito do número de objetos.



(a) $T = 8$: 224 triângulos por orbe **(b)** $T = 24$: 2 208 triângulos por orbe

Figura 4: Desenho em arame (tecla `w`) mostrando o aumento do detalhe T dos orbes.

5.3 Critério de contagem dos polígonos

- Toda a geometria da cena é formada **exclusivamente por triângulos** indexados (`GL_TRIANGLES`). Não há quadriláteros, leques ou faixas: cada quadrilátero de uma superfície paramétrica é gerado como 2 triângulos e já contado assim. Portanto, "polígonos" neste relatório significa **triângulos**.
- A contagem é **exata**: cada malha tem `indices.size()/3` triângulos, e o total por quadro é a soma sobre todos os objetos desenhados:

$$\text{triângulos/quadro} = \underbrace{4\,608 + 12\,288 + 528}_{\text{cenário fixo}=17\,424} + N \cdot 4T(T-1). \tag{2}$$

  O orbe tem $2T$ colunas e T faixas; as faixas dos dois polos colapsam num ponto e nelas só é gerado o triângulo não degenerado de cada quadrilátero, daí $2 \cdot 2T \cdot T - 2 \cdot 2T = 4T(T-1)$ triângulos.
- Contam-se os triângulos **submetidos** ao *pipeline* a cada quadro, antes do recorte e do *back-face culling* (habilitado). Como a câmera é fixa e todos os objetos ficam dentro do campo de visão, praticamente nada é recortado; o *culling* descarta, na GPU, cerca de metade dos triângulos (as faces de trás), mas só depois de os vértices terem sido processados.

- O painel de texto e seu fundo (2 triângulos) são interface e não são contados.
- O valor é mostrado no painel da janela e gravado em cada linha dos arquivos CSV.

6 Metodologia de coleta e medição

6.1 Cálculo do FPS e dos tempos

A cada quadro, a aplicação registra quatro instantes e grandezas (`main.cpp`):

- **tempo de emissão**: tempo de relógio entre o início do quadro e o fim da emissão dos comandos OpenGL (antes de `glutSwapBuffers`). Pode incluir esperas dentro do *driver*;
- **tempo de *swap***: tempo bloqueado em `glutSwapBuffers`;
- **CPU ativa**: ciclos de CPU realmente executados pela thread principal no quadro (`QueryThreadCycleTime`, convertidos em ms pela frequência do contador de ciclos, calibrada na inicialização). Ao contrário dos dois tempos anteriores, *não* inclui o tempo em que a thread ficou bloqueada esperando;
- **tempo de GPU**: medido pela própria GPU com consultas `GL_TIME_ELAPSED` em torno de todos os comandos do quadro. Os resultados são lidos sem bloquear, alguns quadros depois (só são consultadas as consultas com pelo menos 3 quadros de idade, pois perguntar por uma consulta recente força o *driver* a esvaziar a fila e alteraria a própria medição).

Numa janela de medição, FPS médio $= n/\Delta t$, com n o número de quadros concluídos (contados após o *swap*) e Δt o tempo entre o início da janela e o fim do último quadro; o tempo médio por quadro é 1000/FPS. As demais grandezas são médias por quadro.

6.2 Utilização de CPU, GPU e memória

Uma thread auxiliar (fora da thread de renderização) coleta amostras a cada 500 ms (`monitor.cpp`):

- **CPU do processo**: `GetProcessTimes`, em % da CPU inteira (22 threads lógicas), como no Gerenciador de Tarefas;
- **thread principal** e **thread mais ocupada do processo**: tempo de CPU de cada thread (`Toolhelp32` + `GetThreadTimes`), em % de *um* núcleo. A thread mais ocupada pode ser uma thread interna do *driver* OpenGL;
- **GPU**: contadores PDH `\GPU Engine(*)\Utilization Percentage` (motor 3D do processo em cada GPU) e `\GPU Process Memory(*)` (memória de vídeo do processo);
- **NVML**: utilização, controlador de memória, potência, temperatura, clocks e motivos de redução de clock (por exemplo, "limite de potência") da GPU NVIDIA;
- **RAM**: *working set* do processo e carga de memória do sistema.

6.3 Protocolo experimental

O protocolo foi automatizado (`benchmark.cpp`) para garantir procedimentos idênticos em todos os pontos:

1. janela fixa de 1280×720 pixels (a aplicação é *DPI-aware*, então são pixels reais), câmera, animação e cena idênticas; V-Sync desligado pela aplicação e o painel de controle da NVIDIA em "usar configuração do aplicativo"; sem limitador de FPS;
2. para cada configuração (A a F), varredura **grossa** de uma lista crescente de valores de T, começando por uma carga leve, que serve de referência;
3. em cada ponto: aplica a carga, **estabiliza 2 s** (descartados), **mede 10 s**; repete **3 vezes** (cada repetição com nova estabilização) e calcula a média e o desvio-padrão amostral do FPS e a média das demais grandezas;
4. o critério de "abaixo de 60" é o **FPS médio das 3 repetições**;
5. ao encontrar o primeiro ponto com FPS médio < 60, faz **refino por bisseção** em T entre o último ponto ≥ 60 e o primeiro < 60, até que as cargas difiram em no máximo 5% de triângulos;
6. continua a varredura grossa até o FPS cair abaixo de um limiar de parada (35 ou 30 FPS), para mostrar a tendência após o limite;
7. registra, para cada configuração, a **menor quantidade testada** de triângulos com FPS médio < 60 e a situação do teste.

Os testes foram executados em sequência pelo *script* de experimentos, com o notebook ligado à tomada e sem outros aplicativos abertos além dos processos do sistema. Todos os valores brutos (cada repetição) e agregados (cada ponto) estão nos arquivos CSV em `resultados/dados`.

6.4 Experimentos realizados

Tabela 3: Experimentos. Em todos: janela 1280×720, V-Sync desligado, 3 repetições de 10 s.

Exp.	GPU	Envio	Variável (valores da varredura grossa)	Config.
E1	RTX 4060	VBO	$T = 16, 64, 128, 192, \ldots, 512, 640, 768$ ($N = 64$)	A–F
E2	RTX 4060	imediato	$T = 8, 16, 24, 32, 48, 64$ ($N = 64$)	A–F
E3	RTX 4060	VBO	$N = 64, 256, 1\,024, \ldots, 65\,536$ ($T = 8$)	A, F
E4	Intel Arc	VBO	$T = 16, 64, 128, 192, 256, 320$ ($N = 64$)	A–F

O E1 é o experimento principal exigido. E2, E3 e E4 são complementares: mudam *onde* está a carga (CPU *versus* GPU) para testar as hipóteses de gargalo levantadas no E1.

7 Implementação das funcionalidades gráficas

7.1 Organização do código

Tabela 4: Arquivos do projeto (código completo no Anexo).

Arquivo	Responsabilidade
`gl_ext.h/.cpp`	carregamento das funções OpenGL > 1.1 (*shaders*, VBO, consultas, V-Sync)
`malha.h/.cpp`	malhas de triângulos: geração do orbe, do nó e do piso; VBO e modo imediato
`texturas.h/.cpp`	texturas procedurais (1024^2, ruído periódico, *mipmaps*, anisotropia)
`shaders.h/.cpp`	fonte GLSL e compilação das 6 variantes (iluminação × textura)
`cena.h/.cpp`	cena, animação, luzes, configurações A–F e contagem de triângulos
`monitor.h/.cpp`	monitoramento de CPU, GPU, memória, potência (PDH, DXGI, NVML)
`benchmark.h/.cpp`	medição dos quadros e protocolo automático com CSV
`main.cpp`	FreeGLUT, laço de quadros, painel, teclado, capturas, linha de comando
`tools/graficos.py`	gráficos e tabelas do relatório a partir dos CSV

7.2 Geometria

As malhas são geradas proceduralmente por uma função genérica que amostra uma superfície paramétrica $\mathbf{P}(u, v)$ numa grade e calcula a normal de cada vértice como $\partial\mathbf{P}/\partial u \times \partial\mathbf{P}/\partial v$ por diferenças centrais, o que dá normais suaves e contínuas nas emendas. O orbe usa $r(\theta, \phi) = 1 + 0{,}1\sin(8\theta)\sin^2\phi + 0{,}05\sin(5\phi)$ (gomos); o nó toroidal $(2, 3)$ usa o referencial de Frenet da curva central. Cada vértice tem posição, normal e coordenada de textura (32 bytes). Os triângulos são orientados no sentido anti-horário visto de fora, permitindo o *back-face culling*.

A mesma malha pode ser enviada de duas formas (tecla `v`): por **VBO** (vértices e índices na memória da GPU; um `glDrawElements` por objeto) ou em **modo imediato** (`glBegin`/`glEnd` com `glTexCoord2f`, `glNormal3f` e `glVertex3f` para cada vértice de cada triângulo, a cada quadro). Ao mudar T, a malha do orbe é recriada e reenviada para a GPU; os N orbes compartilham a mesma malha e diferem apenas na matriz de modelagem e na cor.

7.3 Iluminação e texturas com GLSL

A iluminação é calculada por fragmento num *fragment shader* GLSL 1.20, que implementa a Equação (1). O *vertex shader* transforma posição e normal para coordenadas do observador. Uma única fonte de *shader* é compilada em **seis variantes**, uma para cada combinação de iluminação (`ILUMINACAO` = 0, 1 ou 2) e textura (`TEXTURA` = 0 ou 1), selecionadas por `\#define`. Assim cada configuração executa somente o código de que precisa: a configuração A não calcula iluminação nem lê texturas, e não há desvios condicionais em tempo de execução que pudessem mascarar o custo de cada recurso. As posições da luz e

a direção do *spot* são animadas na CPU e enviadas como *uniforms* em coordenadas do observador.

As três texturas (1024 × 1024 RGBA, 16 MB no total com *mipmaps*) são geradas por código na inicialização, com ruído de valor periódico (sem emendas ao repetir): ladrilhos de arenito (piso), mármore com veios (orbes) e faixas trançadas (nó). Usam filtragem trilinear e anisotrópica 8x. Sem textura, a cor é a do material; com textura, a cor do material é multiplicada pela cor amostrada.

7.4 Comandos de teclado e reprodução dos experimentos

Tabela 5: Comandos de teclado da aplicação.

Tecla	Ação
`1` a `6`	seleciona a configuração A a F
`l`	alterna a iluminação: nenhuma → omnidirecional → spot
`t`	liga/desliga as texturas
`+` / `-`	aumenta/diminui o detalhe T dos orbes (passo 4 abaixo de 64, 16 acima)
`]` / `[`	ajuste fino de T (± 1)
`*` / `/`	dobra/reduz à metade o número de orbes N
`v`	alterna VBO e modo imediato
`s`	liga/desliga o V-Sync
`w`	desenho em arame
`p`	pausa/retoma a animação
`h`	mostra/oculta o painel
`c`	salva captura de tela em `resultados/capturas`
`m`	mede o ponto atual (3 × 10 s) e grava em `resultados/dados/manual_*.csv`
`r` ou `Home`	restaura a configuração inicial (A, $T = 32$, $N = 64$, VBO, V-Sync desligado)
`ESC` ou `q`	encerra

Para reproduzir os experimentos: (1) compilar com `build.bat`; (2) executar `executar_experimentos.bat` na raiz do projeto, que gera as capturas, executa E1–E4 (cerca de 2h30) e os gráficos; ou (3) executar manualmente, por exemplo, `build\Release \desempenho _gl.exe -benchmark -rotulo teste -configs C -valores 256,512`. No modo interativo, qualquer ponto pode ser medido com a tecla `m`, sem recompilar.

8 Resultados do experimento principal (E1)

8.1 Execução da coleta

Os quatro experimentos somaram 90 pontos de carga × configuração no E1, 60 no E2, 24 no E3 e 60 no E4, cada um com 3 repetições de 10 s após 2 s de estabilização. Durante o E1, na configuração F, a janela da aplicação foi minimizada por engano; minimizada, a janela deixa de receber eventos de desenho do FreeGLUT e a medição ficou parada por cerca de uma hora. A repetição afetada foi descartada e a configuração F foi

medida novamente do início, em arquivo próprio (`e1_vbo_T_F_*.csv`), que substitui a F incompleta. Para que isso não se repita, o protocolo passou a descartar e reiniciar automaticamente qualquer repetição em que o intervalo entre dois quadros exceda 2 s; nenhuma ocorrência foi registrada no restante da coleta. As configurações A a E não foram afetadas. A comparação entre a F original e a nova, nos pontos comuns, mostra diferenças de 0,1 a 0,3% para T entre 128 e 256 (região limitada pela GPU) e de até 10% para $T = 16$, região limitada pela CPU, em que o FPS passa de 1 500 e é mais sensível a variações do clock e do escalonamento da CPU (Seção 9).

8.2 FPS em função da quantidade de triângulos

As Figuras 5 a 8 mostram o FPS médio em função dos triângulos por quadro, com a linha de referência de 60 FPS. Os eixos são logarítmicos porque a carga varia de $7{,}9 \times 10^4$ a $1{,}5 \times 10^8$ triângulos e o FPS de ≈1 800 a ≈32.

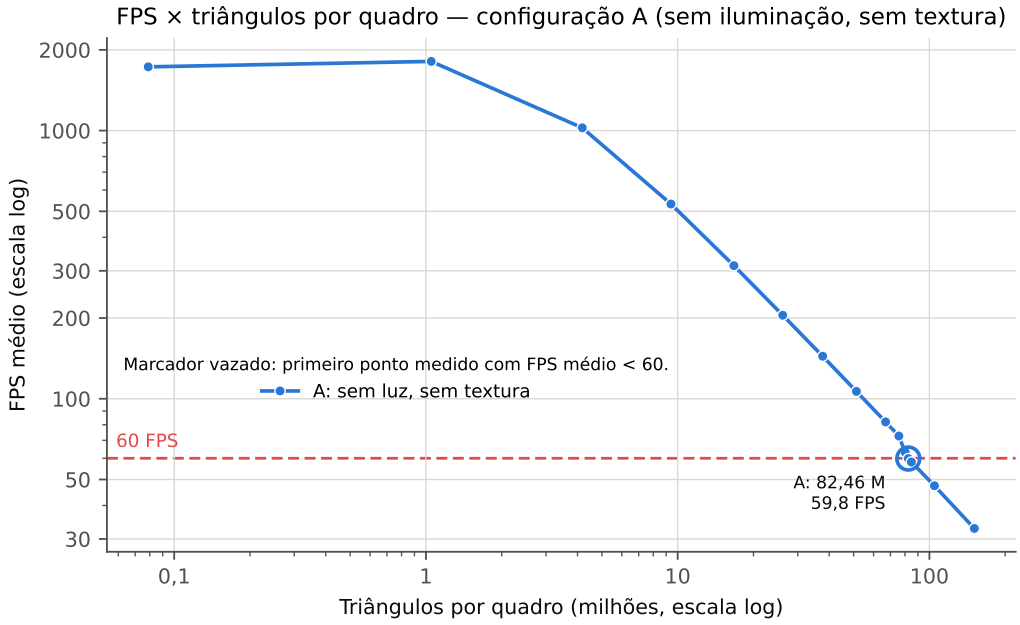


Figura 5: FPS médio × triângulos por quadro na configuração A (E1). O marcador vazado indica a menor carga testada com FPS médio abaixo de 60.

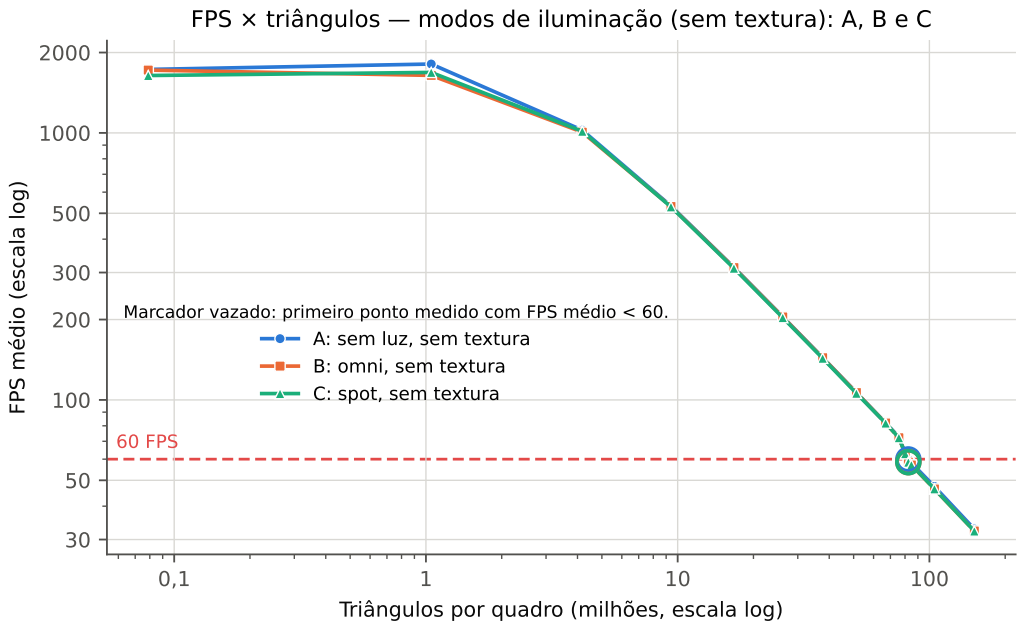


Figura 6: FPS médio × triângulos para os três modos de iluminação sem textura (A, B e C).

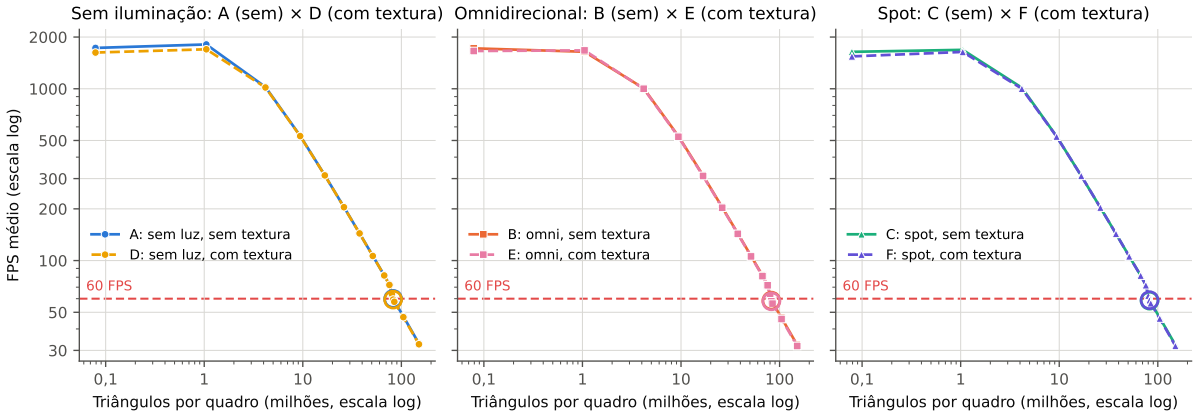


Figura 7: Efeito das texturas: cada painel compara a mesma iluminação sem e com textura (A×D, B×E, C×F).

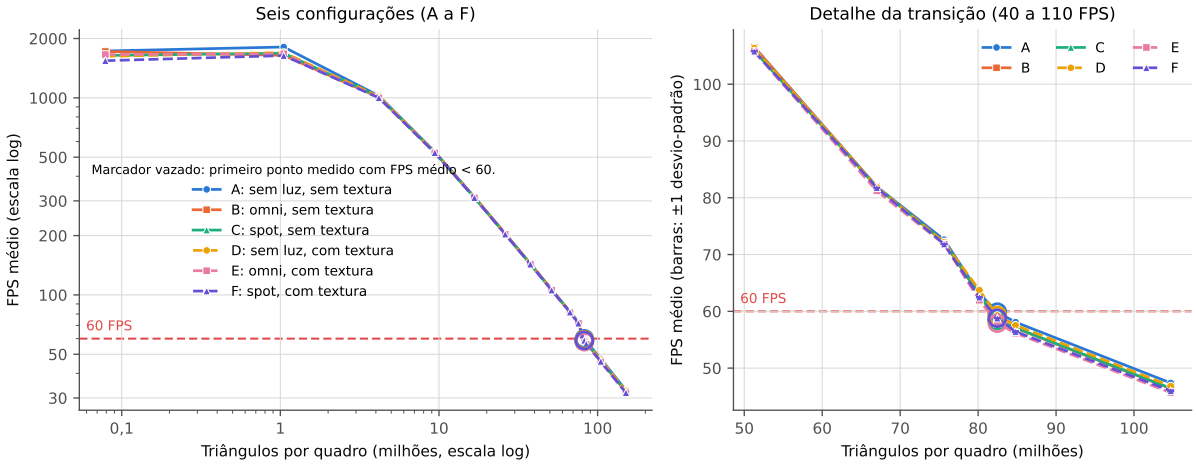


Figura 8: As seis configurações. À direita, a região próxima de 60 FPS em escala linear, com barras de ±1 desvio-padrão das 3 repetições (menores que os marcadores).

As curvas têm três trechos. Até ≈1 milhão de triângulos o FPS é praticamente constante (≈1 550–1 800 FPS): a quantidade de geometria ainda não é o que limita o quadro. A partir de ≈4 milhões o FPS passa a cair na **proporção inversa** dos triângulos (reta de inclinação −1 no gráfico log-log): dobrar os triângulos reduz o FPS à metade. Perto de 80 milhões há um "joelho" (detalhe da Figura 8): entre 75,6 e 80,2 milhões de triângulos (+6%) o FPS cai 13%, mais que o proporcional; a causa é investigada na Seção 9.

As seis curvas são quase sobrepostas. Todas cruzam os 60 FPS **entre os mesmos dois pontos**: 80.155.664 triângulos ($T = 560$), último ponto com FPS médio ≥ 60, e **82.463.760 triângulos** ($T = 568$), primeiro abaixo. Os dois pontos diferem em 2,9% dos triângulos, dentro da tolerância de refino (5%). O desvio-padrão entre as repetições nesse ponto ficou entre 0,02 e 0,05 FPS.

8.3 Tabela-síntese

Tabela 6: Tabela-síntese do experimento principal E1 (GPU dedicada, VBO, aumento de T): menor quantidade testada de triângulos com FPS médio abaixo de 60 em cada configuração.

Cfg.	Último ≥ 60 (FPS)	**Menor testado < 60**	FPS médio ± dp	CPU proc. / thread máx. (%)	GPU 3D (%)	Gargalo (indício mais forte)	Situação
A	80.155.664 (63,2)	**82.463.760**	**59,8** ± 0,02	0,7 / 8	95	GPU (processamento de vértices); no limite de potência	limite encontrado
B	80.155.664 (62,8)	**82.463.760**	**58,5** ± 0,04	0,7 / 9	95	GPU (processamento de vértices); no limite de potência	limite encontrado
C	80.155.664 (62,9)	**82.463.760**	**58,4** ± 0,02	0,7 / 10	97	GPU (processamento de vértices); no limite de potência	limite encontrado
D	80.155.664 (63,8)	**82.463.760**	**59,3** ± 0,02	0,7 / 10	97	GPU (processamento de vértices); no limite de potência	limite encontrado
E	80.155.664 (62,0)	**82.463.760**	**57,9** ± 0,05	0,7 / 9	97	GPU (processamento de vértices); no limite de potência	limite encontrado
F	80.155.664 (62,5)	**82.463.760**	**58,8** ± 0,04	0,6 / 8	95	GPU (processamento de vértices); no limite de potência	limite encontrado

Na coluna “Gargalo”, a classificação é feita pelo `graficos.py` a partir das medições: “GPU” quando o motor 3D da GPU está acima de 90% *e* o tempo de GPU medido pela própria GPU ocupa pelo menos 85% do tempo do quadro; “CPU” quando alguma thread do processo está acima de 85% de um núcleo *e* a GPU abaixo de 60%. A justificativa desse critério está na Seção 9.

8.4 Utilização de CPU e GPU

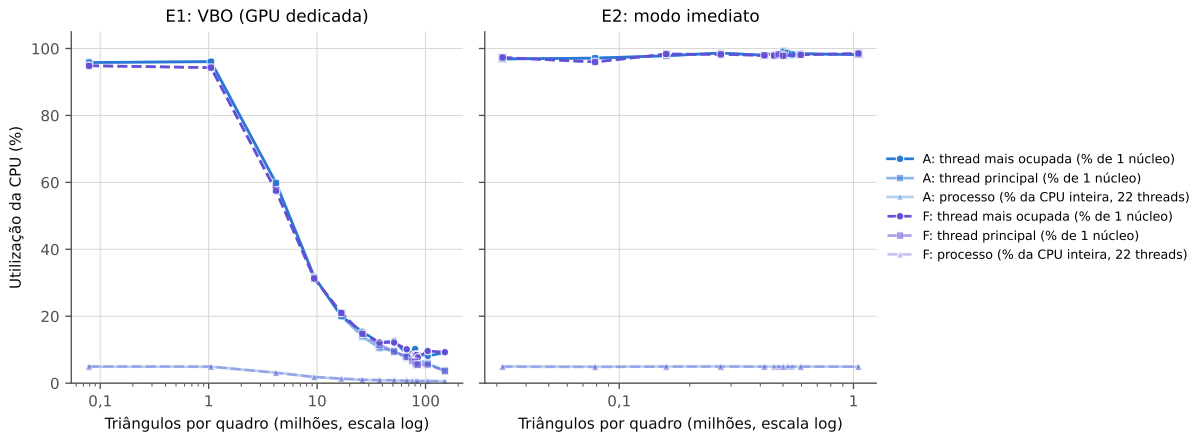


Figura 9: Utilização da CPU × triângulos (configurações A e F). “Thread mais ocupada” e “thread principal” em % de um núcleo; “processo” em % da CPU inteira (22 threads lógicas), como no Gerenciador de Tarefas. À direita, o mesmo para o modo imediato (E2).

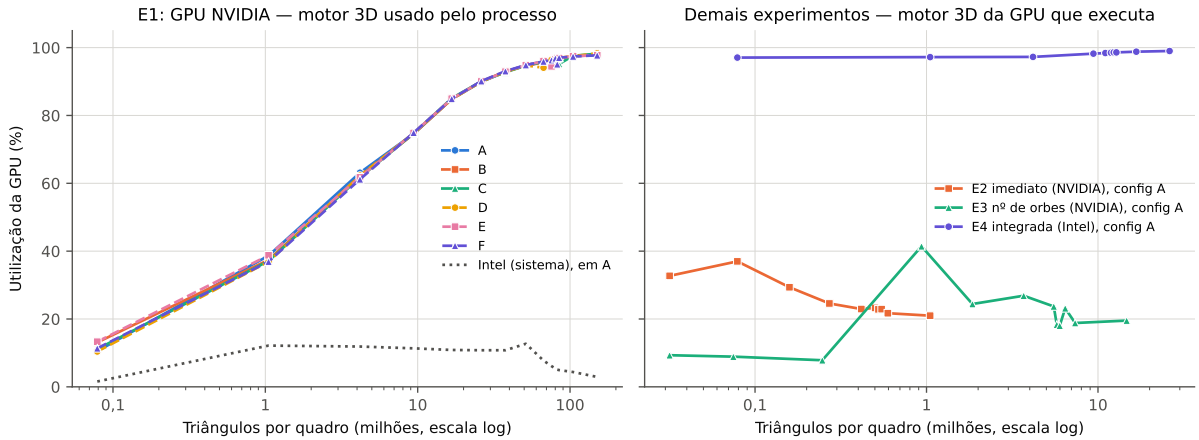


Figura 10: Utilização do motor 3D da GPU pelo processo (contadores PDH) × triângulos. À esquerda, a NVIDIA no E1 (e a Intel, que só compõe a tela); à direita, a GPU que desenha em cada um dos demais experimentos.

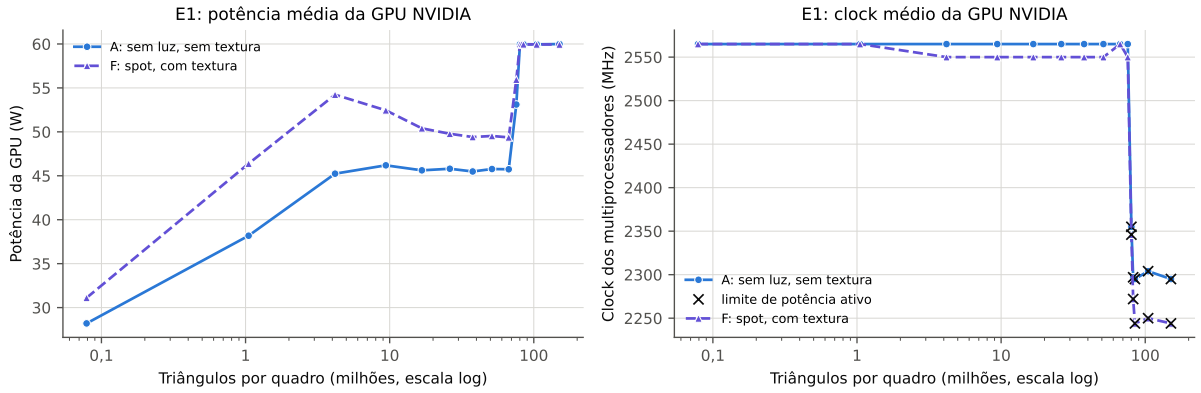


Figura 11: Potência e clock dos multiprocessadores da GPU NVIDIA no E1 (NVML). Os "×" marcam os pontos em que o NVML informou redução de clock por limite de potência.

No E1 a utilização da CPU pelo processo nunca passou de 5% da CPU inteira, e no limite de 60 FPS ficou abaixo de 1% (Figura 9). Em % de um núcleo, a thread principal vai de ≈96% nas cargas leves a menos de 10% nas cargas altas. Ou seja, quanto mais pesada a cena, *menos* a CPU trabalha, porque ela passa a maior parte do quadro esperando a GPU. Já a utilização do motor 3D da NVIDIA sobe de 11% para 95–98% (Figura 10). A potência estabiliza em ≈46 W entre 9 e 67 milhões de triângulos e sobe para o limite de **60 W** a partir de 80 milhões, quando o *driver* passa a reduzir o clock de 2 565 para ≈2 300 MHz (Figura 11). A temperatura ficou entre 64 e 74 °C, sem redução térmica.

9 Investigação dos gargalos

9.1 Decomposição do tempo do quadro

A Figura 12 decompõe o tempo médio de cada quadro (configuração A) em tempo de GPU (`GL_TIME_ELAPSED`), tempo de CPU efetivamente gasto pela thread principal e tempo de emissão dos comandos. É a evidência mais direta de qual recurso determina o FPS: o quadro não pode ser mais rápido que o recurso mais lento, então a curva que *coincide* com o tempo total indica o limitante.

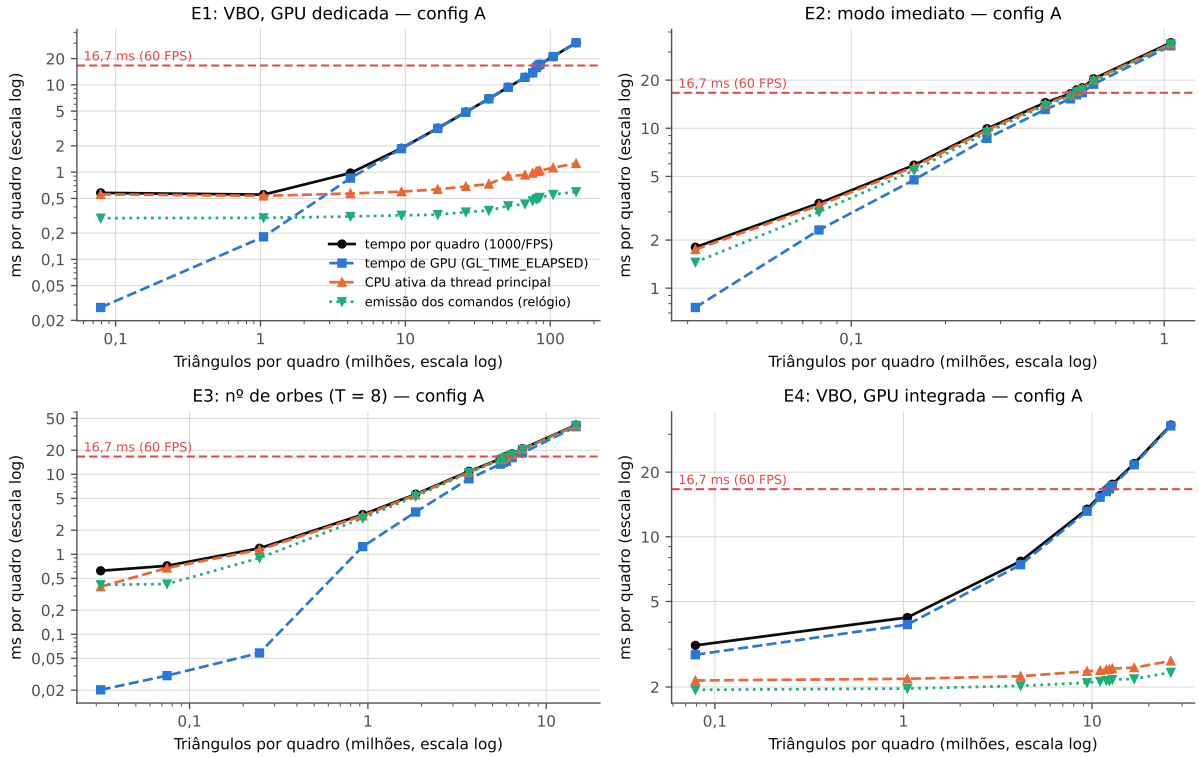


Figura 12: Decomposição do tempo por quadro nos quatro experimentos (configuração A). A linha tracejada vermelha é o orçamento de 16,7 ms de um quadro a 60 FPS.

E1, cargas leves (até ≈1 milhão de triângulos): limite da CPU. Com $T = 16$ o quadro dura 0,58 ms, dos quais 0,56 ms são CPU ativa da thread principal, enquanto a GPU trabalha só 0,03 ms. A GPU fica ociosa 95% do tempo (utilização de 11%) e a thread principal está a 96% de um núcleo. Nessa região o custo é fixo por quadro (67 chamadas de desenho, troca de *shaders* e *uniforms*, troca de *buffers* e a cópia do quadro para a GPU integrada via Optimus), por isso o FPS não depende dos triângulos. A sensibilidade desse regime ao clock e ao escalonamento da CPU explica as diferenças de até 10% entre execuções observadas no re-teste da configuração F.

E1, cargas médias e altas: limite da GPU (processamento de geometria). A partir de $T = 192$ (9,4 milhões) o tempo de GPU coincide com o tempo do quadro (1,85 de 1,88 ms; 16,68 de 16,71 ms no limite), enquanto a CPU ativa cresce só de 0,6 para 1,0 ms. A GPU passa 99,8% do quadro trabalhando; a CPU, menos de 7%. Como o FPS cai na proporção inversa dos triângulos e não depende da iluminação nem da textura (Seção 10), o trabalho limitante está no **processamento de vértices e na montagem/rasterização de triângulos**, não nos fragmentos. Isso é coerente com a geometria: no limite são 82 milhões de triângulos numa janela de 0,92 milhão de pixels, ou seja, a imensa maioria dos triângulos é menor que um pixel e não gera fragmento nenhum, mas todos os seus vértices precisam ser transformados. A vazão medida é de ≈5,5 bilhões de triângulos por segundo a 2 565 MHz (67,0 milhões em 12,18 ms).

O "joelho" perto de 60 FPS: memória e limite de potência. Até $T = 512$ o controlador de memória da GPU fica praticamente parado (0–1% no NVML), mesmo com

a GPU 100% ocupada. Isso acontece porque os 64 orbes compartilham *uma única* malha, que é lida 64 vezes por quadro. Com $T = 512$ essa malha (vértices de 32 bytes mais índices de 32 bits) ocupa ≈28 MB, e a cada orbe ela é relida de dentro da cache L2 da GPU. Com $T = 544$ (31,6 MB) a utilização da memória sobe para 13%; com $T = 560$ (33,5 MB), para 57%; e a partir daí fica em 67–68%. A malha deixou de caber na cache L2, cuja capacidade no chip AD107 é da ordem de 32 MB, e passou a ser buscada na memória GDDR6 a cada orbe. O tráfego de memória eleva o consumo, a GPU atinge o limite de 60 W, e o *driver* reduz o clock em 10% (2 565 → 2 297 MHz) para respeitar esse limite. A vazão cai na mesma proporção: 4,94 bilhões de triângulos por segundo no ponto do limite, 0,898 da anterior, contra a razão de clocks de 0,896.

Ou seja, o FPS de 60 é atingido exatamente quando o sistema troca de regime: antes, limite de *processamento* da GPU com dados na cache; depois, limite de *potência* da GPU provocado pelo acesso à memória. Isso também explica por que as seis configurações cruzam os 60 FPS no mesmo ponto: a transição depende do tamanho da malha, que é o mesmo em todas.

9.2 Teste das hipóteses: levando o gargalo para a CPU (E2 e E3)

Se o E1 é limitado pela GPU, transferir trabalho para a CPU deve mudar completamente o limite e os indicadores. Os experimentos E2 e E3 fazem isso de duas formas.

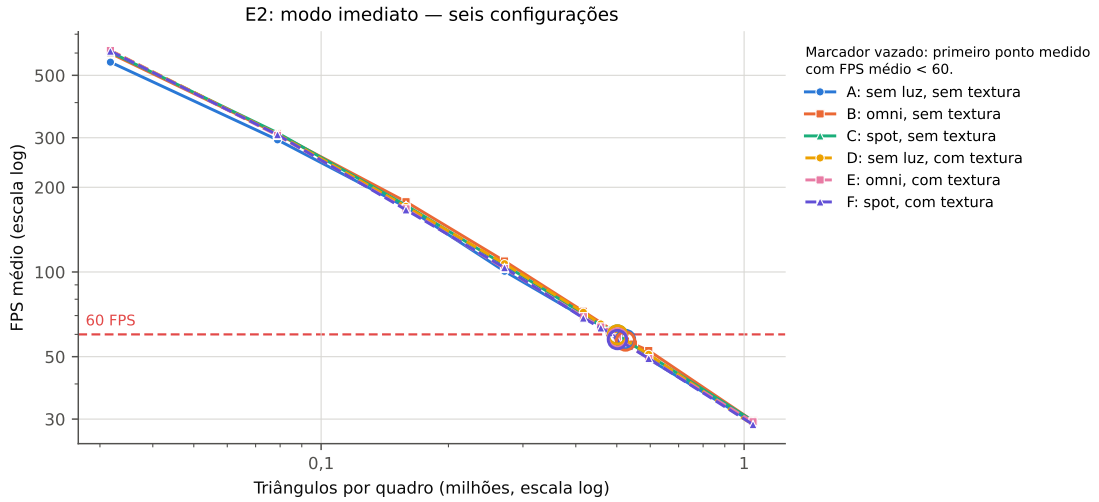


Figura 13: E2: modo imediato (`glBegin`/`glEnd`), seis configurações.

Tabela 7: Tabela-síntese do experimento E2 (modo imediato, GPU dedicada).

Cfg.	Último ≥ 60 (FPS)	**Menor testado < 60**	FPS médio ± dp	CPU proc. / thread máx. (%)	GPU 3D (%)	Gargalo (indício mais forte)	Situação
A	501.776 (60,8)	**524.304**	**57,6** ± 0,42	5,0 / 99	23	CPU: 1 thread saturada (envio de comandos)	limite encontrado
B	501.776 (60,5)	**524.304**	**56,8** ± 1,13	5,0 / 98	23	CPU: 1 thread saturada (envio de comandos)	limite encontrado
C	479.760 (62,8)	**501.776**	**59,7** ± 0,00	5,0 / 99	23	CPU: 1 thread saturada (envio de comandos)	limite encontrado
D	479.760 (60,8)	**501.776**	**59,8** ± 0,31	4,9 / 98	23	CPU: 1 thread saturada (envio de comandos)	limite encontrado
E	479.760 (61,5)	**501.776**	**58,3** ± 0,49	4,9 / 97	20	CPU: 1 thread saturada (envio de comandos)	limite encontrado
F	479.760 (60,9)	**501.776**	**57,5** ± 0,06	4,9 / 98	20	CPU: 1 thread saturada (envio de comandos)	limite encontrado

E2, modo imediato. Cada vértice de cada triângulo passa a ser enviado pela CPU a cada quadro, com três chamadas de função (`glTexCoord2f`, `glNormal3f`, `glVertex3f`). O limite de 60 FPS cai para **≈0,5 milhão** de triângulos (524.304 na configuração A), **160 vezes menos** que com VBO. Os indicadores se invertem em relação ao E1 (Figuras 9, 10 e 12): uma thread do processo fica a 97–99% de um núcleo, a CPU ativa (16,4–17,3 ms) ocupa todo o quadro, e a GPU NVIDIA fica só 20–23% ocupada, com o clock reduzido pelo próprio *driver* por falta de trabalho em parte das execuções. O custo é de ≈3,6 ns por chamada (524 304 triângulos × 3 vértices × 3 chamadas em 17 ms). A utilização do *processo*, ≈5% da CPU, é praticamente uma thread em 22 (100/22 = 4,5%): o gargalo de CPU é invisível olhando só para o total.

Neste regime, o tempo medido por `GL_TIME_ELAPSED` quase coincide com o do quadro (gráfico E2 da Figura 12), mas a utilização da GPU é de 23%. Isso acontece porque a consulta mede o intervalo entre o início e o fim dos comandos *na linha do tempo da GPU*, incluindo os períodos em que ela esperava os comandos chegarem. Por isso o critério de gargalo usa os dois indicadores juntos.

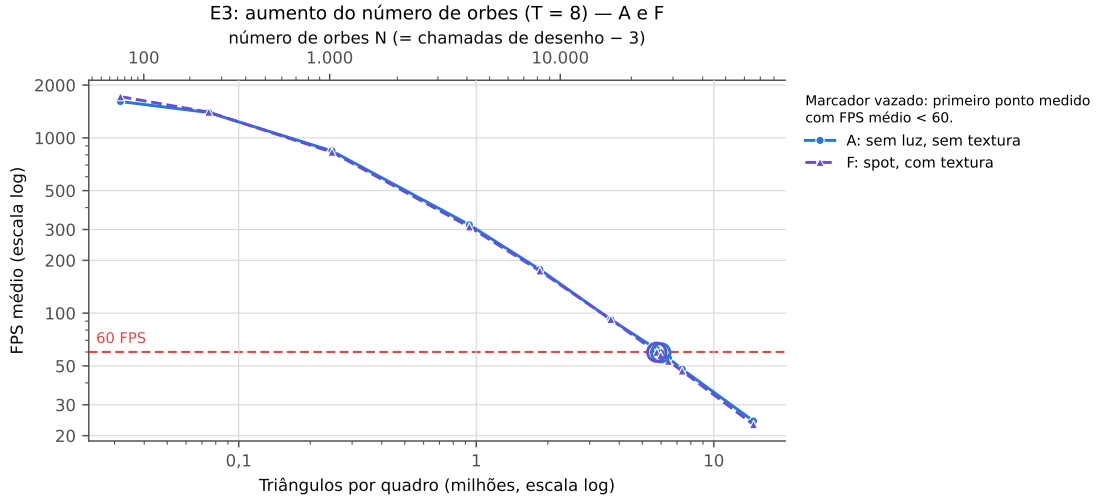


Figura 14: E3: aumento do número de orbes N (cada um uma chamada de desenho), $T = 8$. O eixo superior mostra N.

Tabela 8: Tabela-síntese do experimento E3 (aumento do número de orbes, T = 8).

Cfg.	Último ≥ 60 (FPS)	**Menor testado < 60**	FPS médio ± dp	CPU proc. / thread máx. (%)	GPU 3D (%)	Gargalo (indício mais forte)	Situação
A	5.751.824 (61,9)	**5.981.200**	**59,4** ± 0,33	4,9 / 97	18	CPU: 1 thread saturada (envio de comandos)	limite encontrado
F	5.522.448 (62,1)	**5.751.824**	**59,8** ± 0,06	4,9 / 99	19	CPU: 1 thread saturada (envio de comandos)	limite encontrado

E3, muitos objetos. Com orbes de apenas 224 triângulos, o limite de 60 FPS é atingido com $N \approx 26$ mil orbes, ou **6 milhões** de triângulos (5.981.200 em A). No E1, os mesmos 6 milhões de triângulos divididos em 64 objetos rodam a ≈750 FPS. A diferença está no número de chamadas de desenho: cada orbe exige o envio da matriz de modelagem, da cor e um `glDrawElements`, e custa ≈0,6 µs de CPU (16,4 ms / 26 627 chamadas). Novamente uma thread fica a 97–99%, a GPU a 18–19%, e iluminação e textura são irrelevantes. Ao aumentar N o gargalo é o número de objetos (custo de CPU por chamada), não o de polígonos.

9.3 Mesma aplicação, GPU integrada (E4)

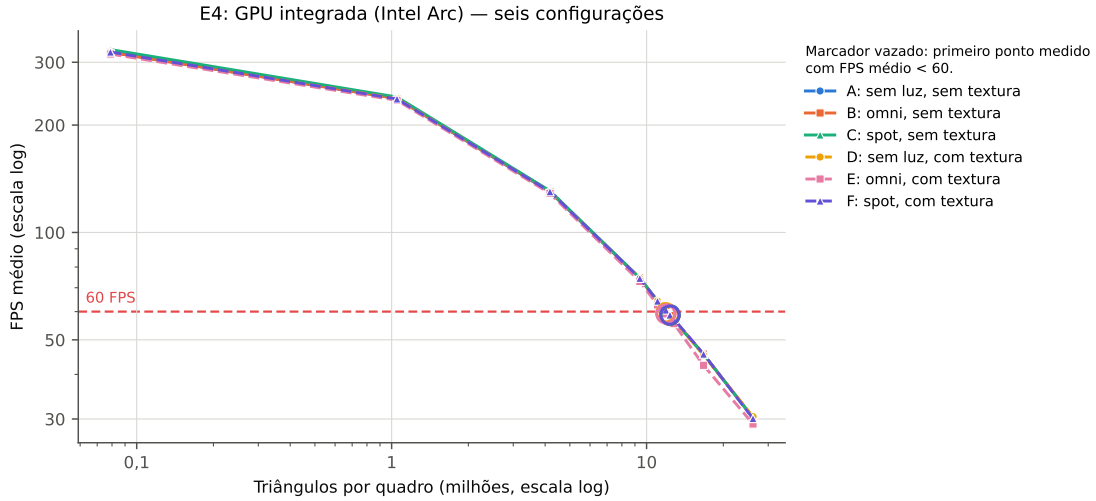


Figura 15: E4: a mesma varredura do E1 executada na GPU integrada Intel Arc.

Tabela 9: Tabela-síntese do experimento E4 (GPU integrada Intel Arc, VBO, aumento de T).

Cfg.	Último ≥ 60 (FPS)	**Menor testado < 60**	FPS médio ± dp	CPU proc. / thread máx. (%)	GPU 3D (%)	Gargalo (indício mais forte)	Situação
A	11.906.064 (60,5)	**12.351.504**	**58,8** ± 0,04	0,9 / 15	99	GPU (processamento de vértices)	limite encontrado
B	11.906.064 (60,5)	**12.351.504**	**58,5** ± 0,08	0,8 / 14	98	GPU (processamento de vértices)	limite encontrado
C	11.906.064 (60,6)	**12.351.504**	**58,6** ± 0,12	0,9 / 14	98	GPU (processamento de vértices)	limite encontrado
D	11.468.816 (62,3)	**11.906.064**	**59,6** ± 0,07	0,9 / 15	98	GPU (processamento de vértices)	limite encontrado
E	11.468.816 (61,0)	**11.906.064**	**59,1** ± 0,08	0,9 / 15	96	GPU (processamento de vértices)	limite encontrado
F	11.906.064 (60,6)	**12.351.504**	**58,7** ± 0,11	0,8 / 14	99	GPU (processamento de vértices)	limite encontrado

Na GPU integrada, o limite fica entre **11,9 e 12,4 milhões** de triângulos, ≈6,7 vezes menos que na RTX 4060. O motor 3D da Intel fica a 96–99% desde as cargas mais leves e o tempo de GPU coincide com o do quadro em toda a faixa (Figura 12): é um gargalo de GPU puro, e a CPU ativa fica em ≈2,1–2,4 ms por quadro (14–15% de um núcleo). Diferente da NVIDIA, a Intel tem um custo fixo de ≈2,8 ms por quadro mesmo com 79 mil triângulos, o mesmo em A e em F, que limita o FPS máximo a ≈320. Descontado esse custo fixo, a vazão de geometria é de ≈0,9 bilhão de triângulos por segundo, cerca de 6 vezes menor que a da RTX. O *driver* da Intel também gasta ≈4 vezes mais CPU por quadro que o da NVIDIA (2,1 contra 0,55 ms), sem que isso chegue a limitar o FPS.

9.4 Síntese dos regimes

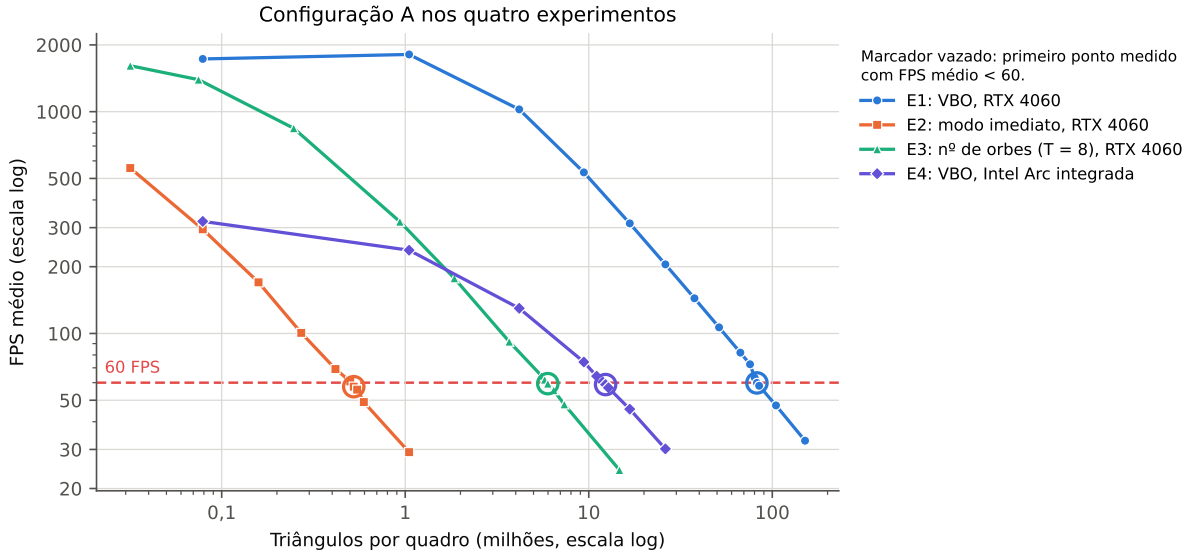


Figura 16: Configuração A nos quatro experimentos: a mesma cena atinge 60 FPS com 0,5 a 82 milhões de triângulos, conforme onde está o gargalo.

Tabela 10: Gargalo identificado em cada experimento, no ponto do limite de 60 FPS (configuração A).

Exp.	Limite (triâng.)	Thread máx.	GPU 3D	GPU/quadro	Gargalo
E1 VBO, RTX	82.463.760	8%	95%	99,8%	GPU: geometria; malha fora da cache L2 e limite de potência (60 W)
E2 imediato, RTX	524.304	99%	23%	–	CPU: uma thread emitindo vértice a vértice
E3 N orbes, RTX	5.981.200	97%	18%	–	CPU: uma thread emitindo uma chamada de desenho por objeto
E4 VBO, Intel	12.351.504	15%	99%	98,3%	GPU integrada: geometria

Em resumo, com a geometria em VBO e poucos objetos, que é a forma recomendada de usar o OpenGL moderno, a CPU praticamente não participa e o limite de 60 FPS é imposto pela GPU. O mesmo número de polígonos pode, no entanto, ficar limitado pela CPU se for enviado de forma ineficiente (modo imediato) ou fragmentado em muitos objetos. Em nenhum caso a utilização total da CPU passou de 5%: o gargalo de CPU, quando existe, é de *uma* thread.

10 Influência da iluminação e das texturas

A Tabela 11 mostra a variação do FPS de cada configuração em relação à A, no E1, em três cargas típicas.

Tabela 11: Variação do FPS em relação à configuração A no E1 (positivo = mais rápido que A).

Carga	FPS de A	B	C	D	E	F
$T = 64$ (1,0 milhão, limite da CPU)	1 811	−9,3%	−7,0%	−6,3%	−7,7%	−9,6%
$T = 256$ (16,7 milhões, GPU)	313,5	−0,2%	−0,7%	−0,1%	−0,9%	−0,7%
$T = 568$ (82,5 milhões, limite)	59,8	−2,3%	−2,3%	−1,0%	−3,2%	−1,7%

- **Na região limitada pela GPU, a iluminação e as texturas custam menos de 1%.** A iluminação é calculada por fragmento e a textura é lida por fragmento, e o número de fragmentos é limitado pelo tamanho da janela (0,92 milhão de pixels), não pelos triângulos. O custo total medido pela GPU de iluminar e texturizar a tela inteira é de apenas 0,01–0,02 ms por quadro: com $T = 16$, o tempo de GPU é 0,028 ms em A e 0,040–0,047 ms em B, E e F. Diante de 16,7 ms de geometria, é desprezível.
- **O spot não é mais caro que a luz omnidirecional** de forma mensurável: o teste do cone (`smoothstep` sobre o cosseno do ângulo) acrescenta poucas operações por fragmento, e fora do cone o termo especular é zerado.
- **No limite de 60 FPS as diferenças chegam a 1–3%**, com A sempre a mais rápida. Essa diferença, um pouco maior que nas cargas médias, aparece justamente quando

a malha sai da cache e o limite de potência entra em ação. Uma explicação plausível é que, nas variantes que usam iluminação ou textura, a GPU precisa transportar para a rasterização mais dados por vértice (posição no espaço do observador, normal, coordenada de textura), enquanto na A o compilador pode eliminar essas saídas. Não alteram o ponto de cruzamento.

- **Na região limitada pela CPU** as diferenças de 6–10% não têm relação com o custo das configurações, já que a GPU trabalha menos de 0,2 ms em todas. Elas estão dentro da variação entre execuções desse regime (10% entre as duas execuções da F com $T = 16$). No E2, também limitado pela CPU, as configurações com iluminação e textura chegaram a ser até 10% *mais* rápidas que a A, o que confirma que as diferenças ali são ruído.
- **Na GPU integrada** (E4), as diferenças ficam em geral abaixo de 2,5% (a maior foi de 6,9%, na E com $T = 256$) e o limite de A, B, C e F é o mesmo; D e E cruzaram um passo antes (11,9 em vez de 12,4 milhões), com FPS de 59,6 e 59,1, a menos de 1 FPS da linha.

A iluminação e as texturas, portanto, **não mudam o limite de polígonos** nesta cena. Seu custo seria relevante numa cena limitada por fragmentos: resolução maior, objetos que ocupem muitos pixels, muitas luzes ou sobreposição de superfícies transparentes. Com o aumento de polígonos proposto pela atividade, a carga cresce por vértice, e a geometria domina.

11 Conclusão

A aplicação desenvolvida permitiu controlar e contar exatamente os triângulos por quadro, alternar entre as seis configurações de iluminação e textura e registrar, para cada ponto, o FPS e a utilização de CPU e GPU, seguindo um protocolo automatizado (2 s de estabilização, 3 repetições de 10 s, V-Sync desligado, janela fixa de 1280×720).

- No experimento principal (GPU dedicada, VBO), **as seis configurações ficam abaixo de 60 FPS a partir de 82.463.760 triângulos por quadro** ($T = 568$), com 80 155 664 triângulos ainda acima de 60 FPS.
- O gargalo é a **GPU**: tempo de GPU igual ao tempo do quadro, motor 3D a 95–97%, CPU do processo abaixo de 1%. O ponto exato do limite coincide com a saída da malha da cache L2, que provoca o aumento do tráfego de memória, o limite de potência de 60 W e uma redução de 10% no clock.
- **Iluminação e texturas influenciam muito pouco** (menos de 1% na maior parte da faixa e até 3% no limite), porque o custo delas é proporcional aos pixels, enquanto o custo que cresce é proporcional aos vértices.
- O mesmo limite pode ser 160 vezes menor quando a CPU vira gargalo (modo imediato: 0,5 milhão) ou 14 vezes menor quando o trabalho é dividido em muitos objetos (6 milhões em 26 mil chamadas de desenho). Nesses casos, uma única thread fica saturada enquanto a utilização total da CPU não passa de 5%.
- Na GPU integrada, o limite é de ≈12 milhões de triângulos, 6,7 vezes menor, e também limitado pela GPU.

As medições mostram que “utilização alta” sozinha não identifica o gargalo: foi preciso cruzar a utilização por thread e por GPU, o tempo do quadro decomposto em tempo de GPU e de CPU e o comportamento do FPS sob mudanças dirigidas da carga.

Apêndice — Todos os pontos medidos

Valores médios das 3 repetições de cada ponto. Os dados por repetição estão em `resultados/dados/*_repeticoes.csv`.

Tabela 12: Todos os pontos medidos no experimento E1 (médias de 3 repetições de 10 s). “r” = ponto de refino; ▶ = primeiro ponto com FPS médio < 60.

Cfg.	T	Triângulos	FPS	dp	Quadro (ms)	GPU (ms)	CPU ativa (ms)	CPU proc. (%)	Thread máx. (%)	GPU 3D (%)	Pot. (W)	Clock (MHz)
A	16	78.864	1.728,7	20,75	0,58	0,03	0,56	4,9	96	11	28	2.565
A	64	1.049.616	1.811,1	25,32	0,55	0,18	0,53	4,9	96	39	38	2.565
A	128	4.178.960	1.024,1	2,12	0,98	0,85	0,57	3,1	60	63	45	2.565
A	192	9.405.456	532,4	0,08	1,88	1,85	0,60	1,9	32	75	46	2.565
A	256	16.729.104	313,5	0,04	3,19	3,17	0,63	1,3	20	85	46	2.565
A	320	26.149.904	204,8	0,03	4,88	4,86	0,68	1,0	15	90	46	2.565
A	384	37.667.856	144,1	0,01	6,94	6,92	0,73	0,8	12	93	45	2.565
A	448	51.282.960	106,5	0,01	9,39	9,36	0,90	0,8	13	95	46	2.565
A	512	66.995.216	82,0	0,00	12,20	12,18	0,93	0,7	9	96	46	2.565
A	544r	75.637.776	72,6	0,00	13,77	13,74	0,97	0,8	10	96	53	2.565
A	560r	80.155.664	63,2	0,05	15,83	15,80	1,02	0,6	10	97	60	2.355
▶A	568r	82.463.760	59,8	0,02	16,71	16,68	1,02	0,7	8	95	60	2.297
A	576r	84.804.624	58,1	0,06	17,21	17,19	1,04	0,7	9	97	60	2.295
A	640	104.711.184	47,4	0,10	21,10	21,08	1,12	0,6	8	97	60	2.304
A	768	150.815.760	32,8	0,01	30,45	30,42	1,26	0,6	9	98	60	2.295
B	16	78.864	1.718,9	45,87	0,58	0,04	0,56	5,0	95	13	32	2.565
B	64	1.049.616	1.642,2	25,13	0,61	0,19	0,56	4,9	93	37	44	2.565
B	128	4.178.960	1.006,5	2,78	0,99	0,86	0,57	3,1	59	62	52	2.565
B	192	9.405.456	529,9	0,12	1,89	1,86	0,58	1,8	31	75	52	2.565
B	256	16.729.104	312,9	0,03	3,20	3,17	0,62	1,3	19	85	50	2.565
B	320	26.149.904	204,5	0,02	4,89	4,87	0,66	1,1	14	90	50	2.565
B	384	37.667.856	143,9	0,01	6,95	6,93	0,74	0,9	12	93	49	2.565
B	448	51.282.960	106,5	0,01	9,39	9,37	0,87	0,9	13	95	49	2.565
B	512	66.995.216	81,9	0,01	12,21	12,18	0,90	0,8	10	96	49	2.565
B	544r	75.637.776	72,2	0,04	13,86	13,83	0,96	0,7	10	96	56	2.551
B	560r	80.155.664	62,8	0,04	15,92	15,89	1,03	0,7	10	97	60	2.346
▶B	568r	82.463.760	58,5	0,04	17,11	17,08	1,05	0,7	9	95	60	2.245
B	576r	84.804.624	57,0	0,03	17,53	17,51	1,03	0,7	9	97	60	2.252
B	640	104.711.184	46,4	0,06	21,56	21,53	1,15	0,7	9	97	60	2.256
B	768	150.815.760	32,3	0,01	30,97	30,94	1,22	0,6	11	98	60	2.254
C	16	78.864	1.640,4	9,19	0,61	0,04	0,57	4,9	93	11	32	2.565
C	64	1.049.616	1.684,8	6,70	0,59	0,19	0,56	5,0	95	37	45	2.565
C	128	4.178.960	1.014,1	1,67	0,99	0,85	0,55	3,0	57	62	53	2.563
C	192	9.405.456	528,4	0,22	1,89	1,87	0,58	1,8	30	75	52	2.550
C	256	16.729.104	311,2	0,08	3,21	3,19	0,62	1,3	18	85	50	2.550
C	320	26.149.904	203,3	0,02	4,92	4,89	0,66	1,1	14	90	49	2.550
C	384	37.667.856	143,2	0,01	6,98	6,96	0,71	0,9	11	93	49	2.550
C	448	51.282.960	105,9	0,01	9,44	9,42	0,84	0,8	11	95	49	2.550
C	512	66.995.216	81,9	0,01	12,21	12,19	0,88	0,8	10	96	49	2.565
C	544r	75.637.776	72,2	0,01	13,85	13,83	0,94	0,7	10	96	56	2.550
C	560r	80.155.664	62,9	0,04	15,90	15,87	1,00	0,7	10	97	60	2.348
▶C	568r	82.463.760	58,4	0,02	17,11	17,09	1,03	0,7	10	97	60	2.244
C	576r	84.804.624	57,1	0,02	17,51	17,48	1,04	0,7	9	95	60	2.256
C	640	104.711.184	46,4	0,03	21,53	21,51	1,14	0,7	9	98	60	2.258
C	768	150.815.760	32,3	0,01	30,98	30,95	1,26	0,6	10	98	60	2.252
D	16	78.864	1.624,7	25,42	0,62	0,03	0,56	4,9	92	10	32	2.565
D	64	1.049.616	1.697,0	18,58	0,59	0,18	0,55	4,9	94	37	42	2.565
D	128	4.178.960	1.017,4	3,02	0,98	0,85	0,55	3,1	59	62	49	2.565
D	192	9.405.456	530,5	0,54	1,89	1,86	0,59	1,8	31	75	49	2.565
D	256	16.729.104	313,1	0,02	3,19	3,17	0,63	1,3	19	85	47	2.565

Cfg.	T	Triângulos	FPS	dp	Quadro (ms)	GPU (ms)	CPU ativa (ms)	CPU proc. (%)	Thread máx. (%)	GPU 3D (%)	Pot. (W)	Clock (MHz)
D	320	26.149.904	204,5	0,03	4,89	4,86	0,66	1,1	14	90	48	2.565
D	384	37.667.856	143,9	0,00	6,95	6,92	0,72	0,9	12	93	47	2.565
D	448	51.282.960	106,3	0,00	9,40	9,38	0,84	0,9	11	95	47	2.565
D	512	66.995.216	81,9	0,00	12,21	12,19	0,89	0,7	10	94	47	2.565
D	544r	75.637.776	72,2	0,00	13,86	13,84	0,95	0,7	10	96	55	2.550
D	560r	80.155.664	63,8	0,04	15,69	15,66	0,99	0,8	15	97	60	2.378
►D	568r	82.463.760	59,3	0,02	16,87	16,84	1,07	0,7	10	97	60	2.276
D	576r	84.804.624	57,6	0,03	17,38	17,35	1,04	0,7	10	97	60	2.271
D	640	104.711.184	46,8	0,09	21,35	21,32	1,14	0,7	10	97	60	2.276
D	768	150.815.760	32,6	0,01	30,70	30,68	1,23	0,6	9	98	60	2.275
E	16	78.864	1.658,8	28,09	0,60	0,05	0,57	5,0	95	13	34	2.555
E	64	1.049.616	1.672,5	12,99	0,60	0,20	0,56	5,0	94	39	47	2.565
E	128	4.178.960	1.000,3	1,99	1,00	0,86	0,58	3,2	60	62	55	2.552
E	192	9.405.456	525,6	0,20	1,90	1,88	0,60	1,9	32	75	54	2.550
E	256	16.729.104	310,8	0,09	3,22	3,19	0,63	1,3	19	85	51	2.550
E	320	26.149.904	203,0	0,00	4,93	4,90	0,71	1,1	15	90	51	2.550
E	384	37.667.856	143,0	0,00	6,99	6,97	0,74	0,9	12	93	51	2.550
E	448	51.282.960	105,7	0,01	9,46	9,43	0,83	0,8	11	95	51	2.550
E	512	66.995.216	81,2	0,00	12,31	12,29	0,90	0,8	10	96	50	2.550
E	544r	75.637.776	71,8	0,01	13,93	13,90	0,94	0,7	9	94	58	2.550
E	560r	80.155.664	62,0	0,05	16,14	16,12	1,01	0,7	10	97	60	2.329
►E	568r	82.463.760	57,9	0,05	17,27	17,24	1,05	0,7	9	97	60	2.239
E	576r	84.804.624	56,1	0,03	17,81	17,78	1,04	0,7	9	97	60	2.234
E	640	104.711.184	45,7	0,04	21,89	21,87	1,13	0,7	10	97	60	2.237
E	768	150.815.760	31,8	0,05	31,42	31,40	1,22	0,6	9	98	60	2.238
F	16	78.864	1.543,2	166,61	0,65	0,04	0,61	5,0	95	11	31	2.565
F	64	1.049.616	1.638,1	11,55	0,61	0,19	0,57	5,0	94	37	46	2.565
F	128	4.178.960	1.000,7	1,48	1,00	0,85	0,57	3,1	57	61	54	2.550
F	192	9.405.456	527,0	0,36	1,90	1,87	0,60	1,8	31	75	52	2.550
F	256	16.729.104	311,3	0,06	3,21	3,19	0,66	1,4	21	85	50	2.550
F	320	26.149.904	203,5	0,01	4,91	4,89	0,71	1,1	15	90	50	2.550
F	384	37.667.856	143,2	0,01	6,99	6,96	0,78	1,0	12	93	49	2.550
F	448	51.282.960	105,8	0,01	9,45	9,43	0,86	0,9	12	95	50	2.550
F	512	66.995.216	81,8	0,00	12,23	12,21	0,95	0,8	10	96	49	2.565
F	544r	75.637.776	71,9	0,00	13,92	13,89	0,89	0,6	8	96	56	2.550
F	560r	80.155.664	62,5	0,05	16,00	15,97	0,95	0,7	9	97	60	2.346
►F	568r	82.463.760	58,8	0,04	17,01	16,98	0,97	0,6	8	95	60	2.272
F	576r	84.804.624	56,4	0,10	17,72	17,69	1,00	0,6	8	97	60	2.244
F	640	104.711.184	46,0	0,10	21,74	21,71	1,09	0,7	10	97	60	2.250
F	768	150.815.760	31,9	0,08	31,36	31,34	1,11	0,5	9	98	60	2.244

Tabela 13: Pontos medidos no experimento E2 (modo imediato).

Cfg.	T	Triângulos	FPS	dp	Quadro (ms)	GPU (ms)	CPU ativa (ms)	CPU proc. (%)	Thread máx. (%)	GPU 3D (%)	Pot. (W)	Clock (MHz)
A	8	31.760	556,9	43,07	1,80	0,76	1,75	5,0	97	33	26	2.565
A	16	78.864	294,9	13,76	3,40	2,31	3,30	5,0	97	37	25	2.565
A	24	158.736	170,0	2,14	5,88	4,76	5,73	5,0	98	29	25	2.565
A	32	271.376	100,6	0,19	9,94	8,65	9,75	5,0	99	25	24	2.565
A	40r	416.784	69,1	0,24	14,47	13,10	14,16	5,0	98	23	24	2.565
A	44r	501.776	60,8	0,36	16,46	15,27	16,21	5,0	99	23	24	2.565
►A	45r	524.304	57,6	0,42	17,38	16,18	17,05	5,0	99	23	24	2.565
A	46r	547.344	55,8	0,07	17,92	16,68	17,64	4,9	98	23	24	2.565
A	48	594.960	49,0	0,97	20,39	18,85	20,09	5,0	98	22	24	2.561
A	64	1.049.616	29,2	0,15	34,29	32,78	33,65	4,9	98	21	24	2.565
B	8	31.760	598,4	4,58	1,67	0,72	1,59	4,9	96	32	26	2.565
B	16	78.864	309,8	6,93	3,23	2,24	3,13	4,9	97	38	25	2.565
B	24	158.736	177,7	1,28	5,63	4,63	5,47	4,9	97	30	25	2.565
B	32	271.376	109,4	0,57	9,14	8,09	8,95	4,9	98	26	25	2.565
B	40r	416.784	72,5	0,94	13,80	12,66	13,55	4,9	98	24	25	2.565
B	44r	501.776	60,5	0,91	16,53	15,31	16,14	4,9	98	23	25	2.565
►B	45r	524.304	56,8	1,13	17,60	16,30	17,26	5,0	98	23	25	2.565
B	46r	547.344	55,4	0,13	18,04	16,82	17,66	4,9	98	23	25	2.565
B	48	594.960	52,6	0,37	19,03	17,87	18,82	4,9	99	23	25	2.565
B	64	1.049.616	29,6	0,16	33,84	32,38	33,22	5,0	99	21	24	2.565

Cfg.	T	Triângulos	FPS	dp	Quadro (ms)	GPU (ms)	CPU ativa (ms)	CPU proc. (%)	Thread máx. (%)	GPU 3D (%)	Pot. (W)	Clock (MHz)
C	8	31.760	604,7	4,24	1,65	0,73	1,60	4,9	97	33	26	2.565
C	16	78.864	312,3	1,94	3,20	2,23	3,11	4,9	98	38	26	2.565
C	24	158.736	172,5	1,23	5,80	4,72	5,61	4,9	96	30	25	2.565
C	32	271.376	105,9	1,27	9,44	8,28	9,22	4,9	97	26	25	2.565
C	40r	416.784	71,0	0,41	14,09	12,90	13,76	4,9	97	23	25	2.565
C	42r	458.256	64,6	0,45	15,47	14,24	15,16	4,9	98	23	25	2.565
C	43r	479.760	62,8	0,12	15,93	14,72	15,59	4,9	98	23	25	2.565
▶C	44r	501.776	59,7	0,00	16,75	15,54	16,43	5,0	99	23	25	2.565
C	48	594.960	50,6	0,23	19,75	18,47	19,32	4,9	97	22	25	2.565
C	64	1.049.616	29,5	0,30	33,85	32,41	33,24	4,9	98	21	24	2.565
D	8	31.760	612,8	5,24	1,63	0,71	1,57	4,8	95	32	26	2.565
D	16	78.864	309,0	2,03	3,24	2,26	3,15	4,9	98	38	25	2.565
D	24	158.736	171,0	1,43	5,85	4,80	5,74	5,0	98	29	25	2.565
D	32	271.376	107,0	0,84	9,35	8,26	9,15	4,9	99	26	25	2.565
D	40r	416.784	71,9	0,13	13,91	12,76	13,64	4,9	98	24	25	2.565
D	42r	458.256	65,3	0,35	15,30	14,08	14,98	4,9	98	23	25	2.565
D	43r	479.760	60,8	1,81	16,46	15,09	16,15	4,9	98	22	24	2.504
▶D	44r	501.776	59,8	0,31	16,74	15,50	16,41	4,9	98	23	25	2.565
D	48	594.960	51,0	0,38	19,63	18,35	19,32	5,0	98	22	25	2.565
D	64	1.049.616	28,6	0,46	34,97	33,29	34,45	5,0	99	20	22	2.393
E	8	31.760	612,5	1,94	1,63	0,69	1,57	4,8	95	32	27	2.565
E	16	78.864	305,7	3,69	3,27	2,22	3,17	4,9	97	36	26	2.565
E	24	158.736	167,6	1,37	5,97	4,80	5,82	4,9	98	27	21	2.269
E	32	271.376	102,9	0,69	9,72	8,40	9,50	5,0	98	23	14	1.372
E	40r	416.784	69,3	0,29	14,43	13,00	14,11	4,9	97	20	15	1.593
E	42r	458.256	63,7	0,45	15,69	14,28	15,40	4,9	98	20	15	1.607
E	43r	479.760	61,5	0,20	16,25	14,81	15,92	4,9	98	20	16	1.662
▶E	44r	501.776	58,3	0,49	17,16	15,68	16,82	4,9	97	20	15	1.531
E	48	594.960	49,2	0,20	20,32	18,80	19,84	4,9	98	20	16	1.626
E	64	1.049.616	29,3	0,05	34,08	32,31	33,43	4,9	99	19	16	1.663
F	8	31.760	609,4	0,78	1,64	0,66	1,57	4,9	97	30	27	2.565
F	16	78.864	307,7	2,10	3,25	2,16	3,12	4,8	96	34	26	2.565
F	24	158.736	166,3	2,73	6,02	4,77	5,88	5,0	98	26	17	1.763
F	32	271.376	103,6	0,41	9,65	8,37	9,46	4,9	98	22	14	1.409
F	40r	416.784	68,6	0,75	14,58	13,12	14,32	4,9	98	20	15	1.570
F	42r	458.256	63,3	0,63	15,81	14,34	15,46	4,9	98	20	15	1.570
F	43r	479.760	60,9	0,23	16,43	15,02	16,07	4,9	98	20	15	1.542
▶F	44r	501.776	57,5	0,06	17,39	15,91	16,98	4,9	98	20	16	1.671
F	48	594.960	49,3	0,40	20,30	18,77	19,95	4,9	98	19	16	1.671
F	64	1.049.616	28,7	0,10	34,86	33,03	34,25	4,9	98	18	15	1.598

Tabela 14: Pontos medidos no experimento E3 (N orbes com T = 8).

Cfg.	N	Triângulos	FPS	dp	Quadro (ms)	GPU (ms)	CPU ativa (ms)	CPU proc. (%)	Thread máx. (%)	GPU 3D (%)	Pot. (W)	Clock (MHz)
A	64	31.760	1.611,1	74,81	0,62	0,02	0,39	4,8	90	9	26	2.565
A	256	74.768	1.394,2	11,86	0,72	0,03	0,67	4,8	93	9	27	2.565
A	1.024	246.800	841,9	9,94	1,19	0,06	1,14	4,9	96	8	26	2.565
A	4.096	934.928	319,1	1,55	3,13	1,25	3,02	4,8	96	41	14	1.170
A	8.192	1.852.432	177,3	0,90	5,64	3,38	5,52	4,9	98	24	13	1.060
A	16.384	3.687.440	91,9	0,10	10,89	8,77	10,61	4,9	97	27	10	807
A	24.576r	5.522.448	64,5	1,03	15,51	13,39	15,27	4,9	98	24	10	728
A	25.600r	5.751.824	61,9	0,08	16,16	14,00	15,77	4,8	96	18	10	719
▶A	26.624r	5.981.200	59,4	0,33	16,84	14,50	16,43	4,9	97	18	10	714
A	28.672r	6.439.952	55,5	0,21	18,02	16,12	17,70	5,0	99	23	10	669
A	32.768	7.357.456	47,9	0,64	20,88	18,52	20,56	4,9	98	19	10	642
A	65.536	14.697.488	24,2	0,23	41,28	39,35	40,45	4,9	98	20	10	572
F	64	31.760	1.718,2	21,62	0,58	0,03	0,54	4,7	92	10	31	2.565
F	256	74.768	1.401,7	4,31	0,71	0,04	0,67	4,8	95	10	31	2.565
F	1.024	246.800	828,5	5,21	1,21	0,07	1,16	4,9	96	9	29	2.565
F	4.096	934.928	310,7	0,69	3,22	1,27	3,09	4,8	97	10	14	1.185
F	8.192	1.852.432	174,2	3,11	5,74	3,44	5,59	4,9	97	24	13	1.046
F	16.384	3.687.440	92,1	0,23	10,86	8,80	10,63	4,8	96	13	10	796
F	24.576r	5.522.448	62,1	0,18	16,09	13,95	15,77	4,9	98	14	10	671
▶F	25.600r	5.751.824	59,8	0,06	16,71	14,47	16,36	4,9	99	19	10	667

Cfg.	N	Triângulos	FPS	dp	Quadro (ms)	GPU (ms)	CPU ativa (ms)	CPU proc. (%)	Thread máx. (%)	GPU 3D (%)	Pot. (W)	Clock (MHz)
F	26.624r	5.981.200	57,4	0,73	17,41	15,05	17,15	4,9	99	19	10	654
F	28.672r	6.439.952	53,2	0,65	18,80	16,81	18,47	4,9	98	20	10	631
F	32.768	7.357.456	46,8	0,11	21,37	18,94	20,96	4,9	98	19	10	628
F	65.536	14.697.488	23,1	0,21	43,24	41,05	42,45	5,0	99	18	10	551

Tabela 15: Pontos medidos no experimento E4 (GPU integrada). Potência e clock da NVML não se aplicam.

Cfg.	T	Triângulos	FPS	dp	Quadro (ms)	GPU (ms)	CPU ativa (ms)	CPU proc. (%)	Thread máx. (%)	GPU 3D (%)	Pot. (W)	Clock (MHz)
A	16	78.864	320,2	0,74	3,12	2,83	2,14	3,4	69	97	–	–
A	64	1.049.616	237,4	0,28	4,21	3,90	2,18	2,6	52	97	–	–
A	128	4.178.960	129,9	0,10	7,70	7,41	2,24	1,5	28	97	–	–
A	192	9.405.456	74,4	0,03	13,45	13,16	2,37	1,0	17	98	–	–
A	208r	11.039.760	64,2	0,04	15,57	15,28	2,39	0,9	18	98	–	–
A	216r	11.906.064	60,5	0,09	16,53	16,24	2,41	0,9	14	98	–	–
▶A	220r	12.351.504	58,8	0,04	17,00	16,71	2,41	0,9	15	99	–	–
A	224r	12.805.136	56,9	0,08	17,57	17,28	2,44	0,9	14	99	–	–
A	256	16.729.104	45,6	0,04	21,94	21,63	2,46	0,8	12	99	–	–
A	320	26.149.904	30,2	0,02	33,14	32,83	2,64	0,6	8	99	–	–
B	16	78.864	319,7	1,49	3,13	2,82	2,15	3,4	67	97	–	–
B	64	1.049.616	237,3	0,33	4,21	3,90	2,18	2,6	52	97	–	–
B	128	4.178.960	130,3	0,07	7,67	7,39	2,26	1,6	30	97	–	–
B	192	9.405.456	74,5	0,01	13,42	13,13	2,34	1,0	18	98	–	–
B	208r	11.039.760	64,0	0,15	15,63	15,35	2,42	1,4	28	98	–	–
B	216r	11.906.064	60,5	0,09	16,53	16,24	2,43	0,9	15	98	–	–
▶B	220r	12.351.504	58,5	0,08	17,09	16,80	2,43	0,8	14	98	–	–
B	224r	12.805.136	56,9	0,15	17,57	17,27	2,46	0,8	14	99	–	–
B	256	16.729.104	45,6	0,06	21,94	21,64	2,51	0,8	12	99	–	–
B	320	26.149.904	30,3	0,04	33,05	32,74	2,67	0,5	7	99	–	–
C	16	78.864	325,6	0,73	3,07	2,77	2,10	3,3	68	97	–	–
C	64	1.049.616	239,7	0,34	4,17	3,87	2,17	2,6	52	97	–	–
C	128	4.178.960	130,8	0,08	7,65	7,35	2,22	1,6	30	97	–	–
C	192	9.405.456	74,6	0,12	13,41	13,10	2,34	1,0	17	98	–	–
C	208r	11.039.760	64,3	0,08	15,54	15,25	2,37	0,9	17	98	–	–
C	216r	11.906.064	60,6	0,03	16,49	16,18	2,35	0,9	14	98	–	–
▶C	220r	12.351.504	58,6	0,12	17,07	16,77	2,41	0,9	14	98	–	–
C	224r	12.805.136	56,6	0,02	17,66	17,35	2,42	0,8	14	99	–	–
C	256	16.729.104	45,5	0,05	21,97	21,62	2,48	0,7	12	99	–	–
C	320	26.149.904	30,3	0,07	33,05	32,73	2,63	0,6	8	99	–	–
D	16	78.864	319,1	0,95	3,13	2,83	2,15	3,3	67	97	–	–
D	64	1.049.616	237,3	0,24	4,21	3,91	2,18	2,6	52	97	–	–
D	128	4.178.960	129,7	0,08	7,71	7,43	2,25	1,5	28	97	–	–
D	192	9.405.456	74,3	0,06	13,47	13,19	2,38	1,0	18	98	–	–
D	208r	11.039.760	64,2	0,13	15,57	15,27	2,40	0,8	16	98	–	–
D	212r	11.468.816	62,3	0,04	16,05	15,76	2,41	0,9	15	98	–	–
▶D	216r	11.906.064	59,6	0,07	16,79	16,50	2,44	0,9	15	98	–	–
D	224r	12.805.136	56,9	0,07	17,56	17,27	2,45	0,9	15	99	–	–
D	256	16.729.104	45,2	0,37	22,10	21,73	2,52	0,7	11	98	–	–
D	320	26.149.904	30,5	0,05	32,83	32,52	2,71	0,6	8	99	–	–
E	16	78.864	316,4	1,03	3,16	2,86	2,15	3,2	65	97	–	–
E	64	1.049.616	234,6	0,52	4,26	3,97	2,17	2,6	52	97	–	–
E	128	4.178.960	128,7	0,85	7,77	7,30	2,30	1,4	27	96	–	–
E	192	9.405.456	72,9	0,04	13,72	13,18	2,41	1,1	18	96	–	–
E	208r	11.039.760	62,6	0,10	15,97	15,38	2,46	0,9	16	96	–	–
E	212r	11.468.816	61,0	0,05	16,40	15,79	2,46	0,9	16	96	–	–
▶E	216r	11.906.064	59,1	0,08	16,91	16,24	2,47	0,9	15	96	–	–
E	224r	12.805.136	55,5	0,09	18,02	17,30	2,50	0,9	14	96	–	–
E	256	16.729.104	42,4	0,07	23,58	23,02	2,56	0,7	11	96	–	–
E	320	26.149.904	29,0	0,30	34,43	33,71	2,68	0,6	7	96	–	–
F	16	78.864	320,4	1,51	3,12	2,81	2,13	3,4	67	97	–	–
F	64	1.049.616	236,7	0,32	4,22	3,92	2,15	2,6	51	97	–	–
F	128	4.178.960	130,1	0,05	7,69	7,40	2,23	1,5	28	97	–	–
F	192	9.405.456	74,4	0,02	13,44	13,15	2,35	1,1	18	98	–	–

Cfg.	T	Triângulos	FPS	dp	Quadro (ms)	GPU (ms)	CPU ativa (ms)	CPU proc. (%)	Thread máx. (%)	GPU 3D (%)	Pot. (W)	Clock (MHz)
F	208r	11.039.760	64,2	0,11	15,57	15,29	2,39	0,8	15	98	–	–
F	216r	11.906.064	60,6	0,03	16,50	16,21	2,39	0,9	14	99	–	–
▶F	220r	12.351.504	58,7	0,11	17,04	16,75	2,41	0,8	14	99	–	–
F	224r	12.805.136	57,1	0,06	17,51	17,21	2,41	0,8	14	99	–	–
F	256	16.729.104	45,6	0,00	21,92	21,63	2,47	0,8	12	99	–	–
F	320	26.149.904	30,1	0,02	33,23	32,92	2,69	0,6	9	99	–	–

Referências

[1] AKENINE-MÖLLER, T. et al. **Real-time rendering**. 4. ed. Boca Raton: CRC Press, 2018.

[2] SHREINER, D. et al. **OpenGL programming guide: the official guide to learning OpenGL, version 4.3**. 8. ed. Boston: Addison-Wesley, 2013.

[3] SEGAL, M.; AKELEY, K. **The OpenGL graphics system: a specification – version 4.6 (compatibility profile)**. The Khronos Group, 2022. Disponível em: `https://registry.khronos.org/OpenGL/specs/gl/glspec46.compatibility.pdf`. Acesso em: 9 nov. 2026.

[4] KESSENICH, J. **The OpenGL Shading Language, version 1.20**. 3Dlabs, 2006. Disponível em: `https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.1.20.pdf`. Acesso em: 9 nov. 2026.

[5] HUGHES, J. F. et al. **Computer graphics: principles and practice**. 3. ed. Upper Saddle River: Addison-Wesley, 2013.

[6] PHONG, B. T. Illumination for computer generated pictures. **Communications of the ACM**, v. 18, n. 6, p. 311–317, 1975.

[7] BLINN, J. F. Models of light reflection for computer synthesized pictures. **ACM SIGGRAPH Computer Graphics**, v. 11, n. 2, p. 192–198, 1977.

[8] WILLIAMS, L. Pyramidal parametrics. **ACM SIGGRAPH Computer Graphics**, v. 17, n. 3, p. 1–11, 1983.

[9] NVIDIA. **GPU Gems**: chapter 28 – Graphics pipeline performance. Disponível em: `https://developer.nvidia.com/gpugems/gpugems/part-v-performance-and-practicalities/chapter-28-graphics-pipeline-performance`. Acesso em: 9 nov. 2026.

[10] THE KHRONOS GROUP. **ARB_timer_query**. Disponível em: `https://registry.khronos.org/OpenGL/extensions/ARB/ARB_timer_query.txt`. Acesso em: 9 nov. 2026.

[11] NVIDIA. **NVML API reference guide**. Disponível em: `https://docs.nvidia.com/deploy/nvml-api/`. Acesso em: 9 nov. 2026.

[12] MICROSOFT. **Performance Counters (PDH)** e **GPU in Task Manager**. Disponível em: `https://learn.microsoft.com/windows/win32/perfctrs/performance-counters-portal`. Acesso em: 9 nov. 2026.

[13] NVIDIA. **Optimus rendering policies** (NvOptimusEnablement). Disponível em: `https://docs.nvidia.com/gameworks/content/technologies/desktop/optimus.htm`. Acesso em: 9 nov. 2026.

[14] FREEGLUT PROJECT. **FreeGLUT 3.8.0**. Disponível em: `https://freeglut.sourceforge.net/`. Acesso em: 9 nov. 2026.

Anexo — Código-Fonte

O projeto é composto pelos arquivos listados a seguir, apresentados na íntegra e sem numeração de linhas.

`CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.20)
project(PDI_Atividade3 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Localização das bibliotecas: informe com -DFreeGLUT_DIR=... (e opcionalmente -DOpenCV_DIR=...)
# ou defina as variáveis de ambiente correspondentes (ver build.bat).
#   FreeGLUT_DIR -> pasta que contém FreeGLUTConfig.cmake (ex.: C:/libs/freeglut/lib/cmake/FreeGLUT)
#   OpenCV_DIR   -> pasta que contém OpenCVConfig.cmake   (ex.: C:/libs/opencv/build)
# O OpenCV é OPCIONAL: é usado apenas para gravar as capturas de tela em PNG.

find_package(OpenGL   REQUIRED)
find_package(FreeGLUT REQUIRED)
find_package(OpenCV QUIET COMPONENTS core imgproc imgcodecs)

set(FONTES
    src/main.cpp
    src/gl_ext.cpp       src/gl_ext.h
    src/malha.cpp        src/malha.h
    src/texturas.cpp     src/texturas.h
    src/shaders.cpp      src/shaders.h
    src/cena.cpp         src/cena.h
    src/monitor.cpp      src/monitor.h
    src/benchmark.cpp    src/benchmark.h)

# Um único projeto, compilado em dois executáveis que diferem apenas na preferência de GPU:
#   desempenho_gl           -> exporta NvOptimusEnablement: usa a GPU dedicada (experimentos principais)
#   desempenho_gl_integrada -> sem a exportação: o Windows/driver escolhe (normalmente a GPU integrada)
foreach(alvo desempenho_gl desempenho_gl_integrada)
    add_executable(${alvo} ${FONTES})
    target_link_libraries(${alvo} PRIVATE FreeGLUT::freeglut OpenGL::GL OpenGL::GLU)
    if(WIN32)
        target_link_libraries(${alvo} PRIVATE pdh dxgi psapi)
    endif()
    if(OpenCV_FOUND)
        target_compile_definitions(${alvo} PRIVATE TEM_OPENCV)
        target_link_libraries(${alvo} PRIVATE ${OpenCV_LIBS})
    endif()
    if(MSVC)
        target_compile_options(${alvo} PRIVATE /utf-8 /W3 /O2)
    endif()
    # Copia as DLLs (freeglut, opencv_world) para junto do executável
    add_custom_command(TARGET ${alvo} POST_BUILD
        COMMAND ${CMAKE_COMMAND} -E copy_if_different
                $<TARGET_RUNTIME_DLLS:${alvo}> $<TARGET_FILE_DIR:${alvo}>
        COMMAND_EXPAND_LISTS)
endforeach()
target_compile_definitions(desempenho_gl PRIVATE PREFERIR_GPU_DEDICADA)

if(OpenCV_FOUND)
    message(STATUS "OpenCV ${OpenCV_VERSION} encontrado: capturas em PNG")
else()
    message(STATUS "OpenCV nao encontrado: capturas em PPM")
endif()
```

`build.bat`

```
@echo off
rem Configura e compila o projeto com o CMake que acompanha o Visual Studio 2022.
rem Ajuste os caminhos abaixo (ou defina as variaveis de ambiente antes de executar).
if not defined FreeGLUT_DIR set "FreeGLUT_DIR=%USERPROFILE%\dev-libs\freeglut\lib\cmake\FreeGLUT"
if not defined OpenCV_DIR   set "OpenCV_DIR=%USERPROFILE%\dev-libs\opencv\build"

set CMAKE="C:\Program Files\Microsoft Visual Studio\2022\Community\Common7\IDE\CommonExtensions\Microsoft\
    CMake\CMake\bin\cmake.exe"
if not exist %CMAKE% set CMAKE=cmake

%CMAKE% -S . -B build -G "Visual Studio 17 2022" -A x64 -DFreeGLUT_DIR="%FreeGLUT_DIR%" -DOpenCV_DIR="%
    OpenCV_DIR%" || exit /b 1
%CMAKE% --build build --config Release || exit /b 1
echo.
echo Executaveis gerados em build\Release
echo   build\Release\desempenho_gl.exe               (modo interativo, GPU dedicada)
echo   build\Release\desempenho_gl.exe --benchmark   (protocolo automatico, ver executar_experimentos.bat)
echo   build\Release\desempenho_gl.exe --capturar    (gera as capturas do relatorio)
echo   build\Release\desempenho_gl_integrada.exe     (mesmo programa, sem preferencia pela GPU dedicada)
```

executar_experimentos.bat

```
@echo off
rem Executa todos os experimentos do relatorio (cerca de 2h30) e gera os graficos.
rem Antes de executar: notebook ligado na tomada, aplicativos em segundo plano fechados,
rem V-Sync do driver em "configuracao do aplicativo" (o programa desliga o V-Sync).
rem Cada ponto = 3 repeticoes x (2 s de estabilizacao + 10 s de medicao), janela 1280x720.
pushd "%~dp0"
set EXE=build\Release\desempenho_gl.exe
set EXEI=build\Release\desempenho_gl_integrada.exe
set SAIDA=resultados\dados

rem Capturas de tela das seis configuracoes e do desenho em arame
%EXE% --capturar

rem E1 (principal): GPU dedicada, VBO, aumento do detalhe T dos 64 orbes, configuracoes A..F
%EXE% --benchmark --rotulo e1_vbo_T --modo vbo --variavel T --N 64 --valores
    16,64,128,192,256,320,384,448,512,640,768 --configs ABCDEF --parada 35 --saida %SAIDA%

rem E2: mesma cena enviada em modo imediato (glBegin/glEnd), configuracoes A..F
%EXE% --benchmark --rotulo e2_imediato_T --modo imediato --variavel T --N 64 --valores 8,16,24,32,48,64 --
    configs ABCDEF --parada 30 --saida %SAIDA%

rem E3: aumento do NUMERO de orbes (T = 8 fixo), uma chamada de desenho por orbe, configuracoes A e F
%EXE% --benchmark --rotulo e3_vbo_N --modo vbo --variavel N --T 8 --valores
    64,256,1024,4096,8192,16384,32768,65536 --configs AF --parada 30 --saida %SAIDA%

rem E4: mesmo protocolo de E1 na GPU integrada (executavel sem preferencia pela GPU dedicada)
%EXEI% --benchmark --rotulo e4_integrada_T --modo vbo --variavel T --N 64 --valores 16,64,128,192,256,320
    --configs ABCDEF --parada 35 --saida %SAIDA%

rem Graficos e tabelas do relatorio
python tools\graficos.py
popd
```

src/gl_ext.h

```
/*
 * gl_ext.h
 * Atividade 3 - Análise de desempenho de CPU e GPU com OpenGL
 *
 * Carregador mínimo das funções do OpenGL posteriores à versão 1.1. No Windows, o
 * opengl32.dll exporta apenas o OpenGL 1.1; as demais funções (shaders GLSL, Vertex Buffer
 * Objects, consultas de tempo da GPU, controle do V-Sync) são obtidas do driver em tempo de
 * execução com glutGetProcAddress. Assim o projeto não depende de GLEW/GLAD.
```

```
 *
 * Cada função é declarada uma única vez na lista LISTA_FUNCOES_GL (técnica "X-macro"):
 * a mesma lista gera o typedef do ponteiro, a declaração extern e o carregamento.
 */
#pragma once

#ifdef _WIN32
#ifndef WIN32_LEAN_AND_MEAN
#define WIN32_LEAN_AND_MEAN
#endif
#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>
#else
#define APIENTRY
#endif

#include <GL/gl.h>

#include <cstddef>
#include <cstdint>
#include <string>

// ---------------------------------------------------------------------------
// Constantes que não existem no gl.h do OpenGL 1.1 (valores do glext.h oficial)
// ---------------------------------------------------------------------------
#define GL_ARRAY_BUFFER_X                 0x8892
#define GL_ELEMENT_ARRAY_BUFFER_X         0x8893
#define GL_STATIC_DRAW_X                  0x88E4
#define GL_FRAGMENT_SHADER_X              0x8B30
#define GL_VERTEX_SHADER_X                0x8B31
#define GL_COMPILE_STATUS_X               0x8B81
#define GL_LINK_STATUS_X                  0x8B82
#define GL_INFO_LOG_LENGTH_X              0x8B84
#define GL_SHADING_LANGUAGE_VERSION_X     0x8B8C
#define GL_TIME_ELAPSED_X                 0x88BF
#define GL_QUERY_RESULT_X                 0x8866
#define GL_QUERY_RESULT_AVAILABLE_X       0x8867
#define GL_GENERATE_MIPMAP_X              0x8191
#define GL_TEXTURE_MAX_ANISOTROPY_X       0x84FE
#define GL_MAX_TEXTURE_MAX_ANISOTROPY_X   0x84FF

typedef char GLcharX;
typedef std::ptrdiff_t GLsizeiptrX;

// ---------------------------------------------------------------------------
// Lista das funções carregadas: X(tipo de retorno, nome, (parâmetros), obrigatória)
// ---------------------------------------------------------------------------
#define LISTA_FUNCOES_GL(X)                                                                  \
    /* Shaders GLSL (OpenGL 2.0) */                                                          \
    X(GLuint, glCreateShader, (GLenum tipo), true)                                           \
    X(void, glShaderSource, (GLuint s, GLsizei n, const GLcharX* const* src, const GLint* tam), true) \
    X(void, glCompileShader, (GLuint s), true)                                               \
    X(void, glGetShaderiv, (GLuint s, GLenum p, GLint* v), true)                             \
    X(void, glGetShaderInfoLog, (GLuint s, GLsizei max, GLsizei* tam, GLcharX* log), true)   \
    X(GLuint, glCreateProgram, (void), true)                                                 \
    X(void, glAttachShader, (GLuint p, GLuint s), true)                                      \
    X(void, glLinkProgram, (GLuint p), true)                                                 \
    X(void, glGetProgramiv, (GLuint p, GLenum q, GLint* v), true)                            \
    X(void, glGetProgramInfoLog, (GLuint p, GLsizei max, GLsizei* tam, GLcharX* log), true)  \
    X(void, glUseProgram, (GLuint p), true)                                                  \
    X(void, glDeleteShader, (GLuint s), true)                                                \
    X(void, glDeleteProgram, (GLuint p), true)                                               \
    X(GLint, glGetUniformLocation, (GLuint p, const GLcharX* nome), true)                    \
    X(void, glUniform1i, (GLint loc, GLint v), true)                                         \
    X(void, glUniform1f, (GLint loc, GLfloat v), true)                                       \
    X(void, glUniform3f, (GLint loc, GLfloat x, GLfloat y, GLfloat z), true)                 \
    /* Vertex Buffer Objects (OpenGL 1.5) */                                                 \
    X(void, glGenBuffers, (GLsizei n, GLuint* b), true)                                      \
    X(void, glDeleteBuffers, (GLsizei n, const GLuint* b), true)                             \
    X(void, glBindBuffer, (GLenum alvo, GLuint b), true)                                     \
    X(void, glBufferData, (GLenum alvo, GLsizeiptrX tam, const void* dados, GLenum uso), true) \
```

```
    /* Consultas de tempo da GPU (OpenGL 1.5 / 3.3) */                                  \
    X(void, glGenQueries, (GLsizei n, GLuint* q), false)                                \
    X(void, glDeleteQueries, (GLsizei n, const GLuint* q), false)                       \
    X(void, glBeginQuery, (GLenum alvo, GLuint q), false)                               \
    X(void, glEndQuery, (GLenum alvo), false)                                           \
    X(void, glGetQueryObjectiv, (GLuint q, GLenum p, GLint* v), false)                  \
    X(void, glGetQueryObjectui64v, (GLuint q, GLenum p, std::uint64_t* v), false)       \
    /* Mipmaps (OpenGL 3.0) */                                                          \
    X(void, glGenerateMipmap, (GLenum alvo), false)                                     \
    /* Intervalo de troca de buffers = V-Sync (WGL_EXT_swap_control) */                 \
    X(int, wglSwapIntervalEXT, (int intervalo), false)                                  \
    X(int, wglGetSwapIntervalEXT, (void), false)

#define GLX_DECLARAR(ret, nome, params, obrig) \
    typedef ret(APIENTRY* PFN_##nome) params;  \
    extern PFN_##nome nome;
LISTA_FUNCOES_GL(GLX_DECLARAR)
#undef GLX_DECLARAR

// Carrega todas as funções. Retorna false se faltar alguma obrigatória (nomes em 'faltando').
bool carregarExtensoesGL(std::string& faltando);

// Recursos opcionais detectados após o carregamento
bool temConsultaTempoGPU();   // GL_TIME_ELAPSED (ARB_timer_query / OpenGL 3.3)
bool temControleVSync();      // wglSwapIntervalEXT
bool temAnisotropia();        // EXT_texture_filter_anisotropic

// Liga/desliga o V-Sync (intervalo 1 ou 0). Retorna o intervalo efetivo (-1 se desconhecido).
int definirVSync(bool ligado);
int consultarVSync();
```

src/gl_ext.cpp

```
/*
 * gl_ext.cpp
 * Carregamento das funções do OpenGL posteriores à versão 1.1 (ver gl_ext.h).
 */
#include "gl_ext.h"

#include <GL/freeglut.h>

#include <cstring>

// Definição dos ponteiros (todos começam nulos)
#define GLX_DEFINIR(ret, nome, params, obrig) PFN_##nome nome = nullptr;
LISTA_FUNCOES_GL(GLX_DEFINIR)
#undef GLX_DEFINIR

static bool g_anisotropia = false;

// Verifica se o nome aparece como palavra inteira na lista de extensões do contexto.
static bool extensaoPresente(const char* nome) {
    const char* lista = reinterpret_cast<const char*>(glGetString(GL_EXTENSIONS));
    if (!lista) return false;
    const size_t n = std::strlen(nome);
    for (const char* p = std::strstr(lista, nome); p; p = std::strstr(p + 1, nome)) {
        const bool inicioOk = (p == lista) || (p[-1] == ' ');
        const bool fimOk = (p[n] == ' ') || (p[n] == '\0');
        if (inicioOk && fimOk) return true;
    }
    return false;
}

bool carregarExtensoesGL(std::string& faltando) {
    faltando.clear();
#define GLX_CARREGAR(ret, nome, params, obrig)                                          \
    nome = reinterpret_cast<PFN_##nome>(glutGetProcAddress(#nome));                     \
    if (!nome && obrig) faltando += std::string(faltando.empty() ? "" : ", ") + #nome;
    LISTA_FUNCOES_GL(GLX_CARREGAR)
#undef GLX_CARREGAR
```

```
    g_anisotropia = extensaoPresente("GL_EXT_texture_filter_anisotropic") ||
                    extensaoPresente("GL_ARB_texture_filter_anisotropic");
    return faltando.empty();
}

bool temConsultaTempoGPU() {
    return glGenQueries && glBeginQuery && glEndQuery && glGetQueryObjectiv && glGetQueryObjectui64v;
}

bool temControleVSync() { return wglSwapIntervalEXT != nullptr; }

bool temAnisotropia() { return g_anisotropia; }

int definirVSync(bool ligado) {
    if (!wglSwapIntervalEXT) return -1;
    wglSwapIntervalEXT(ligado ? 1 : 0);
    return consultarVSync();
}

int consultarVSync() { return wglGetSwapIntervalEXT ? wglGetSwapIntervalEXT() : -1; }
```

src/malha.h

```
/*
 * malha.h
 * Malhas de triângulos da cena: geração procedural, envio para a GPU e desenho.
 *
 * Toda a geometria da aplicação é formada exclusivamente por TRIÂNGULOS indexados
 * (GL_TRIANGLES). A quantidade de polígonos de uma malha é, portanto, exatamente
 * indices.size() / 3 -- não há quadriláteros, leques ou faixas que precisem de conversão.
 *
 * Há dois caminhos de envio da mesma malha ao OpenGL, selecionáveis em tempo de execução:
 *   - VBO      : vértices e índices ficam na memória de vídeo; um glDrawElements por objeto;
 *   - imediato : glBegin/glEnd com glTexCoord/glNormal/glVertex para cada vértice de cada
 *                triângulo, a cada quadro (todo o trabalho de envio recai sobre a CPU).
 */
#pragma once

#include "gl_ext.h"

#include <cstdint>
#include <vector>

struct Vertice {
    float px, py, pz;  // posição
    float nx, ny, nz;  // normal (unitária)
    float u, v;        // coordenadas de textura
};

class Malha {
public:
    std::vector<Vertice> vertices;
    std::vector<std::uint32_t> indices;  // 3 índices por triângulo

    Malha() = default;
    Malha(const Malha&) = delete;
    Malha& operator=(const Malha&) = delete;
    Malha(Malha&& o) noexcept;
    Malha& operator=(Malha&& o) noexcept;
    ~Malha();

    std::uint64_t triangulos() const { return indices.size() / 3; }
    std::size_t bytesGPU() const;  // memória ocupada pelo VBO + IBO

    void enviarParaGPU();  // cria (ou recria) o VBO e o IBO
    void liberarGPU();

    // Desenho em lote: vincular() uma vez, desenharVinculada() para cada objeto que usa
    // esta malha (com a matriz de modelagem já ajustada) e desvincular() ao final.
    void vincular(bool usarVBO) const;
```

```
    void desenharVinculada(bool usarVBO) const;
    void desvincular(bool usarVBO) const;

private:
    GLuint vbo_ = 0, ibo_ = 0;
};

// ---------------------------------------------------------------------------
// Geradores
// ---------------------------------------------------------------------------

// "Orbe": esfera de raio 1 com relevo senoidal (gomos), parametrizada por longitude e
// latitude. T = nível de detalhe: T faixas de latitude e 2T de longitude.
// Os polos usam um só triângulo por faixa (sem triângulos degenerados):
//   triângulos = 2 * (2T) * (T - 1) = 4T(T - 1).
Malha gerarOrbe(int T, float amplitudeRelevo = 0.10f);
std::uint64_t triangulosOrbe(int T);

// Nó toroidal (p, q) = (2, 3) com seção circular. Triângulos = 2 * segmentos * lados.
Malha gerarNoToroidal(int segmentos, int lados);

// Piso quadrado [-tam, tam] x [-tam, tam] em y = 0, dividido em div x div quadrados
// (cada um com 2 triângulos). Triângulos = 2 * div^2.
Malha gerarPiso(int divisoes, float tamanho);
```

src/malha.cpp

```
/*
 * malha.cpp
 * Geração procedural das malhas e desenho por VBO ou modo imediato (ver malha.h).
 */
#include "malha.h"

#include <algorithm>
#include <cmath>
#include <functional>
#include <utility>

namespace {

const float PI = 3.14159265358979f;

struct V3 {
    float x, y, z;
};
V3 operator+(V3 a, V3 b) { return {a.x + b.x, a.y + b.y, a.z + b.z}; }
V3 operator-(V3 a, V3 b) { return {a.x - b.x, a.y - b.y, a.z - b.z}; }
V3 operator*(V3 a, float s) { return {a.x * s, a.y * s, a.z * s}; }
float dot(V3 a, V3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
V3 cross(V3 a, V3 b) { return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x}; }
V3 normalizar(V3 a) {
    const float n = std::sqrt(dot(a, a));
    return n > 1e-20f ? a * (1.0f / n) : V3{0, 1, 0};
}

// Superfície paramétrica P(u, v), u e v em [0, 1], amostrada numa grade de
// (nu + 1) x (nv + 1) vértices. A normal é calculada numericamente como
// dP/du x dP/dv (diferenças centrais), o que dá normais suaves e contínuas na costura.
// Os triângulos são orientados no sentido anti-horário visto pelo lado da normal.
// Com polos = true, as linhas v = 0 e v = 1 colapsam num ponto (esfera): nessas faixas
// é gerado apenas o triângulo não degenerado de cada quadrilátero.
Malha superficie(int nu, int nv, bool polos, float repU, float repV,
                 const std::function<V3(float, float)>& P) {
    Malha m;
    m.vertices.reserve(static_cast<size_t>(nu + 1) * (nv + 1));
    const float h = 1e-3f;
    for (int j = 0; j <= nv; ++j) {
        const float v = static_cast<float>(j) / nv;
        // Nos polos a derivada em u se anula: a normal é estimada um pouco para dentro.
        const float vn = std::clamp(v, 2 * h, 1 - 2 * h);
```

```
        for (int i = 0; i <= nu; ++i) {
            const float u = static_cast<float>(i) / nu;
            const V3 p = P(u, v);
            const V3 du = P(u + h, vn) - P(u - h, vn);
            const V3 dv = P(u, vn + h) - P(u, vn - h);
            const V3 n = normalizar(cross(du, dv));
            m.vertices.push_back({p.x, p.y, p.z, n.x, n.y, n.z, u * repU, v * repV});
        }
    }
    const int linha = nu + 1;
    m.indices.reserve(static_cast<size_t>(nu) * nv * 6);
    for (int j = 0; j < nv; ++j) {
        for (int i = 0; i < nu; ++i) {
            const std::uint32_t a = j * linha + i, b = a + 1;          // (i, j), (i+1, j)
            const std::uint32_t d = (j + 1) * linha + i, c = d + 1;    // (i, j+1), (i+1, j+1)
            if (!(polos && j == 0)) m.indices.insert(m.indices.end(), {a, b, c});
            if (!(polos && j == nv - 1)) m.indices.insert(m.indices.end(), {a, c, d});
        }
    }
    return m;
}

}  // namespace

// ---------------------------------------------------------------------------
// Classe Malha
// ---------------------------------------------------------------------------
Malha::Malha(Malha&& o) noexcept { *this = std::move(o); }

Malha& Malha::operator=(Malha&& o) noexcept {
    if (this != &o) {
        liberarGPU();
        vertices = std::move(o.vertices);
        indices = std::move(o.indices);
        vbo_ = o.vbo_;
        ibo_ = o.ibo_;
        o.vbo_ = o.ibo_ = 0;
    }
    return *this;
}

Malha::~Malha() { liberarGPU(); }

std::size_t Malha::bytesGPU() const {
    return vertices.size() * sizeof(Vertice) + indices.size() * sizeof(std::uint32_t);
}

void Malha::enviarParaGPU() {
    liberarGPU();
    glGenBuffers(1, &vbo_);
    glGenBuffers(1, &ibo_);
    glBindBuffer(GL_ARRAY_BUFFER_X, vbo_);
    glBufferData(GL_ARRAY_BUFFER_X, static_cast<GLsizeiptrX>(vertices.size() * sizeof(Vertice)),
                 vertices.data(), GL_STATIC_DRAW_X);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER_X, ibo_);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER_X,
                 static_cast<GLsizeiptrX>(indices.size() * sizeof(std::uint32_t)), indices.data(),
                 GL_STATIC_DRAW_X);
    glBindBuffer(GL_ARRAY_BUFFER_X, 0);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER_X, 0);
}

void Malha::liberarGPU() {
    if (vbo_ && glDeleteBuffers) glDeleteBuffers(1, &vbo_);
    if (ibo_ && glDeleteBuffers) glDeleteBuffers(1, &ibo_);
    vbo_ = ibo_ = 0;
}

void Malha::vincular(bool usarVBO) const {
    if (!usarVBO) return;  // o modo imediato não usa estado de vértices
    const GLsizei passo = sizeof(Vertice);
    glBindBuffer(GL_ARRAY_BUFFER_X, vbo_);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER_X, ibo_);
```

```
    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_NORMAL_ARRAY);
    glEnableClientState(GL_TEXTURE_COORD_ARRAY);
    glVertexPointer(3, GL_FLOAT, passo, reinterpret_cast<const void*>(offsetof(Vertice, px)));
    glNormalPointer(GL_FLOAT, passo, reinterpret_cast<const void*>(offsetof(Vertice, nx)));
    glTexCoordPointer(2, GL_FLOAT, passo, reinterpret_cast<const void*>(offsetof(Vertice, u)));
}

void Malha::desenharVinculada(bool usarVBO) const {
    if (usarVBO) {
        glDrawElements(GL_TRIANGLES, static_cast<GLsizei>(indices.size()), GL_UNSIGNED_INT, nullptr);
        return;
    }
    // Modo imediato: 3 chamadas glTexCoord/glNormal/glVertex por vértice de cada triângulo.
    glBegin(GL_TRIANGLES);
    for (const std::uint32_t k : indices) {
        const Vertice& p = vertices[k];
        glTexCoord2f(p.u, p.v);
        glNormal3f(p.nx, p.ny, p.nz);
        glVertex3f(p.px, p.py, p.pz);
    }
    glEnd();
}

void Malha::desvincular(bool usarVBO) const {
    if (!usarVBO) return;
    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_NORMAL_ARRAY);
    glDisableClientState(GL_TEXTURE_COORD_ARRAY);
    glBindBuffer(GL_ARRAY_BUFFER_X, 0);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER_X, 0);
}

// ---------------------------------------------------------------------------
// Geradores
// ---------------------------------------------------------------------------
std::uint64_t triangulosOrbe(int T) {
    return 4ull * static_cast<std::uint64_t>(T) * static_cast<std::uint64_t>(T - 1);
}

Malha gerarOrbe(int T, float amplitudeRelevo) {
    T = std::max(T, 2);
    // u -> longitude (0..2pi), v -> colatitude (0 = polo norte, pi = polo sul)
    return superficie(2 * T, T, true, 2.0f, 1.0f, [amplitudeRelevo](float u, float v) {
        const float th = 2 * PI * u, ph = PI * v;
        // Relevo em gomos: 8 ondas na longitude, modulado por sin(ph) para se anular nos polos
        const float r = 1.0f + amplitudeRelevo * std::sin(8 * th) * std::sin(ph) * std::sin(ph) +
                        0.5f * amplitudeRelevo * std::sin(5 * ph);
        return V3{r * std::sin(ph) * std::cos(th), r * std::cos(ph), r * std::sin(ph) * std::sin(th)};
    });
}

Malha gerarNoToroidal(int segmentos, int lados) {
    const float p = 2, q = 3, R = 1.0f, r = 0.42f, tubo = 0.16f;
    auto curva = [=](float t) {
        const float rr = R + r * std::cos(q * t);
        return V3{rr * std::cos(p * t), r * std::sin(q * t), rr * std::sin(p * t)};
    };
    return superficie(segmentos, lados, false, 12.0f, 1.0f, [=](float u, float v) {
        const float t = 2 * PI * u, a = -2 * PI * v, e = 1e-3f;
        // Referencial de Frenet obtido por diferenças finitas da curva
        const V3 c = curva(t), c1 = curva(t + e), c0 = curva(t - e);
        const V3 tan = normalizar(c1 - c0);
        const V3 acel = c1 - c * 2.0f + c0;
        const V3 nor = normalizar(acel - tan * dot(acel, tan));
        const V3 bin = cross(tan, nor);
        return c + (nor * std::cos(a) + bin * std::sin(a)) * tubo;
    });
}

Malha gerarPiso(int divisoes, float tamanho) {
    Malha m;
    const int n = divisoes;
```

```
    for (int j = 0; j <= n; ++j) {
        for (int i = 0; i <= n; ++i) {
            const float x = -tamanho + 2 * tamanho * i / n;
            const float z = -tamanho + 2 * tamanho * j / n;
            m.vertices.push_back({x, 0, z, 0, 1, 0, x / 4.0f, z / 4.0f});
        }
    }
    for (int j = 0; j < n; ++j) {
        for (int i = 0; i < n; ++i) {
            const std::uint32_t a = j * (n + 1) + i, d = a + 1;           // (i,j), (i+1,j)
            const std::uint32_t b = (j + 1) * (n + 1) + i, c = b + 1;     // (i,j+1), (i+1,j+1)
            m.indices.insert(m.indices.end(), {a, b, c, a, c, d});        // normal para +y
        }
    }
    return m;
}
```

src/texturas.h

```
/*
 * texturas.h
 * Texturas procedurais da cena, geradas na CPU na inicialização (nenhum arquivo de imagem
 * é necessário). Todas são RGBA 8 bits, periódicas (sem emendas com GL_REPEAT), com mipmaps,
 * filtragem trilinear e, se disponível, filtragem anisotrópica 8x.
 */
#pragma once

#include "gl_ext.h"

#include <cstdint>
#include <vector>

struct Textura {
    GLuint id = 0;
    int tamanho = 0;
    std::size_t bytes = 0;  // memória estimada na GPU, incluindo mipmaps (~4/3 do nível 0)
};

Textura criarTexturaPiso(int tamanho);     // ladrilhos de pedra com rejunte
Textura criarTexturaMarmore(int tamanho);  // mármore claro com veios (orbes)
Textura criarTexturaListras(int tamanho);  // faixas metálicas trançadas (nó central)

// Envia uma imagem RGBA para a GPU e devolve a textura (usada pelas funções acima).
Textura enviarTextura(const std::vector<std::uint8_t>& rgba, int tamanho);
```

src/texturas.cpp

```
/*
 * texturas.cpp
 * Geração das texturas procedurais com ruído de valor periódico (ver texturas.h).
 */
#include "texturas.h"

#include <algorithm>
#include <cmath>

namespace {

// Pseudoaleatório determinístico em [0, 1) para um ponto inteiro da grade
float hash2(int x, int y, int semente) {
    std::uint32_t h = static_cast<std::uint32_t>(x) * 374761393u + static_cast<std::uint32_t>(y) *
     668265263u +
                      static_cast<std::uint32_t>(semente) * 2246822519u;
    h = (h ^ (h >> 13)) * 1274126177u;
    h ^= h >> 16;
    return (h & 0xFFFFFF) / 16777216.0f;
```

```
}

float suave(float t) { return t * t * (3 - 2 * t); }

// Ruído de valor periódico: a grade de valores se repete a cada 'periodo' células,
// de modo que a textura não apresenta emenda quando repetida.
float ruido(float x, float y, int periodo, int semente) {
    const int x0 = static_cast<int>(std::floor(x)), y0 = static_cast<int>(std::floor(y));
    const float fx = suave(x - x0), fy = suave(y - y0);
    auto v = [&](int i, int j) {
        return hash2(((i % periodo) + periodo) % periodo, ((j % periodo) + periodo) % periodo, semente);
    };
    const float a = v(x0, y0), b = v(x0 + 1, y0), c = v(x0, y0 + 1), d = v(x0 + 1, y0 + 1);
    return (a + (b - a) * fx) * (1 - fy) + (c + (d - c) * fx) * fy;
}

// Soma de oitavas (fractal Brownian motion), resultado aproximadamente em [0, 1]
float fbm(float u, float v, int periodoBase, int oitavas, int semente) {
    float soma = 0, amp = 0.5f, total = 0;
    int periodo = periodoBase;
    for (int o = 0; o < oitavas; ++o) {
        soma += amp * ruido(u * periodo, v * periodo, periodo, semente + 17 * o);
        total += amp;
        amp *= 0.5f;
        periodo *= 2;
    }
    return soma / total;
}

std::uint8_t byte(float x) { return static_cast<std::uint8_t>(std::clamp(x, 0.0f, 1.0f) * 255.0f + 0.5f); }

void pixel(std::vector<std::uint8_t>& img, int tam, int x, int y, float r, float g, float b) {
    const size_t k = (static_cast<size_t>(y) * tam + x) * 4;
    img[k] = byte(r);
    img[k + 1] = byte(g);
    img[k + 2] = byte(b);
    img[k + 3] = 255;
}

}  // namespace

Textura enviarTextura(const std::vector<std::uint8_t>& rgba, int tamanho) {
    Textura t;
    t.tamanho = tamanho;
    t.bytes = static_cast<std::size_t>(tamanho) * tamanho * 4 * 4 / 3;
    glGenTextures(1, &t.id);
    glBindTexture(GL_TEXTURE_2D, t.id);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);  // trilinear
    if (!glGenerateMipmap) glTexParameteri(GL_TEXTURE_2D, GL_GENERATE_MIPMAP_X, GL_TRUE);
    if (temAnisotropia()) {
        GLfloat maximo = 1;
        glGetFloatv(GL_MAX_TEXTURE_MAX_ANISOTROPY_X, &maximo);
        glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAX_ANISOTROPY_X, std::min(8.0f, maximo));
    }
    glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA8, tamanho, tamanho, 0, GL_RGBA, GL_UNSIGNED_BYTE, rgba.data());
    if (glGenerateMipmap) glGenerateMipmap(GL_TEXTURE_2D);
    glBindTexture(GL_TEXTURE_2D, 0);
    return t;
}

Textura criarTexturaPiso(int tam) {
    std::vector<std::uint8_t> img(static_cast<size_t>(tam) * tam * 4);
    const int ladrilhos = 4;  // 4 x 4 ladrilhos por repetição da textura
    for (int y = 0; y < tam; ++y) {
        for (int x = 0; x < tam; ++x) {
            const float u = static_cast<float>(x) / tam, v = static_cast<float>(y) / tam;
            const float lu = u * ladrilhos, lv = v * ladrilhos;
            const int iu = static_cast<int>(lu), iv = static_cast<int>(lv);
            const float fu = lu - iu, fv = lv - iv;
```

```
            // distância até a borda do ladrilho -> rejunte
            const float borda = std::min(std::min(fu, 1 - fu), std::min(fv, 1 - fv));
            const float rejunte = 1.0f - std::clamp((borda - 0.015f) / 0.02f, 0.0f, 1.0f);
            const float variacao = hash2(iu, iv, 7) * 0.25f;  // tom próprio de cada ladrilho
            const float n = fbm(u, v, 8, 5, 3);
            const float base = 0.62f + variacao + 0.30f * (n - 0.5f);
            float r = base * 0.95f, g = base * 0.80f, b = base * 0.62f;  // arenito
            r = r * (1 - rejunte) + 0.22f * rejunte;
            g = g * (1 - rejunte) + 0.20f * rejunte;
            b = b * (1 - rejunte) + 0.19f * rejunte;
            pixel(img, tam, x, y, r, g, b);
        }
    }
    return enviarTextura(img, tam);
}

Textura criarTexturaMarmore(int tam) {
    std::vector<std::uint8_t> img(static_cast<size_t>(tam) * tam * 4);
    for (int y = 0; y < tam; ++y) {
        for (int x = 0; x < tam; ++x) {
            const float u = static_cast<float>(x) / tam, v = static_cast<float>(y) / tam;
            const float n = fbm(u, v, 4, 6, 11);
            // veios: seno da coordenada perturbado pelo ruído (período inteiro em u e v)
            const float veio = std::fabs(std::sin(2 * 3.14159265f * (2 * u + 1 * v) + 9.0f * n));
            const float escuro = std::pow(1.0f - veio, 6.0f);
            const float base = 0.93f - 0.10f * n;
            pixel(img, tam, x, y, base - 0.55f * escuro, base - 0.50f * escuro, base - 0.42f * escuro);
        }
    }
    return enviarTextura(img, tam);
}

Textura criarTexturaListras(int tam) {
    std::vector<std::uint8_t> img(static_cast<size_t>(tam) * tam * 4);
    for (int y = 0; y < tam; ++y) {
        for (int x = 0; x < tam; ++x) {
            const float u = static_cast<float>(x) / tam, v = static_cast<float>(y) / tam;
            const float n = fbm(u, v, 8, 4, 23);
            // faixas diagonais (efeito de cabo trançado) com leve ruído de escovado
            const float faixa = 0.5f + 0.5f * std::sin(2 * 3.14159265f * (4 * u + 2 * v));
            const float claro = 0.55f + 0.45f * std::pow(faixa, 3.0f) + 0.10f * (n - 0.5f);
            pixel(img, tam, x, y, claro, claro * 0.92f, claro * 0.80f);
        }
    }
    return enviarTextura(img, tam);
}
```

src/shaders.h

```
/*
 * shaders.h
 * Programas GLSL da cena. Uma única fonte de shader é compilada em 6 variantes, uma para
 * cada combinação (modo de iluminação x textura), selecionadas por #define. Desse modo cada
 * configuração experimental executa SOMENTE o código de que precisa (não há desvios
 * condicionais em tempo de execução que pudessem mascarar o custo de cada recurso).
 *
 *   Iluminação: 0 = sem iluminação (cor do material)
 *               1 = omnidirecional (point light) com atenuação pela distância
 *               2 = spot (spotlight) com cone interno/externo e atenuação
 *   Textura:    0 = desligada, 1 = ligada (texture2D modulando a cor do material)
 *
 * A iluminação é calculada POR FRAGMENTO (modelo de Blinn-Phong), usando a posição e a normal
 * interpoladas em coordenadas do observador.
 */
#pragma once

#include "gl_ext.h"

#include <string>
```

```
enum class Iluminacao { Nenhuma = 0, Omnidirecional = 1, Spot = 2 };

struct ProgramaCena {
    GLuint id = 0;
    GLint uTextura = -1;        // sampler2D (unidade 0)
    GLint uLuzPos = -1;         // posição da luz (coordenadas do observador)
    GLint uSpotDir = -1;        // direção do eixo do spot (coordenadas do observador)
    GLint uSpotCosExterno = -1; // cosseno do semiângulo externo do cone
    GLint uSpotCosInterno = -1; // cosseno do semiângulo interno (borda suave)
    GLint uEmissivo = -1;       // 1 = objeto emissor (marcador da lâmpada), ignora a luz
};

// Compila as 6 variantes. Retorna false e preenche 'erro' em caso de falha.
bool criarProgramas(ProgramaCena programas[3][2], std::string& erro);
void destruirProgramas(ProgramaCena programas[3][2]);
```

src/shaders.cpp

```
/*
 * shaders.cpp
 * Fontes GLSL 1.20 e compilação das 6 variantes (ver shaders.h).
 *
 * Usa os atributos e matrizes "embutidos" do perfil de compatibilidade (gl_Vertex, gl_Normal,
 * gl_MultiTexCoord0, gl_Color, gl_ModelViewMatrix, gl_NormalMatrix...). Assim o mesmo shader
 * atende tanto ao desenho por VBO (glVertexPointer...) quanto ao modo imediato (glVertex...).
 */
#include "shaders.h"

#include <vector>

namespace {

const char* FONTE_VERTICES = R"GLSL(
#version 120
varying vec3 vPosicao;   // posicao em coordenadas do observador
varying vec3 vNormal;    // normal em coordenadas do observador
varying vec2 vUV;
varying vec4 vCor;
void main() {
    vec4 p = gl_ModelViewMatrix * gl_Vertex;
    vPosicao = p.xyz;
    vNormal = gl_NormalMatrix * gl_Normal;
    vUV = gl_MultiTexCoord0.xy;
    vCor = gl_Color;
    gl_Position = gl_ProjectionMatrix * p;
}
)GLSL";

// ILUMINACAO e TEXTURA são definidos antes desta fonte (ver compilarVariante)
const char* FONTE_FRAGMENTOS = R"GLSL(
varying vec3 vPosicao;
varying vec3 vNormal;
varying vec2 vUV;
varying vec4 vCor;
uniform sampler2D uTextura;
uniform vec3 uLuzPos;
uniform vec3 uSpotDir;
uniform float uSpotCosExterno;
uniform float uSpotCosInterno;
uniform float uEmissivo;

const vec3 LUZ_COR = vec3(1.0, 0.95, 0.85);
const vec3 AMBIENTE = vec3(0.10, 0.10, 0.13);

void main() {
    vec3 base = vCor.rgb;
#if TEXTURA
    base *= texture2D(uTextura, vUV).rgb;
#endif
```

```
#if ILUMINACAO == 0
    vec3 cor = base;
#else
    vec3 N = normalize(vNormal);
    if (!gl_FrontFacing) N = -N;
    vec3 L = uLuzPos - vPosicao;
    float d = length(L);
    L /= d;
    float atenuacao = 1.0 / (1.0 + 0.010 * d + 0.0020 * d * d);
#if ILUMINACAO == 2
    // Spot: intensidade maxima dentro do cone interno, decaindo suavemente ate o externo
    float cosAng = dot(-L, normalize(uSpotDir));
    atenuacao *= smoothstep(uSpotCosExterno, uSpotCosInterno, cosAng);
#endif
    vec3 V = normalize(-vPosicao);
    vec3 H = normalize(L + V);                  // vetor intermediario (Blinn-Phong)
    float difusa = max(dot(N, L), 0.0);
    float especular = difusa > 0.0 ? pow(max(dot(N, H), 0.0), 48.0) : 0.0;
    vec3 cor = base * (AMBIENTE + difusa * atenuacao * LUZ_COR) + 0.45 * especular * atenuacao * LUZ_COR;
#endif

    cor = mix(cor, vCor.rgb, uEmissivo);        // marcador da lampada: cor pura
    gl_FragColor = vec4(cor, 1.0);
}
)GLSL";

GLuint compilar(GLenum tipo, const std::string& fonte, std::string& erro) {
    const GLuint s = glCreateShader(tipo);
    const GLcharX* p = fonte.c_str();
    glShaderSource(s, 1, &p, nullptr);
    glCompileShader(s);
    GLint ok = 0;
    glGetShaderiv(s, GL_COMPILE_STATUS_X, &ok);
    if (!ok) {
        GLint tam = 0;
        glGetShaderiv(s, GL_INFO_LOG_LENGTH_X, &tam);
        std::vector<GLcharX> log(static_cast<size_t>(tam) + 1, 0);
        glGetShaderInfoLog(s, tam, nullptr, log.data());
        erro = std::string(tipo == GL_VERTEX_SHADER_X ? "vertex" : "fragment") + " shader: " + log.data();
        glDeleteShader(s);
        return 0;
    }
    return s;
}

bool compilarVariante(int iluminacao, int textura, ProgramaCena& prog, std::string& erro) {
    const std::string cabecalho = "#version 120\n#define ILUMINACAO " + std::to_string(iluminacao) +
                                  "\n#define TEXTURA " + std::to_string(textura) + "\n";
    const GLuint vs = compilar(GL_VERTEX_SHADER_X, FONTE_VERTICES, erro);
    if (!vs) return false;
    const GLuint fs = compilar(GL_FRAGMENT_SHADER_X, cabecalho + FONTE_FRAGMENTOS, erro);
    if (!fs) {
        glDeleteShader(vs);
        return false;
    }
    prog.id = glCreateProgram();
    glAttachShader(prog.id, vs);
    glAttachShader(prog.id, fs);
    glLinkProgram(prog.id);
    glDeleteShader(vs);  // continuam vivos enquanto anexados ao programa
    glDeleteShader(fs);
    GLint ok = 0;
    glGetProgramiv(prog.id, GL_LINK_STATUS_X, &ok);
    if (!ok) {
        GLint tam = 0;
        glGetProgramiv(prog.id, GL_INFO_LOG_LENGTH_X, &tam);
        std::vector<GLcharX> log(static_cast<size_t>(tam) + 1, 0);
        glGetProgramInfoLog(prog.id, tam, nullptr, log.data());
        erro = std::string("ligacao do programa: ") + log.data();
        return false;
    }
    prog.uTextura = glGetUniformLocation(prog.id, "uTextura");
    prog.uLuzPos = glGetUniformLocation(prog.id, "uLuzPos");
```

```
    prog.uSpotDir = glGetUniformLocation(prog.id, "uSpotDir");
    prog.uSpotCosExterno = glGetUniformLocation(prog.id, "uSpotCosExterno");
    prog.uSpotCosInterno = glGetUniformLocation(prog.id, "uSpotCosInterno");
    prog.uEmissivo = glGetUniformLocation(prog.id, "uEmissivo");
    return true;
}

}  // namespace

bool criarProgramas(ProgramaCena programas[3][2], std::string& erro) {
    for (int luz = 0; luz < 3; ++luz)
        for (int tex = 0; tex < 2; ++tex)
            if (!compilarVariante(luz, tex, programas[luz][tex], erro)) {
                erro = "variante (luz=" + std::to_string(luz) + ", textura=" + std::to_string(tex) + ") " +
      erro;
                return false;
            }
    return true;
}

void destruirProgramas(ProgramaCena programas[3][2]) {
    for (int luz = 0; luz < 3; ++luz)
        for (int tex = 0; tex < 2; ++tex)
            if (programas[luz][tex].id) {
                glDeleteProgram(programas[luz][tex].id);
                programas[luz][tex].id = 0;
            }
}
```

src/cena.h

```
/*
 * cena.h
 * Cena "Jardim de Orbes": um piso de ladrilhos, um nó toroidal dourado girando no centro,
 * N orbes de mármore distribuídos em espiral filotáxica (como sementes de girassol) que
 * giram em carrossel em torno do nó, cada um girando sobre si e oscilando na vertical, e
 * uma lâmpada (esfera emissiva) que marca a posição da fonte de luz.
 *
 * A carga geométrica é controlada por dois parâmetros independentes:
 *   T = nível de detalhe de cada orbe   -> 4T(T-1) triângulos por orbe;
 *   N = número de orbes                  -> N chamadas de desenho (glDrawElements).
 * Triângulos por quadro = cenário fixo (piso + nó + lâmpada) + N * 4T(T-1).
 *
 * As seis configurações experimentais (A..F) combinam o modo de iluminação e a textura:
 *   A: sem luz, sem textura    B: omnidirecional, sem textura    C: spot, sem textura
 *   D: sem luz, com textura    E: omnidirecional, com textura    F: spot, com textura
 */
#pragma once

#include "malha.h"
#include "shaders.h"
#include "texturas.h"

#include <cstdint>
#include <string>
#include <vector>

// Limites dos parâmetros de carga
const int T_MIN = 2, T_MAX = 1024;
const int N_MIN = 1, N_MAX = 65536;

Iluminacao iluminacaoDaConfig(char config);
bool texturaDaConfig(char config);
char configDe(Iluminacao luz, bool textura);
std::string descreverConfig(char config);  // ex.: "C (spot, sem textura)"
std::string nomeIluminacao(Iluminacao luz);

struct EstadoCena {
    char config = 'A';
    int T = 32;            // detalhe dos orbes
```

```
    int N = 64;            // número de orbes
    bool usarVBO = true;   // false = modo imediato (glBegin/glEnd)
    bool aramado = false;  // desenho em arame (glPolygonMode GL_LINE)
};

class Cena {
public:
    bool inicializar(std::string& erro);  // requer contexto OpenGL ativo
    void finalizar();

    void definirConfig(char config);
    void definirDetalhe(int T);  // recria a malha do orbe se T mudou
    void definirObjetos(int N);  // recalcula as posições dos orbes
    void definirVBO(bool usar) { estado_.usarVBO = usar; }
    void definirAramado(bool a) { estado_.aramado = a; }
    const EstadoCena& estado() const { return estado_; }

    // Desenha um quadro completo. 'tempo' = instante da animação (s).
    void desenhar(double tempo, int largura, int altura);

    // Contagem de triângulos submetidos por quadro
    std::uint64_t triangulosCenario() const;          // piso + nó + lâmpada
    std::uint64_t triangulosPorOrbe() const { return orbe_.triangulos(); }
    std::uint64_t triangulosPorQuadro() const;
    static std::uint64_t triangulosPara(int T, int N);  // sem precisar gerar a malha
    int chamadasDesenhoPorQuadro() const { return 3 + estado_.N; }
    double memoriaGeometriaMB() const;
    double memoriaTexturasMB() const;

private:
    struct Orbe {
        float x, z, escala, fase, r, g, b;
    };
    EstadoCena estado_;
    Malha orbe_, piso_, no_, lampada_;
    Textura texPiso_, texMarmore_, texListras_;
    ProgramaCena programas_[3][2];
    std::vector<Orbe> orbes_;
};
```

src/cena.cpp

```
/*
 * cena.cpp
 * Construção, animação e desenho da cena (ver cena.h).
 */
#include "cena.h"

#include <GL/freeglut.h>

#include <algorithm>
#include <cmath>

namespace {

const float PI = 3.14159265358979f;
const float RAIO_INTERNO = 3.2f;   // espaço livre ao redor do nó central
const float RAIO_EXTERNO = 10.0f;  // raio do campo de orbes
const int DIVISOES_PISO = 48;
const int SEGMENTOS_NO = 256, LADOS_NO = 24;
const int T_LAMPADA = 12;
const int TAMANHO_TEXTURA = 1024;

void hsvParaRgb(float h, float s, float v, float& r, float& g, float& b) {
    const float i = std::floor(h * 6), f = h * 6 - i;
    const float p = v * (1 - s), q = v * (1 - f * s), t = v * (1 - (1 - f) * s);
    switch (static_cast<int>(i) % 6) {
        case 0: r = v, g = t, b = p; break;
        case 1: r = q, g = v, b = p; break;
        case 2: r = p, g = v, b = t; break;
```

```
        case 3: r = p, g = q, b = v; break;
        case 4: r = t, g = p, b = v; break;
        default: r = v, g = p, b = q; break;
    }
}

// Transforma um ponto (w = 1) ou uma direção (w = 0) pela matriz 4x4 (ordem de colunas do OpenGL)
void transformar(const float M[16], const float in[3], float w, float out[3]) {
    for (int i = 0; i < 3; ++i) out[i] = M[i] * in[0] + M[4 + i] * in[1] + M[8 + i] * in[2] + M[12 + i] * w
     ;
}

}  // namespace

// -----------------------------------------------------------------------------
// Configurações A..F
// -----------------------------------------------------------------------------
Iluminacao iluminacaoDaConfig(char c) {
    switch (c) {
        case 'B': case 'E': return Iluminacao::Omnidirecional;
        case 'C': case 'F': return Iluminacao::Spot;
        default: return Iluminacao::Nenhuma;
    }
}

bool texturaDaConfig(char c) { return c == 'D' || c == 'E' || c == 'F'; }

char configDe(Iluminacao luz, bool textura) {
    static const char tabela[3][2] = {{'A', 'D'}, {'B', 'E'}, {'C', 'F'}};
    return tabela[static_cast<int>(luz)][textura ? 1 : 0];
}

std::string nomeIluminacao(Iluminacao luz) {
    switch (luz) {
        case Iluminacao::Omnidirecional: return "omnidirecional";
        case Iluminacao::Spot: return "spot";
        default: return "sem iluminacao";
    }
}

std::string descreverConfig(char c) {
    return std::string(1, c) + " (" + nomeIluminacao(iluminacaoDaConfig(c)) + ", " +
           (texturaDaConfig(c) ? "com textura" : "sem textura") + ")";
}

// -----------------------------------------------------------------------------
// Cena
// -----------------------------------------------------------------------------
bool Cena::inicializar(std::string& erro) {
    if (!criarProgramas(programas_, erro)) return false;
    texPiso_ = criarTexturaPiso(TAMANHO_TEXTURA);
    texMarmore_ = criarTexturaMarmore(TAMANHO_TEXTURA);
    texListras_ = criarTexturaListras(TAMANHO_TEXTURA);
    piso_ = gerarPiso(DIVISOES_PISO, RAIO_EXTERNO + 1.5f);
    piso_.enviarParaGPU();
    no_ = gerarNoToroidal(SEGMENTOS_NO, LADOS_NO);
    no_.enviarParaGPU();
    lampada_ = gerarOrbe(T_LAMPADA, 0.0f);
    lampada_.enviarParaGPU();
    const int T = estado_.T;
    estado_.T = -1;  // força a geração
    definirDetalhe(T);
    definirObjetos(estado_.N);
    return true;
}

void Cena::finalizar() {
    destruirProgramas(programas_);
    for (Textura* t : {&texPiso_, &texMarmore_, &texListras_})
        if (t->id) glDeleteTextures(1, &t->id), t->id = 0;
    orbe_.liberarGPU();
    piso_.liberarGPU();
    no_.liberarGPU();
```

```
    lampada_.liberarGPU();
}

void Cena::definirConfig(char c) {
    if (c >= 'A' && c <= 'F') estado_.config = c;
}

void Cena::definirDetalhe(int T) {
    T = std::clamp(T, T_MIN, T_MAX);
    if (T == estado_.T) return;
    estado_.T = T;
    orbe_ = gerarOrbe(T);
    orbe_.enviarParaGPU();
    // A cópia na RAM continua necessária para o modo imediato.
}

void Cena::definirObjetos(int N) {
    N = std::clamp(N, N_MIN, N_MAX);
    estado_.N = N;
    orbes_.clear();
    // Espiral filotáxica: ângulo de ouro entre orbes consecutivos e raio ~ sqrt(i), o que
    // distribui N orbes com densidade uniforme no anel [RAIO_INTERNO, RAIO_EXTERNO].
    const float anguloOuro = PI * (3.0f - std::sqrt(5.0f));
    const float area = PI * (RAIO_EXTERNO * RAIO_EXTERNO - RAIO_INTERNO * RAIO_INTERNO) / N;
    const float escala = std::min(0.9f, 0.42f * std::sqrt(area));  // orbes menores quando N cresce
    for (int i = 0; i < N; ++i) {
        const float raio = std::sqrt(RAIO_INTERNO * RAIO_INTERNO +
                                     (RAIO_EXTERNO * RAIO_EXTERNO - RAIO_INTERNO * RAIO_INTERNO) * (i + 0.5
    f) / N);
        const float ang = i * anguloOuro;
        Orbe o{raio * std::cos(ang), raio * std::sin(ang), escala, 0.7f * i, 0, 0, 0};
        hsvParaRgb(std::fmod(i * 0.618034f, 1.0f), 0.50f, 0.95f, o.r, o.g, o.b);
        orbes_.push_back(o);
    }
}

std::uint64_t Cena::triangulosPara(int T, int N) {
    const std::uint64_t cenario = 2ull * DIVISOES_PISO * DIVISOES_PISO + 2ull * SEGMENTOS_NO * LADOS_NO +
                                  triangulosOrbe(T_LAMPADA);
    return cenario + static_cast<std::uint64_t>(N) * triangulosOrbe(T);
}

std::uint64_t Cena::triangulosCenario() const {
    return piso_.triangulos() + no_.triangulos() + lampada_.triangulos();
}

std::uint64_t Cena::triangulosPorQuadro() const {
    return triangulosCenario() + static_cast<std::uint64_t>(estado_.N) * orbe_.triangulos();
}

double Cena::memoriaGeometriaMB() const {
    return (orbe_.bytesGPU() + piso_.bytesGPU() + no_.bytesGPU() + lampada_.bytesGPU()) / 1048576.0;
}

double Cena::memoriaTexturasMB() const {
    return (texPiso_.bytes + texMarmore_.bytes + texListras_.bytes) / 1048576.0;
}

void Cena::desenhar(double tempo, int largura, int altura) {
    const float t = static_cast<float>(tempo);
    const bool vbo = estado_.usarVBO;
    const Iluminacao luz = iluminacaoDaConfig(estado_.config);
    const bool tex = texturaDaConfig(estado_.config);
    const ProgramaCena& prog = programas_[static_cast<int>(luz)][tex ? 1 : 0];

    glViewport(0, 0, largura, altura);
    glClearColor(0.06f, 0.07f, 0.10f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_CULL_FACE);  // descarta faces traseiras (após o processamento de vértices)
    glCullFace(GL_BACK);
    glPolygonMode(GL_FRONT_AND_BACK, estado_.aramado ? GL_LINE : GL_FILL);
```

```
    // Câmera fixa
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(50.0, static_cast<double>(largura) / std::max(altura, 1), 0.5, 100.0);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    gluLookAt(0.0, 12.5, 19.5, 0.0, 0.8, 0.0, 0.0, 1.0, 0.0);
    float V[16];
    glGetFloatv(GL_MODELVIEW_MATRIX, V);

    // Fontes de luz (coordenadas do mundo), animadas
    //  - omnidirecional: orbita o centro a 6,5 unidades de raio e 4,5 de altura;
    //  - spot: a 7 unidades de altura, varrendo o piso com um cone de 26 graus.
    const float luzPonto[3] = {6.5f * std::cos(0.6f * t), 4.5f, 6.5f * std::sin(0.6f * t)};
    const float spotPos[3] = {2.5f * std::cos(0.25f * t), 7.0f, 2.5f * std::sin(0.25f * t)};
    const float alvo[3] = {6.0f * std::cos(0.45f * t + 1.0f), 0.0f, 6.0f * std::sin(0.45f * t + 1.0f)};
    const float spotDir[3] = {alvo[0] - spotPos[0], alvo[1] - spotPos[1], alvo[2] - spotPos[2]};
    const float* posLuz = (luz == Iluminacao::Spot) ? spotPos : luzPonto;
    float luzOlho[3], dirOlho[3];
    transformar(V, posLuz, 1.0f, luzOlho);
    transformar(V, spotDir, 0.0f, dirOlho);

    glUseProgram(prog.id);
    glUniform1i(prog.uTextura, 0);
    glUniform3f(prog.uLuzPos, luzOlho[0], luzOlho[1], luzOlho[2]);
    glUniform3f(prog.uSpotDir, dirOlho[0], dirOlho[1], dirOlho[2]);
    glUniform1f(prog.uSpotCosExterno, std::cos(26.0f * PI / 180.0f));
    glUniform1f(prog.uSpotCosInterno, std::cos(18.0f * PI / 180.0f));
    glUniform1f(prog.uEmissivo, 0.0f);

    // 1) Piso
    glBindTexture(GL_TEXTURE_2D, tex ? texPiso_.id : 0);
    glColor3f(0.85f, 0.85f, 0.85f);
    piso_.vincular(vbo);
    piso_.desenharVinculada(vbo);
    piso_.desvincular(vbo);

    // 2) Nó toroidal central (animação: rotação em dois eixos)
    glBindTexture(GL_TEXTURE_2D, tex ? texListras_.id : 0);
    glColor3f(1.0f, 0.78f, 0.32f);
    glPushMatrix();
    glTranslatef(0.0f, 2.4f + 0.25f * std::sin(1.3f * t), 0.0f);
    glRotatef(30.0f * t, 0, 1, 0);
    glRotatef(17.0f * t, 1, 0, 0);
    glScalef(1.7f, 1.7f, 1.7f);
    no_.vincular(vbo);
    no_.desenharVinculada(vbo);
    no_.desvincular(vbo);
    glPopMatrix();

    // 3) Orbes (carrossel + rotação própria + oscilação vertical), uma chamada por orbe
    glBindTexture(GL_TEXTURE_2D, tex ? texMarmore_.id : 0);
    orbe_.vincular(vbo);
    glPushMatrix();
    glRotatef(8.0f * t, 0, 1, 0);
    for (const Orbe& o : orbes_) {
        glPushMatrix();
        glTranslatef(o.x, o.escala * (1.15f + 0.25f * std::sin(2.0f * t + o.fase)), o.z);
        glRotatef(40.0f * t + 57.0f * o.fase, 0, 1, 0);
        glScalef(o.escala, o.escala, o.escala);
        glColor3f(o.r, o.g, o.b);
        orbe_.desenharVinculada(vbo);
        glPopMatrix();
    }
    glPopMatrix();
    orbe_.desvincular(vbo);

    // 4) Lâmpada: esfera emissiva na posição da luz (desenhada em todas as configurações,
    //    para que a contagem de triângulos seja idêntica entre A..F)
    glBindTexture(GL_TEXTURE_2D, 0);
    glUniform1f(prog.uEmissivo, 1.0f);
    glColor3f(1.0f, 0.96f, 0.70f);
    glPushMatrix();
```

```
    glTranslatef(posLuz[0], posLuz[1], posLuz[2]);
    glScalef(0.28f, 0.28f, 0.28f);
    lampada_.vincular(vbo);
    lampada_.desenharVinculada(vbo);
    lampada_.desvincular(vbo);
    glPopMatrix();

    glUseProgram(0);
    glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
}
```

src/monitor.h

```
/*
 * monitor.h
 * Monitoramento da utilização de CPU, GPU e memória durante a execução (Windows).
 *
 * Uma thread auxiliar coleta amostras a cada 500 ms (fora da thread de renderização, para
 * não perturbar os quadros) a partir de:
 *   - GetProcessTimes / GetThreadTimes / GetSystemTimes: CPU do processo, da thread de
 *     renderização (a thread principal, onde roda o FreeGLUT) e do sistema;
 *   - Toolhelp32 + GetThreadTimes: tempo de CPU de CADA thread do processo, para achar a
 *     thread mais ocupada (o driver OpenGL cria threads próprias de processamento);
 *   - PDH "\Processor(*)\% Processor Time": utilização de cada núcleo lógico;
 *   - PDH "\GPU Engine(*)\Utilization Percentage": utilização de cada motor de cada GPU,
 *     por processo -- é a mesma fonte usada pelo Gerenciador de Tarefas do Windows. Permite
 *     saber QUAL GPU executa a aplicação e acompanhar separadamente cada GPU;
 *   - PDH "\GPU Process Memory(*)\Dedicated/Shared Usage": memória de vídeo do processo;
 *   - NVML (nvml.dll, carregada dinamicamente, opcional): utilização, controlador de
 *     memória, potência, temperatura, clocks e motivos de redução de clock da GPU NVIDIA;
 *   - DXGI: lista de adaptadores gráficos (nome, fabricante, VRAM, LUID).
 *
 * As amostras são acumuladas numa "janela de medição" (reiniciarJanela / mediaJanela),
 * usada pelo benchmark para obter a média de cada medição de 10 s.
 */
#pragma once

#include <cstdint>
#include <memory>
#include <string>
#include <vector>

struct AdaptadorGPU {
    std::string nome;
    unsigned fabricante = 0;          // 0x10DE NVIDIA, 0x8086 Intel, 0x1002 AMD
    std::uint64_t luid = 0;           // identificador do adaptador (HighPart << 32 | LowPart)
    double vramDedicadaMB = 0;        // memória de vídeo dedicada
    double memoriaCompartilhadaMB = 0;
    bool integrada() const { return fabricante == 0x8086 && vramDedicadaMB < 1024; }
};

// Índices fixos usados nas métricas por GPU
enum { GPU_NVIDIA = 0, GPU_INTEL = 1, GPU_OUTRA = 2, NUM_GRUPOS_GPU = 3 };

struct Metricas {
    int amostras = 0;
    // CPU (%)
    double cpuProcesso = 0;       // fração da CPU inteira (todos os núcleos lógicos) usada pelo processo
    double cpuThreadRender = 0;   // fração de UM núcleo lógico usada pela thread de renderização
    double cpuThreadMax = 0;      // fração de UM núcleo usada pela thread MAIS ocupada do processo
                                  // (pode ser uma thread do driver OpenGL, e não a thread principal)
    double threadsProcesso = 0;   // número de threads do processo
    double cpuSistema = 0;        // utilização total do sistema
    double cpuNucleoMax = 0;      // maior utilização entre os núcleos lógicos
    // GPU (%) por grupo (NVIDIA / Intel / outra)
    double gpu3dProcesso[NUM_GRUPOS_GPU] = {};  // motor 3D usado por ESTE processo
    double gpuSistema[NUM_GRUPOS_GPU] = {};     // motor mais ocupado da GPU (todos os processos)
    double gpuMemProcessoMB[NUM_GRUPOS_GPU] = {};  // memória de vídeo (dedicada + compartilhada) do
     processo
    // NVML (GPU NVIDIA)
```

```
    double nvUtil = 0, nvMemUtil = 0, nvPotenciaW = 0, nvTempC = 0;
    double nvClockSM = 0, nvClockMem = 0, nvVramUsadaMB = 0;
    std::uint64_t nvMotivosReducao = 0;  // OU bit a bit dos motivos de redução de clock observados
    // Memória do sistema
    double ramProcessoMB = 0;    // working set do processo
    double ramSistemaPct = 0;    // carga de memória do sistema
};

class Monitor {
public:
    Monitor();
    ~Monitor();
    Monitor(const Monitor&) = delete;
    Monitor& operator=(const Monitor&) = delete;

    // Deve ser chamado pela thread de renderização (registra o handle dessa thread).
    void iniciar();
    void parar();

    void reiniciarJanela();          // zera os acumuladores
    Metricas mediaJanela() const;    // médias desde o último reiniciarJanela()
    Metricas ultima() const;         // amostra mais recente (para o painel na tela)

    const std::vector<AdaptadorGPU>& adaptadores() const;
    bool temNVML() const;
    std::string nomeGPUNVML() const;

    static int grupoDoFabricante(unsigned fabricante);
    static std::string descreverMotivosReducao(std::uint64_t mascara);

private:
    struct Impl;
    std::unique_ptr<Impl> impl_;
};

// Soma de amostras e média (usadas pela janela de medição e pelas repetições do benchmark)
void acumularMetricas(Metricas& soma, const Metricas& m);  // soma campo a campo; amostras += 1
Metricas mediaMetricas(Metricas soma);                     // divide pela quantidade de amostras

// Informações estáticas do sistema (para o registro dos experimentos)
std::string nomeCPU();
int nucleosFisicos();
int processadoresLogicos();
double memoriaRAMTotalGB();
std::string versaoWindows();
```

src/monitor.cpp

```
/*
 * monitor.cpp
 * Coleta periódica de utilização de CPU, GPU e memória (ver monitor.h).
 */
#include "monitor.h"

#include <algorithm>
#include <chrono>
#include <condition_variable>
#include <cstring>
#include <map>
#include <mutex>
#include <thread>

#ifdef _WIN32
#ifndef WIN32_LEAN_AND_MEAN
#define WIN32_LEAN_AND_MEAN
#endif
#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>
```

```
#include <dxgi.h>
#include <pdh.h>
#include <pdhmsg.h>
#include <psapi.h>
#include <tlhelp32.h>
#include <cwchar>

namespace {

std::string utf8(const wchar_t* w) {
    const int n = WideCharToMultiByte(CP_UTF8, 0, w, -1, nullptr, 0, nullptr, nullptr);
    std::string s(n > 0 ? n - 1 : 0, '\0');
    if (n > 1) WideCharToMultiByte(CP_UTF8, 0, w, -1, s.data(), n, nullptr, nullptr);
    return s;
}

std::uint64_t paraU64(const FILETIME& f) {
    return (static_cast<std::uint64_t>(f.dwHighDateTime) << 32) | f.dwLowDateTime;
}

// Lê todas as instâncias de um contador PDH com curinga "(*)".
struct ItemPDH {
    std::wstring nome;
    double valor;
};
std::vector<ItemPDH> lerArray(PDH_HCOUNTER c) {
    std::vector<ItemPDH> itens;
    if (!c) return itens;
    DWORD bytes = 0, n = 0;
    PDH_STATUS st = PdhGetFormattedCounterArrayW(c, PDH_FMT_DOUBLE | PDH_FMT_NOCAP100, &bytes, &n, nullptr)
     ;
    if (st != PDH_MORE_DATA || bytes == 0) return itens;
    std::vector<unsigned char> buf(bytes);
    auto* arr = reinterpret_cast<PDH_FMT_COUNTERVALUE_ITEM_W*>(buf.data());
    st = PdhGetFormattedCounterArrayW(c, PDH_FMT_DOUBLE | PDH_FMT_NOCAP100, &bytes, &n, arr);
    if (st != ERROR_SUCCESS) return itens;
    itens.reserve(n);
    for (DWORD i = 0; i < n; ++i) {
        const DWORD cs = arr[i].FmtValue.CStatus;
        if (cs == PDH_CSTATUS_VALID_DATA || cs == PDH_CSTATUS_NEW_DATA)
            itens.push_back({arr[i].szName, arr[i].FmtValue.doubleValue});
    }
    return itens;
}

// Extrai pid e LUID de nomes como "pid_1234_luid_0x00000000_0x0000D1E6_phys_0_eng_3_engtype_3D"
bool lerPidLuid(const std::wstring& nome, unsigned& pid, std::uint64_t& luid) {
    unsigned alto = 0, baixo = 0;
#ifdef _MSC_VER
    if (swscanf_s(nome.c_str(), L"pid_%u_luid_0x%x_0x%x", &pid, &alto, &baixo) != 3) return false;
#else
    if (std::swscanf(nome.c_str(), L"pid_%u_luid_0x%x_0x%x", &pid, &alto, &baixo) != 3) return false;
#endif
    luid = (static_cast<std::uint64_t>(alto) << 32) | baixo;
    return true;
}

// Assinaturas mínimas da NVML (NVIDIA Management Library)
struct NvUtilizacao {
    unsigned gpu, memoria;
};
struct NvMemoria {
    unsigned long long total, livre, usada;
};
typedef int (*NvFnInit)();
typedef int (*NvFnHandle)(unsigned, void**);
typedef int (*NvFnUtil)(void*, NvUtilizacao*);
typedef int (*NvFnU32)(void*, unsigned*);
typedef int (*NvFnTipoU32)(void*, int, unsigned*);
typedef int (*NvFnMem)(void*, NvMemoria*);
typedef int (*NvFnU64)(void*, unsigned long long*);
typedef int (*NvFnNome)(void*, char*, unsigned);
```

```
} // namespace

// ---------------------------------------------------------------------------
// Implementação
// ---------------------------------------------------------------------------
struct Monitor::Impl {
    std::vector<AdaptadorGPU> adaptadores;
    DWORD pid = GetCurrentProcessId();
    int logicos = 1;

    HANDLE threadRender = nullptr;
    std::thread trabalhador;
    bool rodando = false;
    mutable std::mutex mtx;
    std::condition_variable cv;
    Metricas soma, ultima;

    // PDH
    PDH_HQUERY consulta = nullptr;
    PDH_HCOUNTER cMotor = nullptr, cMemDed = nullptr, cMemComp = nullptr, cNucleos = nullptr;

    // Tempos da amostra anterior
    std::uint64_t procAnt = 0, threadAnt = 0, idleAnt = 0, kernAnt = 0, userAnt = 0;
    std::map<DWORD, std::uint64_t> tempoThreads;  // tempo de CPU acumulado de cada thread
    std::chrono::steady_clock::time_point relogioAnt;
    bool primeira = true;

    // NVML
    HMODULE nvml = nullptr;
    void* nvDisp = nullptr;
    std::string nvNome;
    NvFnUtil nvUtil = nullptr;
    NvFnU32 nvPotencia = nullptr;
    NvFnTipoU32 nvTemp = nullptr, nvClock = nullptr;
    NvFnMem nvMem = nullptr;
    NvFnU64 nvMotivos = nullptr;
    NvFnInit nvShutdown = nullptr;

    void enumerarAdaptadores() {
        IDXGIFactory1* fab = nullptr;
        if (FAILED(CreateDXGIFactory1(__uuidof(IDXGIFactory1), reinterpret_cast<void**>(&fab)))) return;
        IDXGIAdapter1* ad = nullptr;
        for (UINT i = 0; fab->EnumAdapters1(i, &ad) != DXGI_ERROR_NOT_FOUND; ++i) {
            DXGI_ADAPTER_DESC1 d{};
            ad->GetDesc1(&d);
            ad->Release();
            if (d.Flags & DXGI_ADAPTER_FLAG_SOFTWARE) continue;  // ignora o "Microsoft Basic Render"
            AdaptadorGPU a;
            a.nome = utf8(d.Description);
            a.fabricante = d.VendorId;
            a.luid = (static_cast<std::uint64_t>(static_cast<std::uint32_t>(d.AdapterLuid.HighPart)) << 32)
     |
                     d.AdapterLuid.LowPart;
            a.vramDedicadaMB = d.DedicatedVideoMemory / 1048576.0;
            a.memoriaCompartilhadaMB = d.SharedSystemMemory / 1048576.0;
            adaptadores.push_back(a);
        }
        fab->Release();
    }

    int grupoDoLuid(std::uint64_t luid) const {
        for (const auto& a : adaptadores)
            if (a.luid == luid) return grupoDoFabricante(a.fabricante);
        return GPU_OUTRA;
    }

    void abrirPDH() {
        if (PdhOpenQueryW(nullptr, 0, &consulta) != ERROR_SUCCESS) {
            consulta = nullptr;
            return;
        }
        PdhAddEnglishCounterW(consulta, L"\\GPU Engine(*)\\Utilization Percentage", 0, &cMotor);
        PdhAddEnglishCounterW(consulta, L"\\GPU Process Memory(*)\\Dedicated Usage", 0, &cMemDed);
```

```
        PdhAddEnglishCounterW(consulta, L"\\GPU Process Memory(*)\\Shared Usage", 0, &cMemComp);
        PdhAddEnglishCounterW(consulta, L"\\Processor(*)\\% Processor Time", 0, &cNucleos);
        PdhCollectQueryData(consulta);
    }

    void abrirNVML() {
        nvml = LoadLibraryW(L"nvml.dll");
        if (!nvml) nvml = LoadLibraryW(L"C:\\Program Files\\NVIDIA Corporation\\NVSMI\\nvml.dll");
        if (!nvml) return;
        auto fn = [this](const char* n) { return reinterpret_cast<void*>(GetProcAddress(nvml, n)); };
        auto init = reinterpret_cast<NvFnInit>(fn("nvmlInit_v2"));
        auto handle = reinterpret_cast<NvFnHandle>(fn("nvmlDeviceGetHandleByIndex_v2"));
        auto nome = reinterpret_cast<NvFnNome>(fn("nvmlDeviceGetName"));
        nvShutdown = reinterpret_cast<NvFnInit>(fn("nvmlShutdown"));
        nvUtil = reinterpret_cast<NvFnUtil>(fn("nvmlDeviceGetUtilizationRates"));
        nvPotencia = reinterpret_cast<NvFnU32>(fn("nvmlDeviceGetPowerUsage"));
        nvTemp = reinterpret_cast<NvFnTipoU32>(fn("nvmlDeviceGetTemperature"));
        nvClock = reinterpret_cast<NvFnTipoU32>(fn("nvmlDeviceGetClockInfo"));
        nvMem = reinterpret_cast<NvFnMem>(fn("nvmlDeviceGetMemoryInfo"));
        nvMotivos = reinterpret_cast<NvFnU64>(fn("nvmlDeviceGetCurrentClocksEventReasons"));
        if (!nvMotivos) nvMotivos = reinterpret_cast<NvFnU64>(fn("nvmlDeviceGetCurrentClocksThrottleReasons
    "));
        if (!init || !handle || init() != 0 || handle(0, &nvDisp) != 0) {
            nvDisp = nullptr;
            FreeLibrary(nvml);
            nvml = nullptr;
            return;
        }
        char buf[96] = {};
        if (nome && nome(nvDisp, buf, sizeof(buf)) == 0) nvNome = buf;
    }

    // Uma amostra de todas as fontes
    Metricas amostrar() {
        Metricas m;
        const auto agora = std::chrono::steady_clock::now();
        const double dt100ns = std::chrono::duration<double>(agora - relogioAnt).count() * 1e7;

        FILETIME c, e, k, u, idle, kern, user;
        GetProcessTimes(GetCurrentProcess(), &c, &e, &k, &u);
        const std::uint64_t proc = paraU64(k) + paraU64(u);
        std::uint64_t thr = threadAnt;
        if (threadRender && GetThreadTimes(threadRender, &c, &e, &k, &u)) thr = paraU64(k) + paraU64(u);
        GetSystemTimes(&idle, &kern, &user);  // o tempo de kernel inclui o tempo ocioso
        const std::uint64_t i0 = paraU64(idle), k0 = paraU64(kern), u0 = paraU64(user);

        // Todas as threads do processo: a mais ocupada indica saturação de um único núcleo
        std::map<DWORD, std::uint64_t> novos;
        const HANDLE foto = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
        if (foto != INVALID_HANDLE_VALUE) {
            THREADENTRY32 te{};
            te.dwSize = sizeof(te);
            for (BOOL ok = Thread32First(foto, &te); ok; ok = Thread32Next(foto, &te)) {
                if (te.th32OwnerProcessID != pid) continue;
                const HANDLE h = OpenThread(THREAD_QUERY_LIMITED_INFORMATION, FALSE, te.th32ThreadID);
                if (!h) continue;
                if (GetThreadTimes(h, &c, &e, &k, &u)) {
                    const std::uint64_t t = paraU64(k) + paraU64(u);
                    novos[te.th32ThreadID] = t;
                    const auto it = tempoThreads.find(te.th32ThreadID);
                    if (!primeira && dt100ns > 0 && it != tempoThreads.end())
                        m.cpuThreadMax = std::max(m.cpuThreadMax, 100.0 * (t - it->second) / dt100ns);
                }
                CloseHandle(h);
            }
            CloseHandle(foto);
        }
        m.threadsProcesso = static_cast<double>(novos.size());
        tempoThreads.swap(novos);

        if (!primeira && dt100ns > 0) {
            m.cpuProcesso = 100.0 * (proc - procAnt) / (dt100ns * logicos);
            m.cpuThreadRender = 100.0 * (thr - threadAnt) / dt100ns;
```

```
            const double total = static_cast<double>((k0 - kernAnt) + (u0 - userAnt));
            const double ocioso = static_cast<double>(i0 - idleAnt);
            m.cpuSistema = total > 0 ? 100.0 * (total - ocioso) / total : 0;
        }
        procAnt = proc;
        threadAnt = thr;
        idleAnt = i0;
        kernAnt = k0;
        userAnt = u0;
        relogioAnt = agora;

        if (consulta && PdhCollectQueryData(consulta) == ERROR_SUCCESS) {
            // Núcleos lógicos
            for (const auto& it : lerArray(cNucleos))
                if (it.nome != L"_Total") m.cpuNucleoMax = std::max(m.cpuNucleoMax, std::min(it.valor,
    100.0));
            // Motores das GPUs: soma por motor (todos os processos) e motor 3D deste processo
            std::map<std::wstring, std::pair<int, double>> porMotor;  // chave: "luid_..._eng_N_engtype_X"
            for (const auto& it : lerArray(cMotor)) {
                unsigned p = 0;
                std::uint64_t luid = 0;
                if (!lerPidLuid(it.nome, p, luid)) continue;
                const int g = grupoDoLuid(luid);
                const size_t pos = it.nome.find(L"luid_");
                auto& ref = porMotor[it.nome.substr(pos)];
                ref.first = g;
                ref.second += it.valor;
                if (p == pid && it.nome.find(L"engtype_3D") != std::wstring::npos)
                    m.gpu3dProcesso[g] = std::min(100.0, m.gpu3dProcesso[g] + it.valor);
            }
            for (const auto& [chave, gv] : porMotor)
                m.gpuSistema[gv.first] = std::max(m.gpuSistema[gv.first], std::min(100.0, gv.second));
            // Memória de vídeo do processo
            for (PDH_HCOUNTER cm : {cMemDed, cMemComp})
                for (const auto& it : lerArray(cm)) {
                    unsigned p = 0;
                    std::uint64_t luid = 0;
                    if (lerPidLuid(it.nome, p, luid) && p == pid)
                        m.gpuMemProcessoMB[grupoDoLuid(luid)] += it.valor / 1048576.0;
                }
        }

        if (nvDisp) {
            NvUtilizacao ut{};
            unsigned v = 0;
            NvMemoria mem{};
            unsigned long long motivos = 0;
            if (nvUtil && nvUtil(nvDisp, &ut) == 0) {
                m.nvUtil = ut.gpu;
                m.nvMemUtil = ut.memoria;
            }
            if (nvPotencia && nvPotencia(nvDisp, &v) == 0) m.nvPotenciaW = v / 1000.0;
            if (nvTemp && nvTemp(nvDisp, 0, &v) == 0) m.nvTempC = v;
            if (nvClock && nvClock(nvDisp, 1, &v) == 0) m.nvClockSM = v;
            if (nvClock && nvClock(nvDisp, 2, &v) == 0) m.nvClockMem = v;
            if (nvMem && nvMem(nvDisp, &mem) == 0) m.nvVramUsadaMB = mem.usada / 1048576.0;
            if (nvMotivos && nvMotivos(nvDisp, &motivos) == 0) m.nvMotivosReducao = motivos & ~1ull;  //
    sem "ociosa"
        }

        PROCESS_MEMORY_COUNTERS_EX pmc{};
        if (GetProcessMemoryInfo(GetCurrentProcess(), reinterpret_cast<PROCESS_MEMORY_COUNTERS*>(&pmc),
    sizeof(pmc)))
            m.ramProcessoMB = pmc.WorkingSetSize / 1048576.0;
        MEMORYSTATUSEX ms{};
        ms.dwLength = sizeof(ms);
        if (GlobalMemoryStatusEx(&ms)) m.ramSistemaPct = ms.dwMemoryLoad;
        return m;
    }

    void laco() {
        relogioAnt = std::chrono::steady_clock::now();
        amostrar();  // a primeira amostra só inicializa as referências
```

```
        primeira = false;
        std::unique_lock<std::mutex> trava(mtx);
        while (rodando) {
            cv.wait_for(trava, std::chrono::milliseconds(500), [this] { return !rodando; });
            if (!rodando) break;
            trava.unlock();
            const Metricas m = amostrar();
            trava.lock();
            ultima = m;
            acumularMetricas(soma, m);
        }
    }
};

Monitor::Monitor() : impl_(new Impl) {
    impl_->logicos = processadoresLogicos();
    impl_->enumerarAdaptadores();
}

Monitor::~Monitor() { parar(); }

void Monitor::iniciar() {
    if (impl_->rodando) return;
    DuplicateHandle(GetCurrentProcess(), GetCurrentThread(), GetCurrentProcess(), &impl_->threadRender, 0,
     FALSE,
                    DUPLICATE_SAME_ACCESS);
    impl_->abrirPDH();
    impl_->abrirNVML();
    impl_->rodando = true;
    impl_->trabalhador = std::thread([this] { impl_->laco(); });
}

void Monitor::parar() {
    if (!impl_ || !impl_->rodando) return;
    {
        std::lock_guard<std::mutex> t(impl_->mtx);
        impl_->rodando = false;
    }
    impl_->cv.notify_all();
    if (impl_->trabalhador.joinable()) impl_->trabalhador.join();
    if (impl_->consulta) PdhCloseQuery(impl_->consulta);
    impl_->consulta = nullptr;
    if (impl_->nvml) {
        if (impl_->nvShutdown) impl_->nvShutdown();
        FreeLibrary(impl_->nvml);
        impl_->nvml = nullptr;
        impl_->nvDisp = nullptr;
    }
    if (impl_->threadRender) CloseHandle(impl_->threadRender);
    impl_->threadRender = nullptr;
}

void Monitor::reiniciarJanela() {
    std::lock_guard<std::mutex> t(impl_->mtx);
    impl_->soma = Metricas{};
}

Metricas Monitor::mediaJanela() const {
    std::lock_guard<std::mutex> t(impl_->mtx);
    return mediaMetricas(impl_->soma);
}

Metricas Monitor::ultima() const {
    std::lock_guard<std::mutex> t(impl_->mtx);
    return impl_->ultima;
}

const std::vector<AdaptadorGPU>& Monitor::adaptadores() const { return impl_->adaptadores; }
bool Monitor::temNVML() const { return impl_->nvDisp != nullptr; }
std::string Monitor::nomeGPUNVML() const { return impl_->nvNome; }

// ---------------------------------------------------------------------------
// Informações do sistema
```

```cpp
// ----------------------------------------------------------------------------
std::string nomeCPU() {
    char buf[128] = {};
    DWORD tam = sizeof(buf);
    if (RegGetValueA(HKEY_LOCAL_MACHINE, "HARDWARE\\DESCRIPTION\\System\\CentralProcessor\\0",
                     "ProcessorNameString", RRF_RT_REG_SZ, nullptr, buf, &tam) != ERROR_SUCCESS)
        return "desconhecida";
    std::string s = buf;
    while (!s.empty() && s.back() == ' ') s.pop_back();
    return s;
}

int nucleosFisicos() {
    DWORD tam = 0;
    GetLogicalProcessorInformationEx(RelationProcessorCore, nullptr, &tam);
    std::vector<unsigned char> buf(tam);
    if (!GetLogicalProcessorInformationEx(RelationProcessorCore,
                                          reinterpret_cast<PSYSTEM_LOGICAL_PROCESSOR_INFORMATION_EX>(buf.
     data()), &tam))
        return 0;
    int n = 0;
    for (DWORD pos = 0; pos < tam;) {
        auto* info = reinterpret_cast<PSYSTEM_LOGICAL_PROCESSOR_INFORMATION_EX>(buf.data() + pos);
        ++n;
        pos += info->Size;
    }
    return n;
}

int processadoresLogicos() { return static_cast<int>(GetActiveProcessorCount(ALL_PROCESSOR_GROUPS)); }

double memoriaRAMTotalGB() {
    MEMORYSTATUSEX ms{};
    ms.dwLength = sizeof(ms);
    GlobalMemoryStatusEx(&ms);
    return ms.ullTotalPhys / (1024.0 * 1024.0 * 1024.0);
}

std::string versaoWindows() {
    typedef LONG(WINAPI * FnRtl)(PRTL_OSVERSIONINFOW);
    RTL_OSVERSIONINFOW v{};
    v.dwOSVersionInfoSize = sizeof(v);
    auto rtl = reinterpret_cast<FnRtl>(GetProcAddress(GetModuleHandleW(L"ntdll.dll"), "RtlGetVersion"));
    if (rtl) rtl(&v);
    char edicao[64] = {}, versao[32] = {};
    DWORD t1 = sizeof(edicao), t2 = sizeof(versao);
    RegGetValueA(HKEY_LOCAL_MACHINE, "SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion", "EditionID",
     RRF_RT_REG_SZ,
                 nullptr, edicao, &t1);
    RegGetValueA(HKEY_LOCAL_MACHINE, "SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion", "DisplayVersion",
                 RRF_RT_REG_SZ, nullptr, versao, &t2);
    const int principal = (v.dwMajorVersion == 10 && v.dwBuildNumber >= 22000) ? 11 : static_cast<int>(v.
     dwMajorVersion);
    return "Windows " + std::to_string(principal) + " " + edicao + " " + versao + " (build " +
           std::to_string(v.dwBuildNumber) + ")";
}

#else  // ----------------------------------------------------------------------------
// Outras plataformas: o monitoramento interno não está disponível (use ferramentas externas)
struct Monitor::Impl {
    std::vector<AdaptadorGPU> adaptadores;
};
Monitor::Monitor() : impl_(new Impl) {}
Monitor::~Monitor() = default;
void Monitor::iniciar() {}
void Monitor::parar() {}
void Monitor::reiniciarJanela() {}
Metricas Monitor::mediaJanela() const { return {}; }
Metricas Monitor::ultima() const { return {}; }
const std::vector<AdaptadorGPU>& Monitor::adaptadores() const { return impl_->adaptadores; }
bool Monitor::temNVML() const { return false; }
std::string Monitor::nomeGPUNVML() const { return {}; }
std::string nomeCPU() { return "desconhecida"; }
```

```
int nucleosFisicos() { return static_cast<int>(std::thread::hardware_concurrency()); }
int processadoresLogicos() { return static_cast<int>(std::thread::hardware_concurrency()); }
double memoriaRAMTotalGB() { return 0; }
std::string versaoWindows() { return "nao-Windows"; }
#endif

void acumularMetricas(Metricas& a, const Metricas& b) {
    a.cpuProcesso += b.cpuProcesso;
    a.cpuThreadRender += b.cpuThreadRender;
    a.cpuThreadMax += b.cpuThreadMax;
    a.threadsProcesso += b.threadsProcesso;
    a.cpuSistema += b.cpuSistema;
    a.cpuNucleoMax += b.cpuNucleoMax;
    for (int g = 0; g < NUM_GRUPOS_GPU; ++g) {
        a.gpu3dProcesso[g] += b.gpu3dProcesso[g];
        a.gpuSistema[g] += b.gpuSistema[g];
        a.gpuMemProcessoMB[g] += b.gpuMemProcessoMB[g];
    }
    a.nvUtil += b.nvUtil;
    a.nvMemUtil += b.nvMemUtil;
    a.nvPotenciaW += b.nvPotenciaW;
    a.nvTempC += b.nvTempC;
    a.nvClockSM += b.nvClockSM;
    a.nvClockMem += b.nvClockMem;
    a.nvVramUsadaMB += b.nvVramUsadaMB;
    a.nvMotivosReducao |= b.nvMotivosReducao;  // motivos: união dos observados
    a.ramProcessoMB += b.ramProcessoMB;
    a.ramSistemaPct += b.ramSistemaPct;
    a.amostras += 1;
}

Metricas mediaMetricas(Metricas a) {
    if (a.amostras <= 0) return a;
    const double k = 1.0 / a.amostras;
    a.cpuProcesso *= k;
    a.cpuThreadRender *= k;
    a.cpuThreadMax *= k;
    a.threadsProcesso *= k;
    a.cpuSistema *= k;
    a.cpuNucleoMax *= k;
    for (int g = 0; g < NUM_GRUPOS_GPU; ++g) {
        a.gpu3dProcesso[g] *= k;
        a.gpuSistema[g] *= k;
        a.gpuMemProcessoMB[g] *= k;
    }
    a.nvUtil *= k;
    a.nvMemUtil *= k;
    a.nvPotenciaW *= k;
    a.nvTempC *= k;
    a.nvClockSM *= k;
    a.nvClockMem *= k;
    a.nvVramUsadaMB *= k;
    a.ramProcessoMB *= k;
    a.ramSistemaPct *= k;
    return a;
}

int Monitor::grupoDoFabricante(unsigned fabricante) {
    if (fabricante == 0x10DE) return GPU_NVIDIA;
    if (fabricante == 0x8086) return GPU_INTEL;
    return GPU_OUTRA;
}

std::string Monitor::descreverMotivosReducao(std::uint64_t m) {
    static const std::pair<std::uint64_t, const char*> nomes[] = {
        {0x2, "clocks-aplicacao"},   {0x4, "limite-potencia(SW)"}, {0x8, "desaceleracao-HW"},
        {0x10, "sync-boost"},        {0x20, "termico(SW)"},        {0x40, "termico(HW)"},
        {0x80, "freio-potencia(HW)"}, {0x100, "clock-display"}};
    std::string s;
    for (const auto& [bit, nome] : nomes)
        if (m & bit) s += (s.empty() ? "" : "+") + std::string(nome);
    return s.empty() ? "nenhum" : s;
}
```

src/benchmark.h

```
/*
 * benchmark.h
 * Medição de quadros e protocolo experimental automático.
 *
 * AcumuladorQuadros: soma, numa janela de tempo, o número de quadros e os tempos de cada
 * quadro: tempo de relógio gasto emitindo os comandos OpenGL ("emissão", pode incluir esperas
 * dentro do driver), tempo bloqueado no SwapBuffers, tempo de CPU efetivamente executado pela
 * thread principal (ciclos, exclui esperas) e tempo de GPU medido com GL_TIME_ELAPSED.
 *   FPS médio da janela = quadros / duração da janela;
 *   tempo médio por quadro = 1000 / FPS médio (ms).
 *
 * Benchmark: para cada configuração (A..F), executa o procedimento da seção 2.4:
 *   1) começa pela menor carga da lista ("varredura grossa");
 *   2) aumenta a carga progressivamente;
 *   3) em cada ponto: estabilização + medição (>= 10 s), repetida >= 3 vezes; registra FPS,
 *      tempos, CPU, GPU e memória (médias);
 *   4) ao encontrar o primeiro ponto com FPS médio < 60, faz bisseção entre o último ponto
 *      >= 60 e o primeiro < 60 até que as cargas difiram menos que a tolerância (refino);
 *   5) continua a varredura grossa até o FPS cair abaixo do limiar de parada (para mostrar a
 *      tendência) ou a lista acabar;
 *   6) registra no resumo a MENOR carga testada com FPS médio < 60 e a situação do teste.
 */
#pragma once

#include "cena.h"
#include "monitor.h"

#include <cstdint>
#include <fstream>
#include <map>
#include <string>
#include <vector>

// ---------------------------------------------------------------------------
// Quadros
// ---------------------------------------------------------------------------
struct DadosQuadro {
    double submissaoMs = 0;  // CPU: início do quadro até o fim da emissão dos comandos
    double swapMs = 0;       // CPU bloqueada em glutSwapBuffers
    double cpuAtivaMs = 0;   // ciclos de CPU realmente executados pela thread principal no quadro
    double gpuSomaMs = 0;    // soma dos tempos de GPU (GL_TIME_ELAPSED) que ficaram prontos
    int gpuAmostras = 0;     // neste quadro (resultados de quadros anteriores)
};

struct ResultadoQuadros {
    std::uint64_t quadros = 0;
    double segundos = 0;
    double fps = 0;
    double quadroMs = 0;  // tempo médio por quadro
    double quadroMaxMs = 0;
    double submissaoMs = 0, swapMs = 0, cpuAtivaMs = 0, gpuMs = -1;
};

class AcumuladorQuadros {
public:
    void reiniciar(double agora);
    void adicionar(double agora, const DadosQuadro& d);
    ResultadoQuadros resultado() const;
    double inicio() const { return inicio_; }

private:
    double inicio_ = 0, ultimo_ = 0, anterior_ = 0;
    std::uint64_t quadros_ = 0, quadrosGpu_ = 0;
    double somaSub_ = 0, somaSwap_ = 0, somaAtiva_ = 0, somaGpu_ = 0, maxQuadro_ = 0;
};

// ---------------------------------------------------------------------------
// Protocolo automático
// ---------------------------------------------------------------------------
struct OpcoesBenchmark {
    std::string rotulo = "vbo_T";      // prefixo dos arquivos de saída
```

```
    std::string pastaSaida = "resultados/dados";
    std::string configs = "ABCDEF";
    bool usarVBO = true;
    char variavel = 'T';               // 'T' = detalhe dos orbes, 'N' = número de orbes
    std::vector<int> valores;          // lista grossa (crescente)
    int fixoT = 8, fixoN = 64;         // valor do parâmetro que não varia
    double estabilizacao = 2.0;        // s, descartados antes de cada medição
    double medicao = 10.0;             // s, por repetição
    int repeticoes = 3;
    double fpsLimite = 60.0;
    double fpsParada = 35.0;           // encerra a varredura grossa abaixo deste FPS
    double tolerancia = 0.05;          // refino até tri(hi)/tri(lo) - 1 <= tolerância
    bool apenasPontoAtual = false;     // medição manual (tecla m): um ponto, sem varredura
};

class Benchmark {
public:
    Benchmark(Cena& cena, Monitor& monitor, AcumuladorQuadros& quadros, const OpcoesBenchmark& op,
              const std::string& rendererGL, const std::string& ambiente);

    void iniciar(double agora);
    // Chamado a cada quadro, depois do SwapBuffers. Retorna false quando o protocolo termina.
    bool atualizar(double agora);
    bool terminado() const { return terminado_; }
    std::string status() const;

private:
    enum class Fase { Aplicar, Estabilizar, Medir };
    struct Ponto {
        int valor = 0;
        std::uint64_t triangulos = 0;
        std::string etapa;  // "grossa" ou "refino"
        ResultadoQuadros q;  // médias das repetições
        Metricas m;
        double fpsDesvio = 0, fpsMin = 0, fpsMax = 0;
    };

    Cena& cena_;
    Monitor& monitor_;
    AcumuladorQuadros& quadros_;
    OpcoesBenchmark op_;
    std::string renderer_;

    std::ofstream csvReps_, csvPontos_, csvResumo_;
    bool terminado_ = false;

    // Posição no protocolo
    size_t idxConfig_ = 0;
    char config_ = 'A';
    size_t idxGrosso_ = 0;
    int valorAtual_ = 0;
    std::string etapaAtual_ = "grossa";
    int lo_ = -1, hi_ = -1;          // maior valor >= limite e menor valor < limite já medidos
    bool cruzou_ = false, refinando_ = false, encerrarApos_ = false;
    std::map<int, Ponto> pontosConfig_;

    // Repetições do ponto atual
    Fase fase_ = Fase::Aplicar;
    double tFase_ = 0;
    int rep_ = 0;
    std::vector<ResultadoQuadros> repsQ_;
    std::vector<Metricas> repsM_;
    double tInicioPonto_ = 0;
    double ultimoAgora_ = 0;

    std::uint64_t triangulosPara(int valor) const;
    void aplicarPonto();
    void concluirRepeticao(double agora);
    void concluirPonto();
    void decidirProximo(const Ponto& p);
    void iniciarConfig();
    void concluirConfig();
    bool precisaRefinar() const;
```

```
    void escreverCabecalhos();
};

// Formatação auxiliar
std::string agoraTexto();                       // data e hora local "AAAA-MM-DD HH:MM:SS"
std::string comMilhares(std::uint64_t n);       // 1234567 -> "1.234.567"
```

src/benchmark.cpp

```
/*
 * benchmark.cpp
 * Medição de quadros e protocolo experimental automático (ver benchmark.h).
 */
#include "benchmark.h"

#include <algorithm>
#include <cmath>
#include <cstdio>
#include <ctime>
#include <filesystem>
#include <sstream>

// ---------------------------------------------------------------------------
// Utilitários
// ---------------------------------------------------------------------------
std::string agoraTexto() {
    const std::time_t t = std::time(nullptr);
    std::tm tm{};
#ifdef _WIN32
    localtime_s(&tm, &t);
#else
    localtime_r(&t, &tm);
#endif
    char buf[32];
    std::strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", &tm);
    return buf;
}

std::string comMilhares(std::uint64_t n) {
    std::string s = std::to_string(n);
    for (int i = static_cast<int>(s.size()) - 3; i > 0; i -= 3) s.insert(static_cast<size_t>(i), ".");
    return s;
}

// ---------------------------------------------------------------------------
// AcumuladorQuadros
// ---------------------------------------------------------------------------
void AcumuladorQuadros::reiniciar(double agora) {
    inicio_ = ultimo_ = anterior_ = agora;
    quadros_ = quadrosGpu_ = 0;
    somaSub_ = somaSwap_ = somaAtiva_ = somaGpu_ = maxQuadro_ = 0;
}

void AcumuladorQuadros::adicionar(double agora, const DadosQuadro& d) {
    ++quadros_;
    maxQuadro_ = std::max(maxQuadro_, (agora - anterior_) * 1000.0);
    anterior_ = ultimo_ = agora;
    somaSub_ += d.submissaoMs;
    somaSwap_ += d.swapMs;
    somaAtiva_ += d.cpuAtivaMs;
    somaGpu_ += d.gpuSomaMs;
    quadrosGpu_ += static_cast<std::uint64_t>(d.gpuAmostras);
}

ResultadoQuadros AcumuladorQuadros::resultado() const {
    ResultadoQuadros r;
    r.quadros = quadros_;
    r.segundos = ultimo_ - inicio_;
    if (quadros_ == 0 || r.segundos <= 0) return r;
    r.fps = quadros_ / r.segundos;
```

```
    r.quadroMs = 1000.0 / r.fps;
    r.quadroMaxMs = maxQuadro_;
    r.submissaoMs = somaSub_ / quadros_;
    r.swapMs = somaSwap_ / quadros_;
    r.cpuAtivaMs = somaAtiva_ / quadros_;
    r.gpuMs = quadrosGpu_ ? somaGpu_ / quadrosGpu_ : -1;
    return r;
}

// ---------------------------------------------------------------------------
// Escrita dos CSV
// ---------------------------------------------------------------------------
namespace {

const char* CAB_METRICAS =
    "fps,quadro_ms,quadro_max_ms,submissao_ms,swap_ms,cpu_ativa_ms,gpu_ms,"
    "cpu_proc,cpu_thread,cpu_thread_max,threads,cpu_sis,cpu_nucleo_max,"
    "gpu_nv_proc,gpu_nv_sis,gpu_intel_proc,gpu_intel_sis,"
    "nv_util,nv_mem_util,nv_pot_w,nv_temp_c,nv_clk_sm,nv_clk_mem,nv_vram_mb,"
    "gpu_mem_proc_nv_mb,gpu_mem_proc_intel_mb,ram_proc_mb,ram_sis_pct,motivos_reducao";

std::string metricasCSV(const ResultadoQuadros& q, const Metricas& m) {
    char buf[1024];
    std::snprintf(buf, sizeof(buf),
                  "%.3f,%.4f,%.3f,%.4f,%.4f,%.4f,%.4f,"
                  "%.2f,%.2f,%.2f,%.1f,%.2f,%.2f,"
                  "%.2f,%.2f,%.2f,%.2f,"
                  "%.1f,%.1f,%.2f,%.1f,%.0f,%.0f,%.1f,"
                  "%.1f,%.1f,%.1f,%.1f,%s",
                  q.fps, q.quadroMs, q.quadroMaxMs, q.submissaoMs, q.swapMs, q.cpuAtivaMs, q.gpuMs,
                  m.cpuProcesso, m.cpuThreadRender, m.cpuThreadMax, m.threadsProcesso, m.cpuSistema, m.
    cpuNucleoMax,
                  m.gpu3dProcesso[GPU_NVIDIA], m.gpuSistema[GPU_NVIDIA], m.gpu3dProcesso[GPU_INTEL],
                  m.gpuSistema[GPU_INTEL],
                  m.nvUtil, m.nvMemUtil, m.nvPotenciaW, m.nvTempC, m.nvClockSM, m.nvClockMem, m.
    nvVramUsadaMB,
                  m.gpuMemProcessoMB[GPU_NVIDIA], m.gpuMemProcessoMB[GPU_INTEL], m.ramProcessoMB, m.
    ramSistemaPct,
                  Monitor::descreverMotivosReducao(m.nvMotivosReducao).c_str());
    return buf;
}

// Média de um campo double de um vetor de structs
template <class T, class F>
double media(const std::vector<T>& v, F campo) {
    double s = 0;
    for (const T& x : v) s += campo(x);
    return v.empty() ? 0 : s / v.size();
}

}  // namespace

// ---------------------------------------------------------------------------
// Benchmark
// ---------------------------------------------------------------------------
Benchmark::Benchmark(Cena& cena, Monitor& monitor, AcumuladorQuadros& quadros, const OpcoesBenchmark& op,
                     const std::string& rendererGL, const std::string& ambiente)
    : cena_(cena), monitor_(monitor), quadros_(quadros), op_(op), renderer_(rendererGL) {
    std::filesystem::create_directories(op_.pastaSaida);
    const std::string base = op_.pastaSaida + "/" + op_.rotulo;
    // Medições manuais são acrescentadas ao mesmo arquivo; o protocolo completo recria os arquivos.
    const auto modo = op_.apenasPontoAtual ? std::ios::app : std::ios::trunc;
    const bool novo = !op_.apenasPontoAtual || !std::filesystem::exists(base + "_pontos.csv");
    csvReps_.open(base + "_repeticoes.csv", std::ios::out | modo);
    csvPontos_.open(base + "_pontos.csv", std::ios::out | modo);
    if (!op_.apenasPontoAtual) csvResumo_.open(base + "_resumo.csv", std::ios::out | std::ios::trunc);
    if (novo) escreverCabecalhos();
    std::ofstream amb(base + "_ambiente.txt", std::ios::out | modo);
    amb << ambiente << "\n";
}

void Benchmark::escreverCabecalhos() {
    const std::string id =
```

```
        "rotulo,modo,renderer,config,iluminacao,textura,variavel,objetos,T,tri_por_objeto,triangulos,
     chamadas,etapa,";
    csvReps_ << id << "rep,quadros,segundos," << CAB_METRICAS << ",inicio\n";
    csvPontos_ << id << "reps,fps_dp,fps_min,fps_max," << CAB_METRICAS << ",fim\n";
    if (csvResumo_.is_open())
        csvResumo_ << "rotulo,modo,config,iluminacao,textura,situacao,"
                      "tri_ultimo_acima,valor_ultimo_acima,fps_ultimo_acima,"
                      "tri_primeiro_abaixo,valor_primeiro_abaixo,fps_primeiro_abaixo,fps_dp_primeiro_abaixo
     ,"
                      "quadro_ms,gpu_ms,submissao_ms,swap_ms,cpu_ativa_ms,cpu_proc,cpu_thread,
     cpu_thread_max,cpu_nucleo_max,"
                      "gpu_nv_proc,gpu_intel_proc,nv_util,nv_pot_w,nv_clk_sm,motivos_reducao,"
                      "tri_maior_testado,fps_maior_testado,pontos_medidos\n";
}

std::uint64_t Benchmark::triangulosPara(int valor) const {
    return op_.variavel == 'T' ? Cena::triangulosPara(valor, op_.fixoN) : Cena::triangulosPara(op_.fixoT,
     valor);
}

void Benchmark::iniciar(double agora) {
    ultimoAgora_ = agora;
    idxConfig_ = 0;
    std::printf("\n=== Benchmark '%s' | modo %s | variavel %c | configs %s | %d x %.0f s (+%.0f s estab.)
     ===\n",
                op_.rotulo.c_str(), op_.usarVBO ? "VBO" : "imediato", op_.variavel, op_.configs.c_str(),
                op_.repeticoes, op_.medicao, op_.estabilizacao);
    iniciarConfig();
}

void Benchmark::iniciarConfig() {
    config_ = op_.configs[idxConfig_];
    lo_ = hi_ = -1;
    cruzou_ = refinando_ = encerrarApos_ = false;
    pontosConfig_.clear();
    idxGrosso_ = 0;
    valorAtual_ = op_.valores.front();
    etapaAtual_ = "grossa";
    fase_ = Fase::Aplicar;
    std::printf("\n--- Configuracao %s ---\n", descreverConfig(config_).c_str());
}

void Benchmark::aplicarPonto() {
    cena_.definirConfig(config_);
    cena_.definirVBO(op_.usarVBO);
    cena_.definirAramado(false);
    if (op_.variavel == 'T') {
        cena_.definirObjetos(op_.fixoN);
        cena_.definirDetalhe(valorAtual_);
    } else {
        cena_.definirDetalhe(op_.fixoT);
        cena_.definirObjetos(valorAtual_);
    }
    rep_ = 0;
    repsQ_.clear();
    repsM_.clear();
}

bool Benchmark::atualizar(double agora) {
    if (terminado_) return false;
    // Proteção: um intervalo anormal entre quadros (janela minimizada -- o FreeGLUT deixa de
    // desenhar --, sistema suspenso, etc.) invalida a repetição em andamento, que é descartada
    // e recomeça pela estabilização. O maior quadro legítimo dos experimentos é < 0,2 s.
    const double intervalo = agora - ultimoAgora_;
    ultimoAgora_ = agora;
    if (intervalo > 2.0 && fase_ != Fase::Aplicar) {
        std::printf("  [!] intervalo anormal de %.1f s entre quadros (janela minimizada?): repeticao %d "
                    "descartada e reiniciada\n", intervalo, rep_ + 1);
        std::fflush(stdout);
        fase_ = Fase::Estabilizar;
        tFase_ = agora;
        return true;
    }
```

```
    switch (fase_) {
        case Fase::Aplicar:
            aplicarPonto();  // pode recriar a malha: o quadro seguinte é descartado na estabilização
            tInicioPonto_ = agora;
            fase_ = Fase::Estabilizar;
            tFase_ = agora;
            break;
        case Fase::Estabilizar:
            if (agora - tFase_ >= op_.estabilizacao) {
                quadros_.reiniciar(agora);
                monitor_.reiniciarJanela();
                fase_ = Fase::Medir;
                tFase_ = agora;
            }
            break;
        case Fase::Medir:
            if (agora - tFase_ >= op_.medicao) concluirRepeticao(agora);
            break;
    }
    return !terminado_;
}

void Benchmark::concluirRepeticao(double agora) {
    const ResultadoQuadros q = quadros_.resultado();
    const Metricas m = monitor_.mediaJanela();
    repsQ_.push_back(q);
    repsM_.push_back(m);
    const EstadoCena& e = cena_.estado();
    csvReps_ << op_.rotulo << ',' << (op_.usarVBO ? "VBO" : "imediato") << ",\"" << renderer_ << "\"," <<
     config_
             << ',' << nomeIluminacao(iluminacaoDaConfig(config_)) << ',' << (texturaDaConfig(config_) ? 1
     : 0) << ','
             << op_.variavel << ',' << e.N << ',' << e.T << ',' << cena_.triangulosPorOrbe() << ','
             << cena_.triangulosPorQuadro() << ',' << cena_.chamadasDesenhoPorQuadro() << ',' <<
     etapaAtual_ << ','
             << (rep_ + 1) << ',' << q.quadros << ',' << q.segundos << ',' << metricasCSV(q, m) << ','
             << agoraTexto() << '\n';
    csvReps_.flush();
    std::printf("  [%c] %s=%-5d tri=%-12s rep %d/%d: %8.2f FPS  %7.3f ms  (emissao %.3f ms, CPU ativa %.3f
     ms, GPU %.3f ms)  "
                "CPU %.1f%% thr %.0f%% thrMax %.0f%%  GPU nv %.0f%% intel %.0f%%\n",
                config_, op_.variavel == 'T' ? "T" : "N", valorAtual_, comMilhares(cena_.
     triangulosPorQuadro()).c_str(),
                rep_ + 1, op_.repeticoes, q.fps, q.quadroMs, q.submissaoMs, q.cpuAtivaMs, q.gpuMs, m.
     cpuProcesso, m.cpuThreadRender,
                m.cpuThreadMax, m.gpu3dProcesso[GPU_NVIDIA], m.gpu3dProcesso[GPU_INTEL]);
    std::fflush(stdout);
    if (++rep_ < op_.repeticoes) {
        fase_ = Fase::Estabilizar;  // nova estabilização antes de cada repetição
        tFase_ = agora;
    } else {
        concluirPonto();
    }
}

void Benchmark::concluirPonto() {
    Ponto p;
    p.valor = valorAtual_;
    p.triangulos = cena_.triangulosPorQuadro();
    p.etapa = etapaAtual_;
    // Médias das repetições (o critério de < 60 FPS usa o FPS médio das repetições)
    p.q.quadros = static_cast<std::uint64_t>(media(repsQ_, [](const ResultadoQuadros& r) { return double(r.
     quadros); }));
    p.q.segundos = media(repsQ_, [](const ResultadoQuadros& r) { return r.segundos; });
    p.q.fps = media(repsQ_, [](const ResultadoQuadros& r) { return r.fps; });
    p.q.quadroMs = media(repsQ_, [](const ResultadoQuadros& r) { return r.quadroMs; });
    p.q.quadroMaxMs = media(repsQ_, [](const ResultadoQuadros& r) { return r.quadroMaxMs; });
    p.q.submissaoMs = media(repsQ_, [](const ResultadoQuadros& r) { return r.submissaoMs; });
    p.q.swapMs = media(repsQ_, [](const ResultadoQuadros& r) { return r.swapMs; });
    p.q.cpuAtivaMs = media(repsQ_, [](const ResultadoQuadros& r) { return r.cpuAtivaMs; });
    p.q.gpuMs = media(repsQ_, [](const ResultadoQuadros& r) { return r.gpuMs; });
    double s2 = 0;
    p.fpsMin = 1e30;
```

```
    p.fpsMax = 0;
    for (const auto& r : repsQ_) {
        s2 += (r.fps - p.q.fps) * (r.fps - p.q.fps);
        p.fpsMin = std::min(p.fpsMin, r.fps);
        p.fpsMax = std::max(p.fpsMax, r.fps);
    }
    p.fpsDesvio = repsQ_.size() > 1 ? std::sqrt(s2 / (repsQ_.size() - 1)) : 0;  // desvio-padrão amostral
    Metricas soma;
    for (const auto& m : repsM_) acumularMetricas(soma, m);
    p.m = mediaMetricas(soma);  // média das médias de cada repetição
    pontosConfig_[p.valor] = p;

    const EstadoCena& e = cena_.estado();
    char extra[128];
    std::snprintf(extra, sizeof(extra), "%zu,%.3f,%.3f,%.3f,", repsQ_.size(), p.fpsDesvio, p.fpsMin, p.
     fpsMax);
    csvPontos_ << op_.rotulo << ',' << (op_.usarVBO ? "VBO" : "imediato") << ",\"" << renderer_ << "\"," <<
      config_
               << ',' << nomeIluminacao(iluminacaoDaConfig(config_)) << ',' << (texturaDaConfig(config_) ?
     1 : 0)
               << ',' << op_.variavel << ',' << e.N << ',' << e.T << ',' << cena_.triangulosPorOrbe() << ',
     '
               << p.triangulos << ',' << cena_.chamadasDesenhoPorQuadro() << ',' << p.etapa << ',' << extra
               << metricasCSV(p.q, p.m) << ',' << agoraTexto() << '\n';
    csvPontos_.flush();
    std::printf("  => [%c] %s=%d (%s) tri=%s: FPS medio %.2f +- %.2f %s\n", config_, op_.variavel == 'T' ?
     "T" : "N",
                p.valor, p.etapa.c_str(), comMilhares(p.triangulos).c_str(), p.q.fps, p.fpsDesvio,
                p.q.fps < op_.fpsLimite ? "(< 60)" : "");
    std::fflush(stdout);
    decidirProximo(p);
}

bool Benchmark::precisaRefinar() const {
    if (lo_ < 0 || hi_ < 0 || hi_ - lo_ <= 1) return false;
    return static_cast<double>(triangulosPara(hi_)) / triangulosPara(lo_) - 1.0 > op_.tolerancia;
}

void Benchmark::decidirProximo(const Ponto& p) {
    if (op_.apenasPontoAtual) {
        terminado_ = true;
        return;
    }
    const bool abaixo = p.q.fps < op_.fpsLimite;
    if (!abaixo) {
        if (p.valor > lo_) lo_ = p.valor;
    } else if (hi_ < 0 || p.valor < hi_) {
        hi_ = p.valor;
    }

    if (refinando_) {
        if (precisaRefinar()) {
            valorAtual_ = (lo_ + hi_) / 2;  // bisseção
            fase_ = Fase::Aplicar;
            return;
        }
        refinando_ = false;
        if (encerrarApos_) {
            concluirConfig();
            return;
        }
    } else {
        encerrarApos_ = p.q.fps < op_.fpsParada;
        if (abaixo && !cruzou_) {
            cruzou_ = true;
            if (precisaRefinar()) {
                refinando_ = true;
                etapaAtual_ = "refino";
                valorAtual_ = (lo_ + hi_) / 2;
                fase_ = Fase::Aplicar;
                return;
            }
        }
```

```
        if (encerrarApos_) {
            concluirConfig();
            return;
        }
    }
    // Próximo valor da lista grossa ainda não medido
    while (++idxGrosso_ < op_.valores.size() && pontosConfig_.count(op_.valores[idxGrosso_])) {
    }
    if (idxGrosso_ >= op_.valores.size()) {
        concluirConfig();
        return;
    }
    valorAtual_ = op_.valores[idxGrosso_];
    etapaAtual_ = "grossa";
    fase_ = Fase::Aplicar;
}

void Benchmark::concluirConfig() {
    // Menor carga testada com FPS médio < 60 e maior carga testada (abaixo dela) com FPS >= 60
    const Ponto* abaixo = nullptr;
    const Ponto* acima = nullptr;
    const Ponto* maior = nullptr;
    for (const auto& [v, p] : pontosConfig_) {
        if (p.q.fps < op_.fpsLimite && (!abaixo || p.triangulos < abaixo->triangulos)) abaixo = &p;
        if (!maior || p.triangulos > maior->triangulos) maior = &p;
    }
    for (const auto& [v, p] : pontosConfig_)
        if (p.q.fps >= op_.fpsLimite && (!abaixo || p.triangulos < abaixo->triangulos) &&
            (!acima || p.triangulos > acima->triangulos))
            acima = &p;
    const char* situacao = !abaixo ? "limite nao alcancado"
                           : !acima ? "abaixo de 60 ja na menor carga"
                                    : "limite encontrado";
    const Ponto vazio;
    const Ponto& a = acima ? *acima : vazio;
    const Ponto& b = abaixo ? *abaixo : vazio;
    char buf[1024];
    std::snprintf(buf, sizeof(buf),
                  "%s,%s,%c,%s,%d,%s,%llu,%d,%.3f,%llu,%d,%.3f,%.3f,%.4f,%.4f,%.4f,%.4f,%.4f,%.2f,%.2f,%.2f
    ,%.2f,%.2f,%.2f,"
                  "%.1f,%.2f,%.0f,%s,%llu,%.3f,%zu\n",
                  op_.rotulo.c_str(), op_.usarVBO ? "VBO" : "imediato", config_,
                  nomeIluminacao(iluminacaoDaConfig(config_)).c_str(), texturaDaConfig(config_) ? 1 : 0,
    situacao,
                  static_cast<unsigned long long>(a.triangulos), a.valor, a.q.fps,
                  static_cast<unsigned long long>(b.triangulos), b.valor, b.q.fps, b.fpsDesvio, b.q.
    quadroMs,
                  b.q.gpuMs, b.q.submissaoMs, b.q.swapMs, b.q.cpuAtivaMs, b.m.cpuProcesso, b.m.
    cpuThreadRender, b.m.cpuThreadMax,
                  b.m.cpuNucleoMax,
                  b.m.gpu3dProcesso[GPU_NVIDIA], b.m.gpu3dProcesso[GPU_INTEL], b.m.nvUtil, b.m.nvPotenciaW,
                  b.m.nvClockSM, Monitor::descreverMotivosReducao(b.m.nvMotivosReducao).c_str(),
                  static_cast<unsigned long long>(maior ? maior->triangulos : 0), maior ? maior->q.fps :
    0.0,
                  pontosConfig_.size());
    csvResumo_ << buf;
    csvResumo_.flush();
    std::printf("--- Config %c: %s | ultimo >= 60: %s tri (%.1f FPS) | primeiro < 60: %s tri (%.1f FPS)\n",
     config_,
                situacao, comMilhares(a.triangulos).c_str(), a.q.fps, comMilhares(b.triangulos).c_str(), b.
    q.fps);
    std::fflush(stdout);
    if (++idxConfig_ >= op_.configs.size()) {
        terminado_ = true;
        std::printf("\n=== Benchmark '%s' concluido. Resultados em %s/ ===\n", op_.rotulo.c_str(),
                    op_.pastaSaida.c_str());
        return;
    }
    iniciarConfig();
}

std::string Benchmark::status() const {
    if (terminado_) return "benchmark concluido";
```

```
    char buf[256];
    const double dt = ultimoAgora_ - tFase_;
    const char* fase = fase_ == Fase::Medir ? "medindo" : "estabilizando";
    const double total = fase_ == Fase::Medir ? op_.medicao : op_.estabilizacao;
    std::snprintf(buf, sizeof(buf), "[%s] config %c | %c=%d (%s) | rep %d/%d | %s %.1f/%.0f s", op_.rotulo.
     c_str(),
                  config_, op_.variavel, valorAtual_, etapaAtual_.c_str(), std::min(rep_ + 1, op_.
     repeticoes),
                  op_.repeticoes, fase, std::min(dt, total), total);
    return buf;
}
```

src/main.cpp

```
/*
 * main.cpp
 * Atividade 3 - Análise de desempenho de CPU e GPU utilizando OpenGL
 *
 * Aplicação única com a cena "Jardim de Orbes" (ver cena.h), renderizada continuamente,
 * com medição de FPS, contagem de triângulos por quadro, seis configurações de iluminação e
 * textura (A..F), monitoramento de CPU/GPU (monitor.h) e protocolo automático de busca do
 * limite de 60 FPS (benchmark.h).
 *
 * Teclado (também resumido no painel da janela):
 *   1..6 ............ configuração A..F          l ......... alterna iluminação (nenhuma/omni/spot)
 *   t ............... liga/desliga texturas      + / - ..... detalhe T dos orbes (passo 4 ou 16)
 *   ] / [ ........... detalhe T +1 / -1          * / / ..... número de orbes x2 / /2
 *   v ............... VBO <-> modo imediato      s ......... liga/desliga V-Sync
 *   w ............... desenho em arame           p ......... pausa/retoma a animação
 *   h ............... mostra/oculta o painel     c ......... salva captura de tela
 *   m ............... mede o ponto atual (3 x 10 s) e grava em resultados/dados/manual_*.csv
 *   r ou Home ....... restaura a configuração inicial
 *   ESC ou q ........ encerra
 *
 * Linha de comando:
 *   desempenho_gl                       modo interativo
 *   desempenho_gl --info                imprime o ambiente (CPU, GPUs, OpenGL) e encerra
 *   desempenho_gl --capturar            gera as capturas do relatório em resultados/capturas
 *   desempenho_gl --benchmark [opções]  executa o protocolo automático e encerra
 *       --rotulo NOME       prefixo dos arquivos CSV            (padrão: vbo_T)
 *       --modo vbo|imediato caminho de envio da geometria       (padrão: vbo)
 *       --variavel T|N      parâmetro que aumenta a carga        (padrão: T)
 *       --valores a,b,c...  lista grossa de valores crescentes
 *       --configs ABCDEF    configurações a medir
 *       --T n / --N n       valor fixo do outro parâmetro        (padrão: T=8, N=64)
 *       --estab s --medir s --reps n --parada fps --saida pasta
 *   --vsync liga o V-Sync (por padrão é desligado para não limitar o FPS).
 */
#include "benchmark.h"
#include "cena.h"
#include "gl_ext.h"
#include "monitor.h"

#include <GL/freeglut.h>

#ifdef _WIN32
#include <intrin.h>
#endif

#ifdef TEM_OPENCV
#include <opencv2/core.hpp>
#include <opencv2/imgcodecs.hpp>
#include <opencv2/imgproc.hpp>
#endif

#include <algorithm>
#include <cctype>
#include <chrono>
#include <cstdio>
#include <cstdlib>
```

```cpp
#include <cstring>
#include <deque>
#include <filesystem>
#include <fstream>
#include <memory>
#include <sstream>
#include <string>
#include <vector>

// Em notebooks com duas GPUs (NVIDIA Optimus / AMD PowerXpress), exportar estes símbolos pede
// ao driver que crie o contexto OpenGL na GPU dedicada. O alvo "desempenho_gl_integrada" é
// compilado SEM estes símbolos, para permitir a comparação com a GPU integrada.
#if defined(_WIN32) && defined(PREFERIR_GPU_DEDICADA)
extern "C" {
__declspec(dllexport) unsigned long NvOptimusEnablement = 0x00000001;
__declspec(dllexport) int AmdPowerXpressRequestHighPerformance = 1;
}
static const bool PEDE_GPU_DEDICADA = true;
#else
static const bool PEDE_GPU_DEDICADA = false;
#endif

// ---------------------------------------------------------------------------
// Configuração
// ---------------------------------------------------------------------------
static const int LARGURA = 1280;  // resolução fixa de todos os experimentos
static const int ALTURA = 720;
static const int T_INICIAL = 32, N_INICIAL = 64;
static const char* DIR_CAPTURAS = "resultados/capturas";
static const int NUM_CONSULTAS = 16;  // conjunto de consultas GL_TIME_ELAPSED em uso simultâneo

// ---------------------------------------------------------------------------
// Estado global
// ---------------------------------------------------------------------------
static Cena cena;
static Monitor monitor;
static AcumuladorQuadros quadrosBench, quadrosPainel;
static ResultadoQuadros painel;  // FPS exibido (atualizado a cada 0,5 s)
static std::unique_ptr<Benchmark> benchmark;
static bool sairAoTerminarBenchmark = false;

static int janelaL = LARGURA, janelaA = ALTURA;
static bool mostrarPainel = true, pausado = false, vsync = false;
static double tempoAnim = 0, ultimoTempo = 0, ultimoConsole = 0;
static std::string rendererGL, vendorGL, versaoGL, versaoGLSL, ambiente;

static GLuint consultas[NUM_CONSULTAS] = {};
struct ConsultaPendente {
    int id;                 // índice em consultas[]
    std::uint64_t quadro;   // quadro em que foi emitida
};
static std::deque<ConsultaPendente> consultasPendentes;  // aguardando resultado (ordem de emissão)
static const std::uint64_t IDADE_MIN_CONSULTA = 3;  // só consulta resultados de >= 3 quadros atrás
static double ciclosPorMs = 0;  // frequência do contador de ciclos (TSC), calibrada na inicialização
static std::vector<int> consultasLivres;
static std::uint64_t numQuadro = 0;

static std::string capturaPendente;  // nome do arquivo a gravar no próximo quadro
static int contadorCapturas = 0;

// Modo --capturar: sequência de tomadas com a animação congelada no mesmo instante
struct Tomada {
    std::string nome;
    char config;
    int T, N;
    bool aramado;
};
static std::vector<Tomada> tomadas;
static size_t idxTomada = 0;
static double tTomada = 0;
static bool aguardandoGravacao = false;
static const double TEMPO_ANIM_CAPTURA = 2.6;
```

```
// Ciclos de CPU efetivamente executados pela thread atual (não conta o tempo bloqueado/esperando)
static std::uint64_t ciclosThread() {
#ifdef _WIN32
    ULONG64 c = 0;
    QueryThreadCycleTime(GetCurrentThread(), &c);
    return c;
#else
    return 0;
#endif
}

// QueryThreadCycleTime conta em ciclos do contador de tempo do processador (TSC), de frequência
// constante; ela é estimada comparando __rdtsc com o relógio de alta resolução por 200 ms.
static void calibrarCiclos() {
#ifdef _WIN32
    const auto r0 = std::chrono::steady_clock::now();
    const unsigned long long c0 = __rdtsc();
    while (std::chrono::steady_clock::now() - r0 < std::chrono::milliseconds(200)) {
    }
    const unsigned long long c1 = __rdtsc();
    const double ms = std::chrono::duration<double, std::milli>(std::chrono::steady_clock::now() - r0).
     count();
    ciclosPorMs = (c1 - c0) / ms;
#endif
}

static double agora() {
    static const auto inicio = std::chrono::steady_clock::now();
    return std::chrono::duration<double>(std::chrono::steady_clock::now() - inicio).count();
}

// ---------------------------------------------------------------------------
// Ambiente (registrado junto com os resultados)
// ---------------------------------------------------------------------------
static std::string descreverAmbiente() {
    std::ostringstream s;
    char linha[256];
    s << "Data/hora: " << agoraTexto() << "\n";
    s << "Sistema operacional: " << versaoWindows() << "\n";
    s << "CPU: " << nomeCPU() << " | " << nucleosFisicos() << " nucleos fisicos / " << processadoresLogicos
     ()
      << " processadores logicos\n";
    std::snprintf(linha, sizeof(linha), "RAM: %.1f GB\n", memoriaRAMTotalGB());
    s << linha;
    s << "Adaptadores graficos (DXGI):\n";
    for (const auto& a : monitor.adaptadores()) {
        std::snprintf(linha, sizeof(linha), "  - %s | fabricante 0x%04X | VRAM dedicada %.0f MB |
     compartilhada %.0f MB | %s\n",
                      a.nome.c_str(), a.fabricante, a.vramDedicadaMB, a.memoriaCompartilhadaMB,
                      a.integrada() ? "integrada" : "dedicada");
        s << linha;
    }
    s << "OpenGL: " << versaoGL << "\n";
    s << "GLSL: " << versaoGLSL << "\n";
    s << "GL_VENDOR: " << vendorGL << "\n";
    s << "GL_RENDERER (dispositivo que atende a aplicacao): " << rendererGL << "\n";
    s << "Executavel pede GPU dedicada (NvOptimusEnablement): " << (PEDE_GPU_DEDICADA ? "sim" : "nao") << "
     \n";
    s << "V-Sync (intervalo de troca): " << consultarVSync()
      << (consultarVSync() == 0 ? " (desligado)" : consultarVSync() > 0 ? " (ligado)" : " (desconhecido)")
     << "\n";
    s << "Janela: " << janelaL << " x " << janelaA << " pixels\n";
    s << "Contador de ciclos da CPU (TSC): " << static_cast<int>(ciclosPorMs / 1000.0) << " MHz\n";
    s << "Consulta de tempo da GPU (GL_TIME_ELAPSED): " << (temConsultaTempoGPU() ? "sim" : "nao") << "\n";
    s << "Filtragem anisotropica: " << (temAnisotropia() ? "sim (8x)" : "nao") << "\n";
    s << "NVML: " << (monitor.temNVML() ? monitor.nomeGPUNVML() : std::string("indisponivel")) << "\n";
    std::snprintf(linha, sizeof(linha), "Texturas: 3 x 1024x1024 RGBA8 + mipmaps (%.1f MB)\n", cena.
     memoriaTexturasMB());
    s << linha;
    std::snprintf(linha, sizeof(linha), "Triangulos do cenario fixo (piso + no + lampada): %llu\n",
                  static_cast<unsigned long long>(cena.triangulosCenario()));
    s << linha;
    return s.str();
```

```
}

// ---------------------------------------------------------------------------
// Captura de tela (PNG com OpenCV; PPM sem ele)
// ---------------------------------------------------------------------------
static bool salvarCaptura(const std::string& caminhoSemExtensao) {
    std::filesystem::create_directories(std::filesystem::path(caminhoSemExtensao).parent_path());
    std::vector<unsigned char> px(static_cast<size_t>(janelaL) * janelaA * 3);
    glPixelStorei(GL_PACK_ALIGNMENT, 1);
    glReadBuffer(GL_BACK);
    glReadPixels(0, 0, janelaL, janelaA, GL_RGB, GL_UNSIGNED_BYTE, px.data());
#ifdef TEM_OPENCV
    cv::Mat rgb(janelaA, janelaL, CV_8UC3, px.data()), bgr;
    cv::cvtColor(rgb, bgr, cv::COLOR_RGB2BGR);  // OpenGL: R,G,B | imwrite: B,G,R
    cv::flip(bgr, bgr, 0);                      // OpenGL: origem no canto inferior esquerdo
    const bool ok = cv::imwrite(caminhoSemExtensao + ".png", bgr);
#else
    std::ofstream f(caminhoSemExtensao + ".ppm", std::ios::binary);
    f << "P6\n" << janelaL << " " << janelaA << "\n255\n";
    for (int y = janelaA - 1; y >= 0; --y) f.write(reinterpret_cast<const char*>(&px[static_cast<size_t>(y)
      * janelaL * 3]), janelaL * 3);
    const bool ok = f.good();
#endif
    std::printf("Captura %s: %s\n", ok ? "salva" : "FALHOU", caminhoSemExtensao.c_str());
    return ok;
}

// ---------------------------------------------------------------------------
// Painel de informações (texto ASCII com fonte bitmap do GLUT)
// ---------------------------------------------------------------------------
static void texto(int x, int y, const std::string& s) {
    glRasterPos2i(x, y);
    glutBitmapString(GLUT_BITMAP_9_BY_15, reinterpret_cast<const unsigned char*>(s.c_str()));
}

static void desenharPainel() {
    const EstadoCena& e = cena.estado();
    const Metricas m = monitor.ultima();
    std::vector<std::string> linhas;
    char b[320];
    std::snprintf(b, sizeof(b), "Config %s | %s | V-Sync %s | %dx%d%s", descreverConfig(e.config).c_str(),
                  e.usarVBO ? "VBO" : "modo imediato", vsync ? "LIGADO" : "desligado", janelaL, janelaA,
                  pausado ? " | PAUSADO" : "");
    linhas.push_back(b);
    std::snprintf(b, sizeof(b), "FPS %.1f | quadro %.2f ms | emissao %.2f + swap %.2f ms | CPU ativa %.2f
     ms | GPU %.2f ms",
                  painel.fps, painel.quadroMs, painel.submissaoMs, painel.swapMs, painel.cpuAtivaMs, painel
     .gpuMs);
    linhas.push_back(b);
    std::snprintf(b, sizeof(b), "Triangulos/quadro %s | %d orbes x T=%d (%s tri/orbe) | chamadas %d",
                  comMilhares(cena.triangulosPorQuadro()).c_str(), e.N, e.T,
                  comMilhares(cena.triangulosPorOrbe()).c_str(), cena.chamadasDesenhoPorQuadro());
    linhas.push_back(b);
    std::snprintf(b, sizeof(b), "CPU: processo %.1f%% | thread render %.0f%% | thread mais ocupada %.0f%% (
     de 1 nucleo) | sistema %.0f%%",
                  m.cpuProcesso, m.cpuThreadRender, m.cpuThreadMax, m.cpuSistema);
    linhas.push_back(b);
    std::snprintf(b, sizeof(b), "GPU 3D (processo): NVIDIA %.0f%% | Intel %.0f%%   NVML: %.0f%% %.0f W %.0f
      C %.0f MHz",
                  m.gpu3dProcesso[GPU_NVIDIA], m.gpu3dProcesso[GPU_INTEL], m.nvUtil, m.nvPotenciaW, m.
     nvTempC, m.nvClockSM);
    linhas.push_back(b);
    linhas.push_back("Renderer: " + rendererGL);
    if (benchmark)
        linhas.push_back(benchmark->status());
    else
        linhas.push_back("1-6 config | l luz | t textura | +/- [ ] detalhe | * / orbes | v VBO | s vsync |
     m medir | c captura | r reset");

    const int alturaLinha = 17, margem = 8;
    const int altura = static_cast<int>(linhas.size()) * alturaLinha + 2 * margem;
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
```

```
    glOrtho(0, janelaL, 0, janelaA, -1, 1);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glDisable(GL_DEPTH_TEST);
    glDisable(GL_CULL_FACE);
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glColor4f(0.0f, 0.0f, 0.0f, 0.62f);
    glBegin(GL_QUADS);  // fundo do painel (não contado: é interface, não cena)
    glVertex2i(0, janelaA - altura);
    glVertex2i(janelaL, janelaA - altura);
    glVertex2i(janelaL, janelaA);
    glVertex2i(0, janelaA);
    glEnd();
    glDisable(GL_BLEND);
    for (size_t i = 0; i < linhas.size(); ++i) {
        if (i == 1) glColor3f(1.0f, 0.9f, 0.3f);  // linha do FPS em destaque
        else glColor3f(0.92f, 0.92f, 0.92f);
        texto(margem, janelaA - margem - alturaLinha * static_cast<int>(i + 1) + 4, linhas[i]);
    }
}

// ---------------------------------------------------------------------------
// Laço de quadros
// ---------------------------------------------------------------------------
static void finalizarAplicacao() {
    monitor.parar();
    glutLeaveMainLoop();
}

static void aplicarTomada(const Tomada& t) {
    cena.definirConfig(t.config);
    cena.definirDetalhe(t.T);
    cena.definirObjetos(t.N);
    cena.definirAramado(t.aramado);
}

static void display() {
    const double t0 = agora();
    if (!pausado && tomadas.empty()) tempoAnim += t0 - ultimoTempo;
    ultimoTempo = t0;

    const std::uint64_t ciclos0 = ciclosThread();

    // Tempo de GPU: recolhe (sem bloquear) os resultados já prontos das consultas de quadros
    // anteriores, na ordem em que foram emitidas. A GPU executa com alguns quadros de atraso
    // em relação à CPU; consultas muito recentes não são verificadas, porque perguntar pela
    // disponibilidade de uma consulta ainda não enviada força o driver a esvaziar (flush) a
    // fila de comandos, o que alteraria o próprio desempenho medido.
    DadosQuadro d;
    int consultaAtual = -1;
    if (temConsultaTempoGPU()) {
        while (!consultasPendentes.empty() && consultasPendentes.front().quadro + IDADE_MIN_CONSULTA <=
     numQuadro) {
            const int id = consultasPendentes.front().id;
            GLint disponivel = 0;
            glGetQueryObjectiv(consultas[id], GL_QUERY_RESULT_AVAILABLE_X, &disponivel);
            if (!disponivel) break;
            std::uint64_t ns = 0;
            glGetQueryObjectui64v(consultas[id], GL_QUERY_RESULT_X, &ns);
            d.gpuSomaMs += ns / 1.0e6;
            ++d.gpuAmostras;
            consultasPendentes.pop_front();
            consultasLivres.push_back(id);
        }
        if (!consultasLivres.empty()) {  // sem consulta livre, este quadro não é cronometrado
            consultaAtual = consultasLivres.back();
            consultasLivres.pop_back();
            glBeginQuery(GL_TIME_ELAPSED_X, consultas[consultaAtual]);
        }
    }

    cena.desenhar(tomadas.empty() ? tempoAnim : TEMPO_ANIM_CAPTURA, janelaL, janelaA);
```

```
    if (mostrarPainel) desenharPainel();

    if (consultaAtual >= 0) {
        glEndQuery(GL_TIME_ELAPSED_X);
        consultasPendentes.push_back({consultaAtual, numQuadro});
    }
    const double t1 = agora();  // fim da emissão dos comandos pela CPU

    if (!capturaPendente.empty()) {
        salvarCaptura(capturaPendente);
        capturaPendente.clear();
    }
    glutSwapBuffers();
    const double t2 = agora();
    ++numQuadro;

    d.submissaoMs = (t1 - t0) * 1000.0;
    d.swapMs = (t2 - t1) * 1000.0;
    d.cpuAtivaMs = ciclosPorMs > 0 ? (ciclosThread() - ciclos0) / ciclosPorMs : 0;
    quadrosBench.adicionar(t2, d);
    quadrosPainel.adicionar(t2, d);
    if (t2 - quadrosPainel.inicio() >= 0.5) {
        painel = quadrosPainel.resultado();
        quadrosPainel.reiniciar(t2);
    }

    if (benchmark) {
        if (!benchmark->atualizar(t2)) {
            benchmark.reset();
            if (sairAoTerminarBenchmark) finalizarAplicacao();
        }
    } else if (!tomadas.empty()) {
        // Modo --capturar: espera 1,5 s em cada tomada (FPS do painel estabilizado), pede a
        // gravação (feita no quadro seguinte, antes do SwapBuffers) e então passa à próxima.
        if (aguardandoGravacao && capturaPendente.empty()) {
            aguardandoGravacao = false;
            if (++idxTomada >= tomadas.size()) {
                finalizarAplicacao();
                return;
            }
            aplicarTomada(tomadas[idxTomada]);
            tTomada = t2;
        } else if (!aguardandoGravacao && t2 - tTomada >= 1.5) {
            capturaPendente = std::string(DIR_CAPTURAS) + "/" + tomadas[idxTomada].nome;
            aguardandoGravacao = true;
        }
    } else if (t2 - ultimoConsole >= 2.0) {
        ultimoConsole = t2;
        const Metricas m = monitor.ultima();
        std::printf("[%c] T=%d N=%d tri=%s | %.1f FPS (%.2f ms) | GPU %.2f ms | CPU %.1f%% (thread %.0f%%)
     | "
                    "GPU nv %.0f%% intel %.0f%%\n",
                    cena.estado().config, cena.estado().T, cena.estado().N, comMilhares(cena.
     triangulosPorQuadro()).c_str(),
                    painel.fps, painel.quadroMs, painel.gpuMs, m.cpuProcesso, m.cpuThreadRender,
                    m.gpu3dProcesso[GPU_NVIDIA], m.gpu3dProcesso[GPU_INTEL]);
    }
}

static void ocioso() { glutPostRedisplay(); }

static void redimensionar(int l, int a) {
    janelaL = std::max(l, 1);
    janelaA = std::max(a, 1);
    // Os experimentos exigem resolução constante: durante o benchmark a janela é restaurada.
    if ((benchmark || !tomadas.empty()) && (l != LARGURA || a != ALTURA)) glutReshapeWindow(LARGURA, ALTURA
     );
}

// ---------------------------------------------------------------------------
// Teclado
// ---------------------------------------------------------------------------
static void restaurar() {
```

```
    cena.definirConfig('A');
    cena.definirDetalhe(T_INICIAL);
    cena.definirObjetos(N_INICIAL);
    cena.definirVBO(true);
    cena.definirAramado(false);
    vsync = false;
    definirVSync(false);
    pausado = false;
    mostrarPainel = true;
    std::printf("Configuracao inicial restaurada.\n");
}

static void iniciarMedicaoManual() {
    const EstadoCena& e = cena.estado();
    OpcoesBenchmark op;
    op.rotulo = "manual";
    op.configs = std::string(1, e.config);
    op.usarVBO = e.usarVBO;
    op.variavel = 'T';
    op.valores = {e.T};
    op.fixoN = e.N;
    op.apenasPontoAtual = true;
    benchmark = std::make_unique<Benchmark>(cena, monitor, quadrosBench, op, rendererGL, descreverAmbiente
    ());
    benchmark->iniciar(agora());
}

static void teclado(unsigned char k, int, int) {
    const EstadoCena& e = cena.estado();
    if (k == 27 || k == 'q' || k == 'Q') {
        finalizarAplicacao();
        return;
    }
    if (benchmark) return;  // durante uma medição o estado da cena não pode mudar
    const int passo = e.T < 64 ? 4 : 16;
    switch (k) {
        case '1': case '2': case '3': case '4': case '5': case '6':
            cena.definirConfig(static_cast<char>('A' + (k - '1')));
            break;
        case 'l': case 'L': {
            const int luz = (static_cast<int>(iluminacaoDaConfig(e.config)) + 1) % 3;
            cena.definirConfig(configDe(static_cast<Iluminacao>(luz), texturaDaConfig(e.config)));
            break;
        }
        case 't': case 'T':
            cena.definirConfig(configDe(iluminacaoDaConfig(e.config), !texturaDaConfig(e.config)));
            break;
        case '+': case '=': cena.definirDetalhe(e.T + passo); break;
        case '-': case '_': cena.definirDetalhe(e.T - passo); break;
        case ']': cena.definirDetalhe(e.T + 1); break;
        case '[': cena.definirDetalhe(e.T - 1); break;
        case '*': cena.definirObjetos(e.N * 2); break;
        case '/': cena.definirObjetos(e.N / 2); break;
        case 'v': case 'V': cena.definirVBO(!e.usarVBO); break;
        case 's': case 'S':
            vsync = !vsync;
            std::printf("V-Sync: intervalo efetivo = %d\n", definirVSync(vsync));
            break;
        case 'w': case 'W': cena.definirAramado(!e.aramado); break;
        case 'p': case 'P': pausado = !pausado; break;
        case 'h': case 'H': mostrarPainel = !mostrarPainel; break;
        case 'c': case 'C': {
            char nome[128];
            std::snprintf(nome, sizeof(nome), "%s/captura_%c_T%d_N%d_%02d", DIR_CAPTURAS, e.config, e.T, e.
    N,
                          ++contadorCapturas);
            capturaPendente = nome;
            break;
        }
        case 'm': case 'M': iniciarMedicaoManual(); break;
        case 'r': case 'R': restaurar(); break;
        default: return;
    }
```

```
    const EstadoCena& n = cena.estado();
    std::printf("-> config %s | T=%d N=%d | %s triangulos/quadro | %s\n", descreverConfig(n.config).c_str()
     , n.T, n.N,
                comMilhares(cena.triangulosPorQuadro()).c_str(), n.usarVBO ? "VBO" : "imediato");
}

static void especial(int k, int, int) {
    if (k == GLUT_KEY_HOME && !benchmark) restaurar();
}

// ---------------------------------------------------------------------------
// Linha de comando
// ---------------------------------------------------------------------------
static std::vector<int> lerLista(const std::string& s) {
    std::vector<int> v;
    std::stringstream ss(s);
    std::string item;
    while (std::getline(ss, item, ',')) v.push_back(std::atoi(item.c_str()));
    std::sort(v.begin(), v.end());
    v.erase(std::unique(v.begin(), v.end()), v.end());
    return v;
}

int main(int argc, char** argv) {
#ifdef _WIN32
    SetProcessDPIAware();  // garante que 1280x720 sejam pixels reais (sem ampliação pelo Windows)
#endif
    calibrarCiclos();
    bool modoBenchmark = false, modoCaptura = false, modoInfo = false;
    OpcoesBenchmark op;
    for (int i = 1; i < argc; ++i) {
        const std::string a = argv[i];
        auto proximo = [&](void) -> std::string { return i + 1 < argc ? argv[++i] : ""; };
        if (a == "--benchmark") modoBenchmark = true;
        else if (a == "--capturar") modoCaptura = true;
        else if (a == "--info") modoInfo = true;
        else if (a == "--vsync") vsync = true;
        else if (a == "--rotulo") op.rotulo = proximo();
        else if (a == "--modo") op.usarVBO = proximo() != "imediato";
        else if (a == "--variavel") op.variavel = static_cast<char>(std::toupper(proximo()[0]));
        else if (a == "--valores") op.valores = lerLista(proximo());
        else if (a == "--configs") op.configs = proximo();
        else if (a == "--T") op.fixoT = std::atoi(proximo().c_str());
        else if (a == "--N") op.fixoN = std::atoi(proximo().c_str());
        else if (a == "--estab") op.estabilizacao = std::atof(proximo().c_str());
        else if (a == "--medir") op.medicao = std::atof(proximo().c_str());
        else if (a == "--reps") op.repeticoes = std::atoi(proximo().c_str());
        else if (a == "--parada") op.fpsParada = std::atof(proximo().c_str());
        else if (a == "--saida") op.pastaSaida = proximo();
        else {
            std::fprintf(stderr, "Opcao desconhecida: %s\n", a.c_str());
            return 1;
        }
    }
    if (op.valores.empty())
        op.valores = op.variavel == 'T'
                         ? std::vector<int>{16, 32, 64, 96, 128, 192, 256, 320, 384, 448, 512, 640, 768,
     896, 1024}
                         : std::vector<int>{64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768,
     65536};

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA | GLUT_DEPTH);
    glutInitWindowSize(LARGURA, ALTURA);
    glutInitWindowPosition(40, 40);
    glutCreateWindow("Atividade 3 - Desempenho de CPU e GPU com OpenGL (Jardim de Orbes)");
    glutSetOption(GLUT_ACTION_ON_WINDOW_CLOSE, GLUT_ACTION_GLUTMAINLOOP_RETURNS);

    std::string faltando;
    if (!carregarExtensoesGL(faltando)) {
        std::fprintf(stderr, "Funcoes OpenGL indisponiveis: %s\n", faltando.c_str());
        return 1;
    }
```

```
    vendorGL = reinterpret_cast<const char*>(glGetString(GL_VENDOR));
    rendererGL = reinterpret_cast<const char*>(glGetString(GL_RENDERER));
    versaoGL = reinterpret_cast<const char*>(glGetString(GL_VERSION));
    versaoGLSL = reinterpret_cast<const char*>(glGetString(GL_SHADING_LANGUAGE_VERSION_X));
    definirVSync(vsync);
    if (temConsultaTempoGPU()) {
        glGenQueries(NUM_CONSULTAS, consultas);
        for (int i = 0; i < NUM_CONSULTAS; ++i) consultasLivres.push_back(i);
    }

    std::string erro;
    if (!cena.inicializar(erro)) {
        std::fprintf(stderr, "Erro ao inicializar a cena: %s\n", erro.c_str());
        return 1;
    }
    cena.definirDetalhe(T_INICIAL);
    cena.definirObjetos(N_INICIAL);
    monitor.iniciar();
    ambiente = descreverAmbiente();
    std::printf("%s\n", ambiente.c_str());
    if (modoInfo) {
        monitor.parar();
        return 0;
    }

    glutDisplayFunc(display);
    glutIdleFunc(ocioso);
    glutReshapeFunc(redimensionar);
    glutKeyboardFunc(teclado);
    glutSpecialFunc(especial);

    if (modoBenchmark) {
        sairAoTerminarBenchmark = true;
        mostrarPainel = true;  // o painel faz parte do quadro em todos os experimentos
        benchmark = std::make_unique<Benchmark>(cena, monitor, quadrosBench, op, rendererGL, ambiente);
        benchmark->iniciar(agora());
    } else if (modoCaptura) {
        for (char c = 'A'; c <= 'F'; ++c) tomadas.push_back({std::string("cena_") + c, c, 48, 64, false});
        tomadas.push_back({"aramado_T8", 'A', 8, 64, true});
        tomadas.push_back({"aramado_T24", 'A', 24, 64, true});
        tomadas.push_back({"orbes_N1024_T16", 'E', 16, 1024, false});
        aplicarTomada(tomadas[0]);
        tTomada = agora() + 0.5;
    } else {
        std::printf("Modo interativo. Teclas: 1-6 config | l luz | t textura | +/- [ ] detalhe | * / orbes
     | "
                    "v VBO/imediato | s V-Sync | w arame | p pausa | h painel | c captura | m medir | r
     reset | ESC\n");
    }

    ultimoTempo = agora();
    quadrosPainel.reiniciar(ultimoTempo);
    quadrosBench.reiniciar(ultimoTempo);
    glutMainLoop();

    monitor.parar();
    if (temConsultaTempoGPU()) glDeleteQueries(NUM_CONSULTAS, consultas);
    cena.finalizar();
    return 0;
}
```

tools/graficos.py

```
"""
graficos.py
Atividade 3 - Gera os gráficos (resultados/graficos) e as tabelas LaTeX (relatorio/gerado)
a partir dos CSV produzidos pelo protocolo automático (resultados/dados).

Uso (na raiz do projeto):  python tools/graficos.py
Requer: matplotlib.
```

```
"""
import csv
import os
import sys

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter, LogLocator, NullFormatter

RAIZ = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DADOS = os.path.join(RAIZ, "resultados", "dados")
SAIDA_G = os.path.join(RAIZ, "resultados", "graficos")
SAIDA_T = os.path.join(RAIZ, "relatorio", "gerado")
LIMITE = 60.0

CONFIGS = "ABCDEF"
# Paleta categórica validada (ordem fixa) + codificação secundária: marcador e traço
COR = {"A": "#2a78d6", "B": "#eb6834", "C": "#1baf7a", "D": "#eda100", "E": "#e87ba4", "F": "#6250d6"}
MARCADOR = {"A": "o", "B": "s", "C": "^", "D": "o", "E": "s", "F": "^"}
TRACO = {"A": "-", "B": "-", "C": "-", "D": "--", "E": "--", "F": "--"}
NOME = {
    "A": "A: sem luz, sem textura", "B": "B: omni, sem textura", "C": "C: spot, sem textura",
    "D": "D: sem luz, com textura", "E": "E: omni, com textura", "F": "F: spot, com textura",
}
TINTA = "#0b0b0b"
TINTA2 = "#52514e"
GRADE = "#d9d8d4"
REF = "#e34948"  # linha de referência de 60 FPS

plt.rcParams.update({
    "font.family": "DejaVu Sans", "font.size": 9, "axes.titlesize": 10, "axes.labelsize": 9,
    "axes.edgecolor": TINTA2, "axes.labelcolor": TINTA, "xtick.color": TINTA2, "ytick.color": TINTA2,
    "axes.spines.top": False, "axes.spines.right": False, "legend.frameon": False,
    "legend.fontsize": 8, "lines.linewidth": 1.6, "lines.markersize": 4.5, "savefig.dpi": 200,
})


# ---------------------------------------------------------------------------
# Leitura
# ---------------------------------------------------------------------------
def ler(rotulo, sufixo="pontos"):
    caminho = os.path.join(DADOS, f"{rotulo}_{sufixo}.csv")
    if not os.path.exists(caminho):
        return []
    with open(caminho, encoding="utf-8") as f:
        linhas = list(csv.DictReader(f))
    for r in linhas:
        for k, v in r.items():
            try:
                r[k] = float(v) if any(c in v for c in ".e") else int(v)
            except (ValueError, TypeError):
                pass
    return linhas


def ler_combinado(rotulo):
    """Lê <rotulo>_pontos.csv e aplica arquivos de reposição <rotulo>_<CONFIGS>_pontos.csv:
    as configurações remedidas num arquivo de reposição substituem as do arquivo principal
    (usado quando uma execução foi interrompida e uma configuração precisou ser medida de novo)."""
    pontos = ler(rotulo)
    if not os.path.isdir(DADOS):
        return pontos
    for nome in sorted(os.listdir(DADOS)):
        prefixo = rotulo + "_"
        if not (nome.startswith(prefixo) and nome.endswith("_pontos.csv")):
            continue
        cfgs = nome[len(prefixo):-len("_pontos.csv")]
        if not cfgs or any(c not in CONFIGS for c in cfgs):
            continue
        repos = ler(rotulo + "_" + cfgs)
        pontos = [r for r in pontos if r["config"] not in cfgs] + repos
```

```
        print(f"{rotulo}: configuracoes {cfgs} substituidas por {nome}")
    return pontos


def serie(pontos, config):
    p = sorted((r for r in pontos if r["config"] == config), key=lambda r: r["triangulos"])
    return p


# ------------------------------------------------------------------------
# Formatação pt-BR
# ------------------------------------------------------------------------
def milhar(n):
    return f"{int(round(n)):,}".replace(",", ".")


def dec(x, casas=1):
    return f"{x:,.{casas}f}".replace(",", "X").replace(".", ",").replace("X", ".")


def mega(x, casas=2):
    return dec(x / 1e6, casas)


def fmt_eixo_tri(v, _):
    m = v / 1e6
    if m >= 1:
        return f"{m:g}".replace(".", ",")
    return f"{m:.2g}".replace(".", ",")


def fmt_eixo_num(v, _):
    return f"{v:g}".replace(".", ",")


def preparar_log(ax, xlabel="Triângulos por quadro (milhões, escala log)", ylabel="FPS médio (escala log)")
    :
    ax.set_xscale("log")
    ax.set_yscale("log")
    ax.xaxis.set_major_formatter(FuncFormatter(fmt_eixo_tri))
    ax.xaxis.set_minor_formatter(NullFormatter())
    ax.yaxis.set_major_locator(LogLocator(base=10, subs=(1, 2, 3, 5)))
    ax.yaxis.set_major_formatter(FuncFormatter(fmt_eixo_num))
    ax.yaxis.set_minor_formatter(NullFormatter())
    ax.grid(True, which="major", color=GRADE, linewidth=0.6)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)


def linha60(ax):
    ax.axhline(LIMITE, color=REF, linewidth=1.2, linestyle=(0, (4, 2)), zorder=1)
    ax.annotate("60 FPS", xy=(0.0, LIMITE), xycoords=("axes fraction", "data"), xytext=(4, 3),
                textcoords="offset points", ha="left", va="bottom", color=REF, fontsize=8)


def primeiro_abaixo(pts):
    abaixo = [r for r in pts if r["fps"] < LIMITE]
    return min(abaixo, key=lambda r: r["triangulos"]) if abaixo else None


def plotar_fps(ax, pontos, configs, marcar=True, rotular_limite=True, sufixo=""):
    for c in configs:
        p = serie(pontos, c)
        if not p:
            continue
        x = [r["triangulos"] for r in p]
        y = [r["fps"] for r in p]
        ax.plot(x, y, TRACO[c], color=COR[c], marker=MARCADOR[c], label=NOME[c] + sufixo, zorder=3,
                markeredgecolor="white", markeredgewidth=0.6)
        if marcar:
            b = primeiro_abaixo(p)
            if b:
                ax.plot([b["triangulos"]], [b["fps"]], marker="o", markersize=10, markerfacecolor="none",
```

```
                        markeredgecolor=COR[c], markeredgewidth=1.6, zorder=4)
                if rotular_limite:
                    ax.annotate(f"{c}: {mega(b['triangulos'])} M\n{dec(b['fps'])} FPS",
                                xy=(b["triangulos"], b["fps"]), xytext=(-10, -22), textcoords="offset
    points",
                                fontsize=7.5, color=TINTA, ha="right")


def salvar(fig, nome):
    os.makedirs(SAIDA_G, exist_ok=True)
    fig.tight_layout()
    fig.savefig(os.path.join(SAIDA_G, nome + ".pdf"), bbox_inches="tight")
    fig.savefig(os.path.join(SAIDA_G, nome + ".png"), bbox_inches="tight")
    plt.close(fig)
    print("grafico", nome)


# ---------------------------------------------------------------------------
# Gráficos
# ---------------------------------------------------------------------------
def graficos(e1, e2, e3, e4):
    nota = "Marcador vazado: primeiro ponto medido com FPS médio < 60."
    # 1) Configuração A
    fig, ax = plt.subplots(figsize=(6.4, 4.0))
    preparar_log(ax)
    plotar_fps(ax, e1, "A")
    linha60(ax)
    ax.set_title("FPS × triângulos por quadro -- configuração A (sem iluminação, sem textura)")
    ax.legend(loc="lower left", bbox_to_anchor=(0.0, 0.26), title=nota, title_fontsize=7.5)
    salvar(fig, "g1_fps_A")

    # 2) Iluminação: A, B, C
    fig, ax = plt.subplots(figsize=(6.4, 4.0))
    preparar_log(ax)
    plotar_fps(ax, e1, "ABC", rotular_limite=False)
    linha60(ax)
    ax.set_title("FPS × triângulos -- modos de iluminação (sem textura): A, B e C")
    ax.legend(loc="lower left", bbox_to_anchor=(0.0, 0.26), title=nota, title_fontsize=7.5)
    salvar(fig, "g2_fps_iluminacao")

    # 3) Textura: pares equivalentes (A×D, B×E, C×F)
    fig, axs = plt.subplots(1, 3, figsize=(10, 3.6), sharey=True)
    for ax, par, titulo in zip(axs, ("AD", "BE", "CF"), ("Sem iluminação", "Omnidirecional", "Spot")):
        preparar_log(ax, ylabel="FPS médio (escala log)" if par == "AD" else "")
        plotar_fps(ax, e1, par, rotular_limite=False)
        linha60(ax)
        ax.set_title(f"{titulo}: {par[0]} (sem) × {par[1]} (com textura)")
        ax.legend(loc="lower left", bbox_to_anchor=(0.0, 0.26))
    salvar(fig, "g3_fps_textura")

    # 4) Seis configurações + detalhe próximo ao limite (escala linear)
    fig, axs = plt.subplots(1, 2, figsize=(10, 4.0), gridspec_kw={"width_ratios": [1.25, 1]})
    preparar_log(axs[0])
    plotar_fps(axs[0], e1, CONFIGS, rotular_limite=False)
    linha60(axs[0])
    axs[0].set_title("Seis configurações (A a F)")
    axs[0].legend(loc="lower left", bbox_to_anchor=(0.0, 0.26), title=nota, title_fontsize=7.5)
    ax = axs[1]
    perto = [r for r in e1 if 40 <= r["fps"] <= 110]
    for c in CONFIGS:
        p = [r for r in serie(e1, c) if r in perto]
        if p:
            ax.plot([r["triangulos"] / 1e6 for r in p], [r["fps"] for r in p], TRACO[c], color=COR[c],
                    marker=MARCADOR[c], label=c, markeredgecolor="white", markeredgewidth=0.6)
            ax.errorbar([r["triangulos"] / 1e6 for r in p], [r["fps"] for r in p], yerr=[r["fps_dp"] for r
    in p],
                        fmt="none", ecolor=COR[c], elinewidth=0.8, capsize=2)
            b = primeiro_abaixo(serie(e1, c))
            if b:
                ax.plot([b["triangulos"] / 1e6], [b["fps"]], "o", markersize=10, markerfacecolor="none",
                        markeredgecolor=COR[c], markeredgewidth=1.6)
    linha60(ax)
    ax.grid(True, color=GRADE, linewidth=0.6)
```

```
    ax.xaxis.set_major_formatter(FuncFormatter(fmt_eixo_num))
    ax.set_xlabel("Triângulos por quadro (milhões)")
    ax.set_ylabel("FPS médio (barras: ±1 desvio-padrão)")
    ax.set_title("Detalhe da transição (40 a 110 FPS)")
    ax.legend(ncol=3, loc="upper right")
    salvar(fig, "g4_fps_seis_configuracoes")

    # 5) CPU × triângulos (VBO e modo imediato)
    fig, axs = plt.subplots(1, 2, figsize=(10, 3.8), sharey=True)
    for ax, dados, titulo in ((axs[0], e1, "E1: VBO (GPU dedicada)"), (axs[1], e2, "E2: modo imediato")):
        for c in "AF":
            p = serie(dados, c)
            if not p:
                continue
            x = [r["triangulos"] for r in p]
            ax.plot(x, [r["cpu_thread_max"] for r in p], TRACO[c], color=COR[c], marker="o",
                    label=f"{c}: thread mais ocupada (% de 1 núcleo)", markeredgecolor="white",
     markeredgewidth=0.6)
            ax.plot(x, [r["cpu_thread"] for r in p], TRACO[c], color=COR[c], marker="s", alpha=0.55,
                    label=f"{c}: thread principal (% de 1 núcleo)", markeredgecolor="white",
     markeredgewidth=0.6)
            ax.plot(x, [r["cpu_proc"] for r in p], TRACO[c], color=COR[c], marker="^", alpha=0.35,
                    label=f"{c}: processo (% da CPU inteira, 22 threads)", markeredgecolor="white",
     markeredgewidth=0.6)
        ax.set_xscale("log")
        ax.xaxis.set_major_formatter(FuncFormatter(fmt_eixo_tri))
        ax.xaxis.set_minor_formatter(NullFormatter())
        ax.set_ylim(0, 105)
        ax.grid(True, color=GRADE, linewidth=0.6)
        ax.set_xlabel("Triângulos por quadro (milhões, escala log)")
        ax.set_title(titulo)
    axs[0].set_ylabel("Utilização da CPU (%)")
    axs[1].legend(loc="center left", bbox_to_anchor=(1.0, 0.5), fontsize=7.5)
    salvar(fig, "g5_cpu")

    # 6) GPU × triângulos (utilização do motor 3D e NVML)
    fig, axs = plt.subplots(1, 2, figsize=(10, 3.8), sharey=True)
    ax = axs[0]
    for c in CONFIGS:
        p = serie(e1, c)
        if p:
            ax.plot([r["triangulos"] for r in p], [r["gpu_nv_proc"] for r in p], TRACO[c], color=COR[c],
                    marker=MARCADOR[c], label=c, markeredgecolor="white", markeredgewidth=0.6)
    p = serie(e1, "A")
    if p:
        ax.plot([r["triangulos"] for r in p], [r["gpu_intel_sis"] for r in p], ":", color=TINTA2,
                label="Intel (sistema), em A")
    ax.set_title("E1: GPU NVIDIA -- motor 3D usado pelo processo")
    ax = axs[1]
    for dados, nome, cor, mk in ((e2, "E2 imediato (NVIDIA)", COR["B"], "s"), (e3, "E3 nº de orbes (NVIDIA)
     ", COR["C"], "^"),
                                 (e4, "E4 integrada (Intel)", COR["F"], "o")):
        p = serie(dados, "A")
        if not p:
            continue
        chave = "gpu_intel_proc" if dados is e4 else "gpu_nv_proc"
        ax.plot([r["triangulos"] for r in p], [r[chave] for r in p], "-", color=cor, marker=mk, label=nome
     + ", config A",
                markeredgecolor="white", markeredgewidth=0.6)
    ax.set_title("Demais experimentos -- motor 3D da GPU que executa")
    for ax in axs:
        ax.set_xscale("log")
        ax.xaxis.set_major_formatter(FuncFormatter(fmt_eixo_tri))
        ax.xaxis.set_minor_formatter(NullFormatter())
        ax.set_ylim(0, 105)
        ax.grid(True, color=GRADE, linewidth=0.6)
        ax.set_xlabel("Triângulos por quadro (milhões, escala log)")
        ax.legend(loc="center right", fontsize=7.5, ncol=1)
    axs[0].set_ylabel("Utilização da GPU (%)")
    salvar(fig, "g6_gpu")

    # 7) Potência e clock da GPU NVIDIA (E1, config A e F) - dois gráficos, uma escala cada
    fig, axs = plt.subplots(1, 2, figsize=(10, 3.4))
```

```
for c in "AF":
    p = serie(e1, c)
    if not p:
        continue
    x = [r["triangulos"] for r in p]
    axs[0].plot(x, [r["nv_pot_w"] for r in p], TRACO[c], color=COR[c], marker=MARCADOR[c], label=NOME[c
],
                markeredgecolor="white", markeredgewidth=0.6)
    axs[1].plot(x, [r["nv_clk_sm"] for r in p], TRACO[c], color=COR[c], marker=MARCADOR[c], label=NOME[
c],
                markeredgecolor="white", markeredgewidth=0.6)
    lim = [r for r in p if "potencia" in str(r["motivos_reducao"])]
    if lim:
        axs[1].plot([r["triangulos"] for r in lim], [r["nv_clk_sm"] for r in lim], "x", color=TINTA,
markersize=6,
                    label="limite de potência ativo" if c == "A" else None)
axs[0].set_ylabel("Potência da GPU (W)")
axs[0].set_title("E1: potência média da GPU NVIDIA")
axs[1].set_ylabel("Clock dos multiprocessadores (MHz)")
axs[1].set_title("E1: clock médio da GPU NVIDIA")
for ax in axs:
    ax.set_xscale("log")
    ax.xaxis.set_major_formatter(FuncFormatter(fmt_eixo_tri))
    ax.xaxis.set_minor_formatter(NullFormatter())
    ax.grid(True, color=GRADE, linewidth=0.6)
    ax.set_xlabel("Triângulos por quadro (milhões, escala log)")
    ax.legend(fontsize=7.5)
salvar(fig, "g7_potencia_clock")

# 8) Decomposição do tempo por quadro (A) nos quatro experimentos
fig, axs = plt.subplots(2, 2, figsize=(10, 6.4))
for ax, dados, titulo, var in ((axs[0][0], e1, "E1: VBO, GPU dedicada", "T"), (axs[0][1], e2, "E2: modo
 imediato", "T"),
                               (axs[1][0], e3, "E3: nº de orbes (T = 8)", "N"), (axs[1][1], e4, "E4:
VBO, GPU integrada", "T")):
    p = serie(dados, "A")
    if not p:
        ax.set_visible(False)
        continue
    x = [r["triangulos"] for r in p]
    ax.plot(x, [r["quadro_ms"] for r in p], "-", color=TINTA, marker="o", label="tempo por quadro
(1000/FPS)")
    ax.plot(x, [r["gpu_ms"] for r in p], "--", color=COR["A"], marker="s", label="tempo de GPU (
GL_TIME_ELAPSED)")
    ax.plot(x, [r["cpu_ativa_ms"] for r in p], "--", color=COR["B"], marker="^",
            label="CPU ativa da thread principal")
    ax.plot(x, [r["submissao_ms"] for r in p], ":", color=COR["C"], marker="v", label="emissão dos
comandos (relógio)")
    ax.axhline(1000 / LIMITE, color=REF, linewidth=1.2, linestyle=(0, (4, 2)))
    ax.annotate("16,7 ms (60 FPS)", xy=(0.0, 1000 / LIMITE), xycoords=("axes fraction", "data"), xytext
=(4, 3),
                textcoords="offset points", color=REF, fontsize=7.5)
    ax.set_xscale("log")
    ax.set_yscale("log")
    ax.xaxis.set_major_formatter(FuncFormatter(fmt_eixo_tri))
    ax.xaxis.set_minor_formatter(NullFormatter())
    ax.yaxis.set_major_locator(LogLocator(base=10, subs=(1, 2, 5)))
    ax.yaxis.set_major_formatter(FuncFormatter(fmt_eixo_num))
    ax.yaxis.set_minor_formatter(NullFormatter())
    ax.grid(True, color=GRADE, linewidth=0.6)
    ax.set_title(titulo + " -- config A")
    ax.set_xlabel("Triângulos por quadro (milhões, escala log)")
    ax.set_ylabel("ms por quadro (escala log)")
axs[0][0].legend(fontsize=7.5, loc="lower right")
salvar(fig, "g8_tempos_quadro")

# 9) Comparação dos regimes: config A em E1..E4
fig, ax = plt.subplots(figsize=(8.4, 4.0))
preparar_log(ax)
for dados, nome, cor, mk in ((e1, "E1: VBO, RTX 4060", COR["A"], "o"), (e2, "E2: modo imediato, RTX
 4060", COR["B"], "s"),
                             (e3, "E3: nº de orbes (T = 8), RTX 4060", COR["C"], "^"),
                             (e4, "E4: VBO, Intel Arc integrada", COR["F"], "D")):
```

```
        p = serie(dados, "A")
        if not p:
            continue
        ax.plot([r["triangulos"] for r in p], [r["fps"] for r in p], "-", color=cor, marker=mk, label=nome,
                markeredgecolor="white", markeredgewidth=0.6)
        b = primeiro_abaixo(p)
        if b:
            ax.plot([b["triangulos"]], [b["fps"]], "o", markersize=10, markerfacecolor="none",
    markeredgecolor=cor,
                    markeredgewidth=1.6)
    linha60(ax)
    ax.set_title("Configuração A nos quatro experimentos")
    ax.legend(loc="upper left", bbox_to_anchor=(1.01, 1.0), title=nota.replace(" com", "\ncom"),
     title_fontsize=7.5, alignment="left")
    salvar(fig, "g9_regimes")

    # 10-12) FPS A..F nos experimentos complementares
    for dados, nome, titulo, xlab in ((e2, "g10_fps_imediato", "E2: modo imediato -- seis configurações",
     None),
                                      (e3, "g11_fps_objetos", "E3: aumento do número de orbes (T = 8) -- A
     e F", None),
                                      (e4, "g12_fps_integrada", "E4: GPU integrada (Intel Arc) -- seis
     configurações", None)):
        if not dados:
            continue
        fig, ax = plt.subplots(figsize=(8.4, 4.0))
        preparar_log(ax)
        plotar_fps(ax, dados, CONFIGS, rotular_limite=False)
        linha60(ax)
        ax.set_title(titulo)
        ax.legend(loc="upper left", bbox_to_anchor=(1.01, 1.0), title=nota.replace(" com", "\ncom"),
    title_fontsize=7.5, alignment="left")
        if dados is e3:
            sec = ax.secondary_xaxis("top", functions=(lambda t: (t - 17424) / 224, lambda n: n * 224 +
    17424))
            sec.set_xlabel("número de orbes N (= chamadas de desenho - 3)")
            sec.xaxis.set_major_formatter(FuncFormatter(lambda v, _: milhar(v)))
        salvar(fig, nome)


# ---------------------------------------------------------------------------
# Tabelas LaTeX
# ---------------------------------------------------------------------------
def gargalo(r):
    """Classificação a partir das medições (a discussão no relatório justifica o critério)."""
    if r is None:
        return "--"
    gpu = max(r["gpu_nv_proc"], r["gpu_intel_proc"])
    razao_gpu = r["gpu_ms"] / r["quadro_ms"] if r["quadro_ms"] > 0 else 0
    if gpu >= 90 and razao_gpu >= 0.85:
        txt = "GPU (processamento de vértices)"
        # os motivos de redução vêm do NVML: só valem quando é a NVIDIA que está desenhando
        if "potencia" in str(r["motivos_reducao"]) and r["gpu_nv_proc"] >= 90:
            txt += "; no limite de potência"
        return txt
    if r["cpu_thread_max"] >= 85 and gpu < 60:
        return "CPU: 1 thread saturada (envio de comandos)"
    return "indefinido"


def situacao(pts):
    b = primeiro_abaixo(pts)
    if not b:
        return "limite não alcançado", None, None
    acima = [r for r in pts if r["fps"] >= LIMITE and r["triangulos"] < b["triangulos"]]
    if not acima:
        return "abaixo de 60 já na menor carga", None, b
    return "limite encontrado", max(acima, key=lambda r: r["triangulos"]), b


def tabela_sintese(dados, rotulo, legenda, configs=CONFIGS, chave_gpu="gpu_nv_proc"):
    linhas = []
    for c in configs:
```

```
        pts = serie(dados, c)
        if not pts:
            continue
        sit, a, b = situacao(pts)
        maior = max(pts, key=lambda r: r["triangulos"])
        if b:
            gpu = b[chave_gpu]
            linhas.append(
                f"{c} & {milhar(a['triangulos']) if a else '--'} ({dec(a['fps']) if a else '--'}) & "
                f"\\textbf{{{milhar(b['triangulos'])}}} & \\textbf{{{dec(b['fps'])}}} $\\pm$ {dec(b['fps_dp
    '], 2)} & "
                f"{dec(b['cpu_proc'])} / {dec(b['cpu_thread_max'], 0)} & {dec(gpu, 0)} & "
                f"{gargalo(b)} & {sit} \\\\")
        else:
            linhas.append(f"{c} & -- & -- (maior: {milhar(maior['triangulos'])}) & {dec(maior['fps'])} & "
                          f"{dec(maior['cpu_proc'])} / {dec(maior['cpu_thread_max'], 0)} & {dec(maior[
    chave_gpu], 0)} & "
                          f"{gargalo(maior)} & {sit} \\\\")
    corpo = "\n".join(linhas)
    tex = f"""\\begin{{table}}[H]
\\centering
\\caption{{{legenda}}}
\\label{{tab:sintese_{rotulo}}}
\\scriptsize
\\setlength{{\\tabcolsep}}{{3pt}}
\\begin{{tabularx}}{{\\textwidth}}{{c r r r c c L P{{1.6cm}}}}
\\toprule
Cfg. & Último $\\geq$ 60 & \\textbf{{Menor testado}} & FPS médio & CPU proc. / & GPU 3D & Gargalo & Situaçã
    o \\\\
 & (FPS) & \\textbf{{$<$ 60}} & $\\pm$ dp & thread máx. (\\%) & (\\%) & (indício mais forte) & \\\\
\\midrule
{corpo}
\\bottomrule
\\end{{tabularx}}
\\end{{table}}
"""
    escrever(f"sintese_{rotulo}.tex", tex)


def tabela_pontos(dados, rotulo, legenda, var="T", chave_gpu="gpu_nv_proc", nv=True):
    linhas = []
    for c in CONFIGS:
        pts = sorted(serie(dados, c), key=lambda r: r["triangulos"])
        if not pts:
            continue
        b = primeiro_abaixo(pts)
        for r in pts:
            marca = "$\\blacktriangleright$" if r is b else ""
            v = r["T"] if var == "T" else r["objetos"]
            ref = "r" if r["etapa"] == "refino" else ""
            extra = f"{dec(r['nv_pot_w'], 0)} & {milhar(r['nv_clk_sm'])}" if nv else "-- & --"
            linhas.append(
                f"{marca}{c} & {milhar(v)}{ref} & {milhar(r['triangulos'])} & {dec(r['fps'])} & {dec(r['
    fps_dp'], 2)} & "
                f"{dec(r['quadro_ms'], 2)} & {dec(r['gpu_ms'], 2)} & {dec(r['cpu_ativa_ms'], 2)} & "
                f"{dec(r['cpu_proc'])} & {dec(r['cpu_thread_max'], 0)} & {dec(r[chave_gpu], 0)} & {extra}
    \\\\")
        linhas.append("\\midrule")
    if linhas and linhas[-1] == "\\midrule":
        linhas.pop()
    corpo = "\n".join(linhas)
    rot_var = "T" if var == "T" else "N"
    cab = (f"Cfg. & {rot_var} & Triângulos & FPS & dp & Quadro & GPU & CPU ativa & CPU proc. & Thread máx.
    & GPU 3D & Pot. & Clock \\\\\n"
           " & & & & & (ms) & (ms) & (ms) & (\\%) & (\\%) & (\\%) & (W) & (MHz) \\\\")
    tex = f"""{{\\scriptsize
\\setlength{{\\tabcolsep}}{{2.6pt}}
\\begin{{longtable}}{{l r r r r r r r r r r r}}
\\caption{{{legenda}}}\\label{{tab:pontos_{rotulo}}}\\\\
\\toprule
{cab}
\\midrule
\\endfirsthead
```

```
\\toprule
{cab}
\\midrule
\\endhead
{corpo}
\\bottomrule
\\end{{longtable}}
}}
"""
    escrever(f"pontos_{rotulo}.tex", tex)


def macros(e1, e2, e3, e4):
    """Valores usados no texto do relatório (\\newcommand), para não digitar números à mão."""
    nomes = {"e1": "Vbo", "e2": "Imed", "e3": "Obj", "e4": "Int"}
    out = ["% Gerado por tools/graficos.py -- nao editar"]
    for chave, dados in (("e1", e1), ("e2", e2), ("e3", e3), ("e4", e4)):
        for c in CONFIGS:
            pts = serie(dados, c)
            if not pts:
                continue
            sit, a, b = situacao(pts)
            n = nomes[chave] + c
            if b:
                out.append(f"\\newcommand{{\\lim{n}}}{{{milhar(b['triangulos'])}}}")
                out.append(f"\\newcommand{{\\limM{n}}}{{{mega(b['triangulos'], 1)}}}")
                out.append(f"\\newcommand{{\\fps{n}}}{{{dec(b['fps'])}}}")
                out.append(f"\\newcommand{{\\gpu{n}}}{{{dec(b['gpu_nv_proc'] if chave != 'e4' else b['
    gpu_intel_proc'], 0)}}}")
                out.append(f"\\newcommand{{\\thr{n}}}{{{dec(b['cpu_thread_max'], 0)}}}")
                out.append(f"\\newcommand{{\\cpu{n}}}{{{dec(b['cpu_proc'])}}}")
            if a:
                out.append(f"\\newcommand{{\\acima{n}}}{{{milhar(a['triangulos'])}}}")
                out.append(f"\\newcommand{{\\fpsAcima{n}}}{{{dec(a['fps'])}}}")
            out.append(f"\\newcommand{{\\pontos{n}}}{{{len(pts)}}}")
            menor = min(pts, key=lambda r: r["triangulos"])
            out.append(f"\\newcommand{{\\fpsMenor{n}}}{{{dec(menor['fps'], 0)}}}")
    escrever("valores.tex", "\n".join(out) + "\n")


def escrever(nome, texto):
    os.makedirs(SAIDA_T, exist_ok=True)
    with open(os.path.join(SAIDA_T, nome), "w", encoding="utf-8") as f:
        f.write(texto)
    print("tabela", nome)


def main():
    e1, e2, e3, e4 = (ler_combinado(r) for r in ("e1_vbo_T", "e2_imediato_T", "e3_vbo_N", "e4_integrada_T")
    )
    if not e1:
        sys.exit("Sem dados em resultados/dados (execute executar_experimentos.bat).")
    graficos(e1, e2, e3, e4)
    tabela_sintese(e1, "e1", "Tabela-síntese do experimento principal E1 (GPU dedicada, VBO, aumento de T):
     menor "
                   "quantidade testada de triângulos com FPS médio abaixo de 60 em cada configuração.")
    tabela_sintese(e2, "e2", "Tabela-síntese do experimento E2 (modo imediato, GPU dedicada).")
    tabela_sintese(e3, "e3", "Tabela-síntese do experimento E3 (aumento do número de orbes, T = 8).",
    configs="AF")
    tabela_sintese(e4, "e4", "Tabela-síntese do experimento E4 (GPU integrada Intel Arc, VBO, aumento de T)
    .",
                   chave_gpu="gpu_intel_proc")
    tabela_pontos(e1, "e1", "Todos os pontos medidos no experimento E1 (médias de 3 repetições de 10 s). "
                  "``r'' = ponto de refino; $\\blacktriangleright$ = primeiro ponto com FPS médio $<$ 60.")
    tabela_pontos(e2, "e2", "Pontos medidos no experimento E2 (modo imediato).")
    tabela_pontos(e3, "e3", "Pontos medidos no experimento E3 (N orbes com T = 8).", var="N")
    tabela_pontos(e4, "e4", "Pontos medidos no experimento E4 (GPU integrada). Potência e clock da NVML "
                  "não se aplicam.", chave_gpu="gpu_intel_proc", nv=False)
    macros(e1, e2, e3, e4)


if __name__ == "__main__":
    main()
```